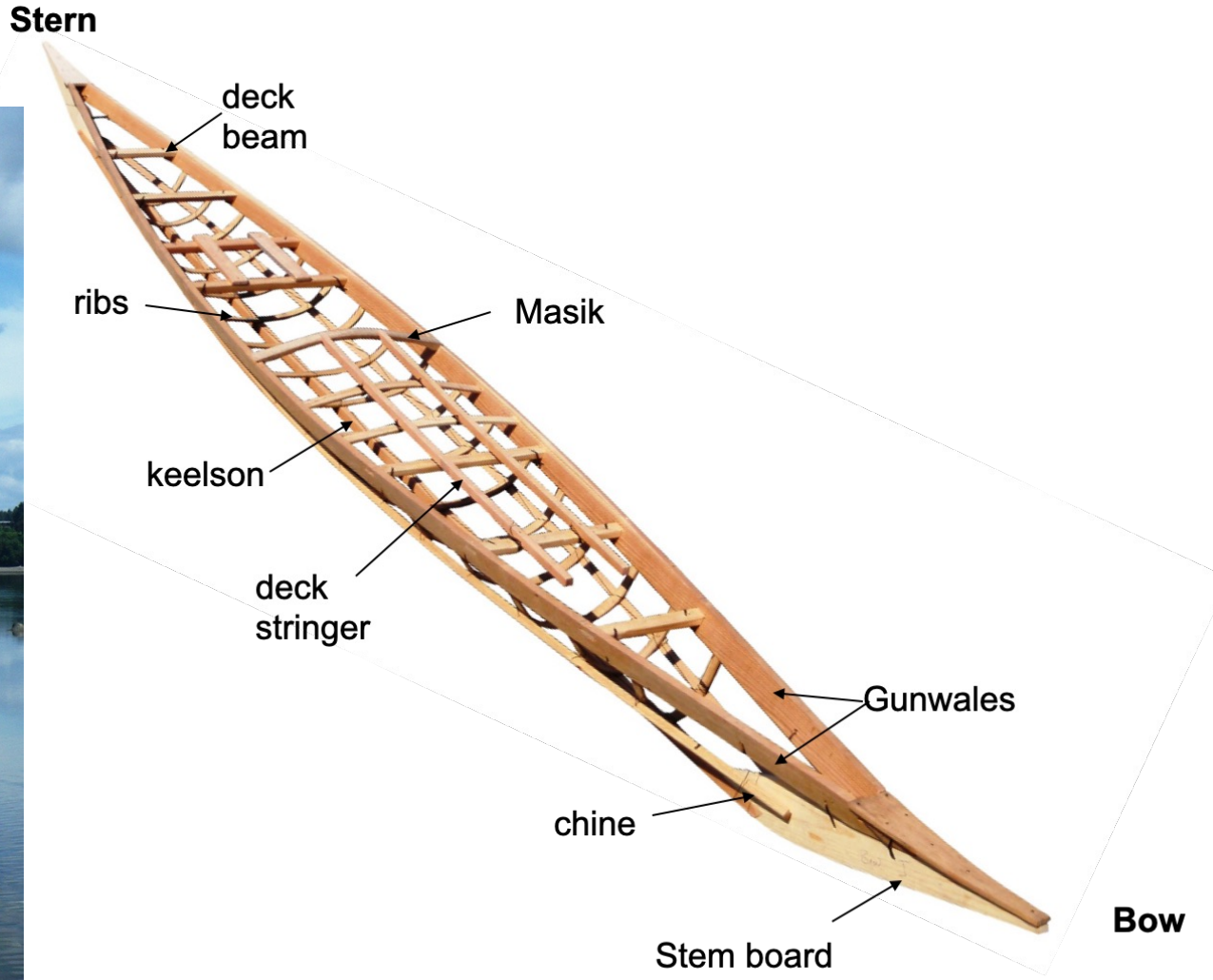
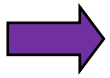
OpenMP and friends: Beyond Parallel for and SMP*





*SMP: Symmetric MultiProcessor

Outline



- Irregular Parallelism and tasks
- Other Systems for task-level parallelism: TBB
- Non-Uniform Memory Architectures (NUMA)
- Programming your GPU with OpenMP
- Recap

Irregular Parallelism

- Let's call a problem "irregular" when one or both of the following hold:
  - Data Structures are sparse or involve indirect memory references
  - Control structures are not basic for-loops

- Example: Traversing Linked lists:

```
p = listhead ;
while (p) {
  process(p);
  p=p->next;
}
```

- Using what we've learned so far, traversing a linked list in parallel using OpenMP is difficult.

Exercise: Traversing linked lists

- Consider the program linked.c
  - Traverses a linked list computing a sequence of Fibonacci numbers at each node.
- Parallelize this program selecting from the following list of constructs:

```
#pragma omp parallel
#pragma omp for
#pragma omp parallel for
#pragma omp for reduction(op:list)
#pragma omp critical
int omp_get_num_threads();
int omp_get_thread_num();
double omp_get_wtime();
schedule(static[,chunk]) or schedule(dynamic[,chunk])
private(), firstprivate(), default(none)
```

- Hint: Just worry about the while loop that is timed inside main(). You don't need to make any changes to the "list functions"

Linked Lists with OpenMP: My solution

- See the file solutions/linked_notasks.c

```
while (p != NULL) {
     p = p->next;
      count++;
}
struct node *parr = (struct node*) malloc(count*sizeof(struct node));
p = head;
for(i=0; i<count; i++) {
      parr[i] = p;
      p = p->next;
   }
#pragma omp parallel
{
    #pragma omp for schedule(static,1)
    for(i=0; i<count; i++)
       processwork(parr[i]);
}
```

Count number of items in the linked list

Copy pointer to each node into an array

Process nodes in parallel with a for loop

Linked Lists with OpenMP (without tasks)

- See the file solutions/linked_notasks.c

```
while (p != NULL) {
     p = p->next;
      count++;
}
struct node *parr = (struct node*) malloc(count*sizeof(struct node));
p = head;
for(i=0; i<count; i++) {
      parr[i] = p;
      p = p->next;
   }
#pragma omp parallel
{
    #pragma omp for schedule(static,1)
    for(i=0; i<count; i++)
       processwork(parr[i]);
}
```

Count number of items in the linked list

Copy pointer to each node into an array

Process nodes in parallel with a for loop

With so much code to add and three passes through the data, this is really ugly.

There has got to be a better way to do this

Number of threads	Schedule	
	Default	Static,1
1	48 seconds	45 seconds
2	39 seconds	28 seconds

Results on an Intel dual core 1.83 GHz CPU, Intel IA-32 compiler 10.1 build 2

Solutions from other people

- A much more elegant solution based on the SPMD pattern

```
#pragma omp parallel firstprivate(p)
 {
    int id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    int count = 0;
    while(p!=NULL && count < id){
       count ++;
       p = p->next;
    }
    while (p != NULL) {
       processwork(p);
       int count = 0;
       while(p!=NULL && count < nthreads){
          count ++;
          p = p->next;
       }
    }
 }
```

Each thread has its own copy of the list pointer set to the head of the list

Each thread advances its list pointer by a number equal to the thread ID

Increment the thread's list pointer by the number of threads

Solutions from other people

- A particularly elegant, SPMD solution

```
#pragma omp parallel firstprivate(p)
  {
     int i = 0;
     int thread_num = omp_get_thread_num();
     int num_threads = omp_get_num_threads();
     while (p != NULL)
     {
           if(i % num_threads == thread_num)
                 processwork(p);
           p = p->next;
           ++i;
     }
  }
```

Each thread has its own copy of the list pointer set to the head of the list

Each of the threads traverses the list, but only one thread processes the work for any give node using the modulus of the count with the number of threads

Linked Lists with OpenMP

```
while (p != NULL) {
      p = p->next;
      count++;
}
struct node *parr = (struct node*) malloc(count*sizeof(struct
    node));
p = head;
for(i=0; i<count; i++) {
      parr[i] = p;
      p = p->next;
  }
#pragma omp parallel
{
    #pragma omp for schedule(static,1)
    for(i=0; i<count; i++)
       processwork(parr[i]);
}
```

```
#pragma omp parallel firstprivate(p) default(none)
  {
     int i = 0;
     int thread_num = omp_get_thread_num();
     int num_threads = omp_get_num_threads();
     while (p != NULL)
     {
           if(i % num_threads == thread_num)
              processwork(p);
           p = p->next;
           ++i;
     }
  }
```

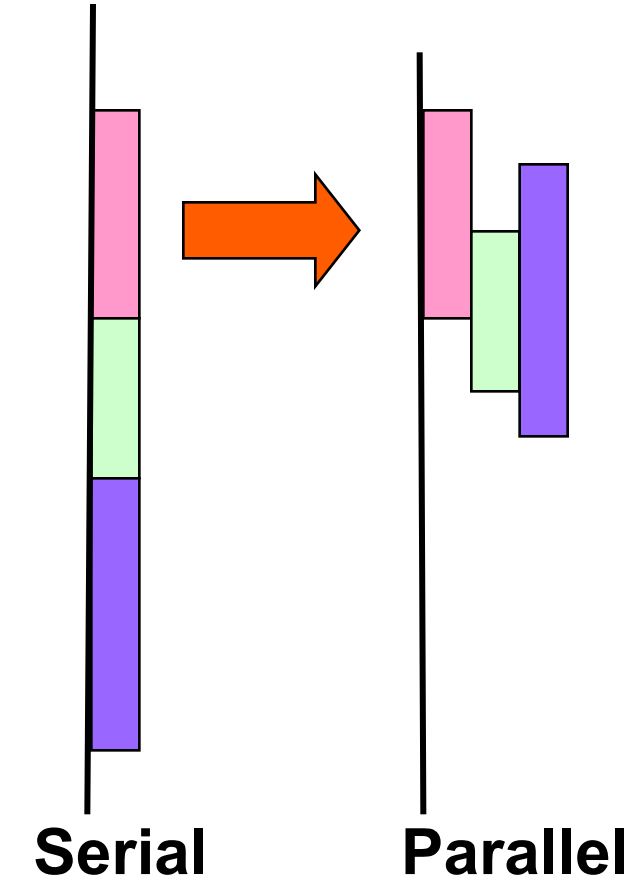
```
#pragma omp parallel firstprivate(p)
 {
    int id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    int count = 0;
    while(p!=NULL && count < id){
      count ++;
      p = p->next;
    }
    while (p != NULL) {
      processwork(p);
      int count = 0;
      while(p!=NULL && count < nthreads){
        count ++;
        p = p->next;
      }
    }
 }
```

With so much code to add and multiple passes through the data, all of these approaches are kind of ugly.

There has got to be a better way to handle irregular problems

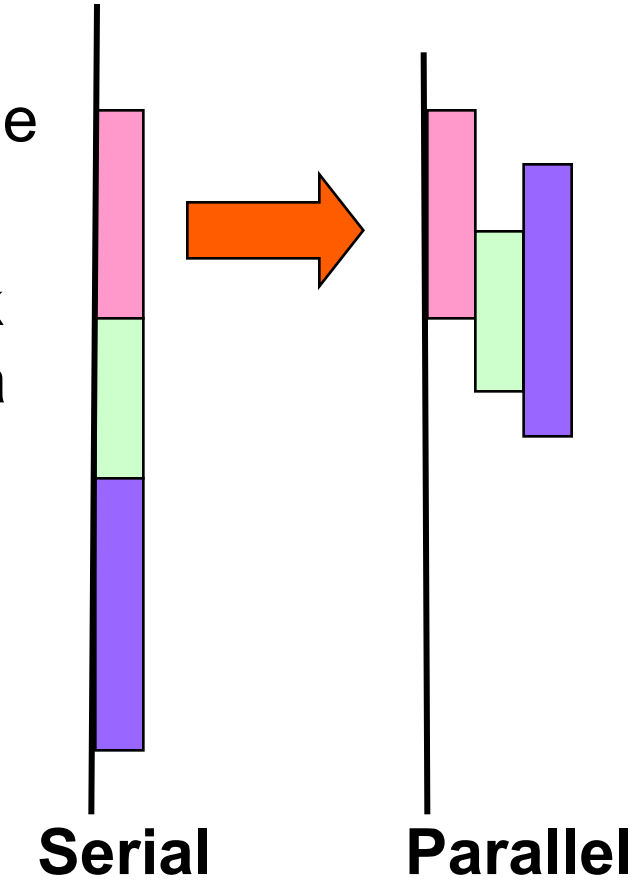
What are Tasks?

- Tasks are independent units of work
- Tasks are composed of:
  - code to execute
  - data to compute with
- Threads are assigned to perform the work of each task.
  - The thread that encounters the task construct may execute the task immediately.
  - The threads may defer execution until later



What are Tasks?

- The task construct includes a structured block of code
- Inside a parallel region, a thread encountering a task construct will package up the code block and its data for execution
- Tasks can be nested: i.e., a task may itself generate tasks.



A common Pattern is to have one thread create the tasks while the other threads wait at a barrier and execute the tasks

Single Worksharing Construct

- The **single** construct denotes a block of code that is executed by only one thread (not necessarily the primary* thread).
- A barrier is implied at the end of the single block (can remove the barrier with a *nowait* clause).

```
#pragma omp parallel
{
        do_many_things();
    #pragma omp single
        {    exchange_boundaries();   }
        do_many_other_things();

}
```

*This used to be called the “master thread”. The term “master” has been deprecated in OpenMP 5.1 and replaced with the term “primary”.

Task Directive

`#pragma omp task` *[clauses]*

structured-block

```
#pragma omp parallel
{
  #pragma omp single
   {
     #pragma omp task
        fred();
     #pragma omp task
        daisy();
     #pragma omp task
        billy();
   }
}
```

Create some threads

One Thread packages tasks

Tasks executed by some thread in some order

All tasks complete before this barrier is released

Exercise: Simple tasks

- Write a program using tasks that will “randomly” generate one of two strings:
  - “I think “ “race” “car” “s are fun”
  - “I think “ “car” “race” “s are fun”

- Hint: use tasks to print the indeterminate part of the output (i.e. the “race” or “car” parts).

- This is called a “Race Condition”. It occurs when the result of a program depends on how the OS schedules the threads.

- NOTE: A “data race” is when threads “race to update a shared variable”. They produce race conditions. Programs containing data races are undefined (in OpenMP but also ANSI standards C++’11 and beyond).

```
#pragma omp parallel
#pragma omp task
#pragma omp single
```

This exercise comes from Ruud van der Pas of Oracle

Racey Cars: Solution

```
#include <stdio.h>
#include <omp.h>
int main()
{  printf("I think");
   #pragma omp parallel
   {
      #pragma omp single
      {
         #pragma omp task
            printf(" car");
         #pragma omp task
            printf(" race");
      }
   }
   printf("s");
   printf(" are fun!\n");
}
```

Data Scoping with Tasks

- Variables can be shared, private or firstprivate with respect to task

- These concepts are a little bit different compared with threads:
  - If a variable is shared on a task construct, the references to it inside the construct are to the storage with that name at the point where the task was encountered
  - If a variable is private on a task construct, the references to it inside the construct are to new uninitialized storage that is created when the task is executed
  - If a variable is firstprivate on a construct, the references to it inside the construct are to new storage that is created and initialized with the value of the existing storage of that name when the task is encountered

Data Scoping Defaults

- The behavior you want for tasks is usually firstprivate, because the task may not be executed until later (and variables may have gone out of scope)
  - Variables that are private when the task construct is encountered are firstprivate by default

- Variables that are shared in all constructs starting from the innermost enclosing parallel construct are shared by default

```
#pragma omp parallel shared(A) private(B)
{
    ...
#pragma omp task
    {
        int C;
        compute(A, B, C);
    }
}
```

A is shared
B is firstprivate
C is private

Exercise: Traversing linked lists

- Consider the program linked.c
  - Traverses a linked list computing a sequence of Fibonacci numbers at each node.
- Parallelize this program selecting from the following list of constructs:

```
#pragma omp parallel
#pragma omp single
#pragma omp task
int omp_get_num_threads();
int omp_get_thread_num();
double omp_get_wtime();
private(), firstprivate()
```

- Hint: Just worry about the contents of main(). You don't need to make any changes to the "list functions"

Parallel Linked List Traversal

```
#pragma omp parallel
{
  #pragma omp single
   {
    p = listhead ;
    while (p) {
       #pragma omp task firstprivate(p)
              {
                process (p);
              }
       p=next (p) ;
     }
   }
}
```

Only one thread packages tasks

makes a copy of `p` when the task is packaged

When/Where are Tasks Complete?

- At thread barriers (explicit or implicit)
  - all tasks generated inside a region must complete at the next barrier encountered by the threads in that region. Common examples:
    - **Tasks generated inside a single construct**: all tasks complete before exiting the barrier on the single.
    - **Tasks generated inside a parallel region**: all tasks complete before exiting the barrier at the end of the parallel region.

- At taskwait directive
  - i.e. Wait until all tasks defined in the current task have completed.

```
#pragma omp taskwait
```

  - Note: applies only to tasks generated in the current task, not to “descendants” .

Example

```
#pragma omp parallel
{
  #pragma omp single
   {
     #pragma omp task
        fred();
     #pragma omp task
        daisy();
     #pragma omp taskwait
     #pragma omp task
        billy();
   }
}
```

`fred()` and `daisy()` must complete before `billy()` starts, but this does not include tasks created inside **fred()** and **daisy()**

`All tasks` including those created inside **fred()** and **daisy()** must complete before exiting this barrier

Example

```
#pragma omp parallel
{
  #pragma omp single nowait
   {
     #pragma omp task
        fred();
     #pragma omp task
        daisy();
     #pragma omp taskwait
     #pragma omp task
        billy();
   }
}
```

The barrier at the end of the single is expensive and not needed since you get the barrier at the end of the parallel region. So use nowait to turn it off.

`All tasks` including those created inside **fred()** and **daisy()** must complete before exiting this barrier

Example: Fibonacci numbers

```
int fib (int n)
{
   int x,y;
   if (n < 2) return n;

   x = fib(n-1);
   y = fib (n-2);
   return (x+y);
}

int main()
{
   int NW = 5000;
   fib(NW);
}
```

- $F_n = F_{n-1} + F_{n-2}$
- Inefficient $O(2^n)$ recursive implementation!

Parallel Fibonacci

```
int fib (int n)
{   int x,y;
   if (n < 2) return n;

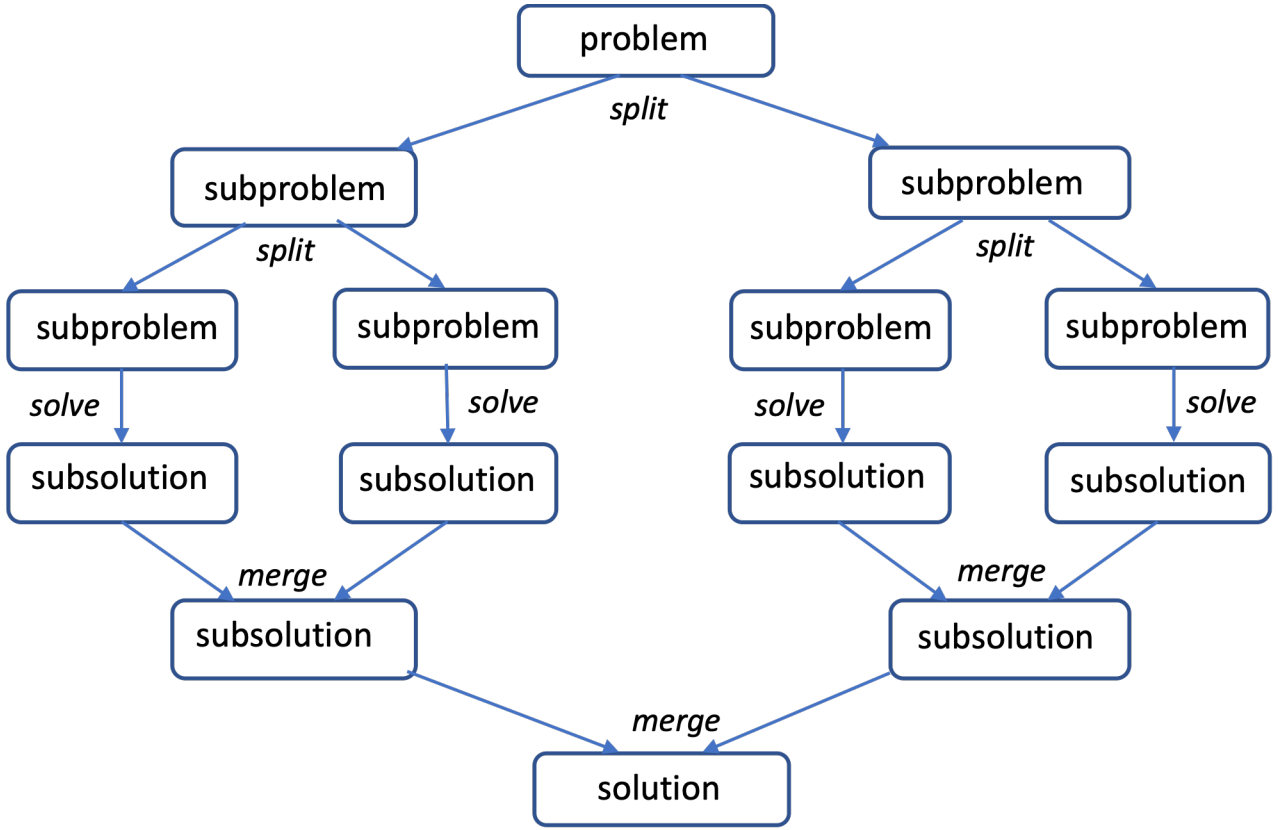
#pragma omp task shared(x)
   x = fib(n-1);
#pragma omp task shared(y)
   y = fib (n-2);
#pragma omp taskwait
   return (x+y);
}

Int main()
{  int NW = 5000;
   #pragma omp parallel
   {
      #pragma omp single
           fib(NW);
   }
}
```

- Binary tree of tasks
- Traversed using a recursive function
- A task cannot complete until all tasks below it in the tree are complete (enforced with taskwait)
- `x,y` are local, and so by default they are private to current task
  - must be shared on child tasks so they don't create their own firstprivate copies at this level!

Divide and Conquer

- Split the problem into smaller sub-problems; continue until the sub-problems can be solved directly



- 3 Options for parallelism:
  - Do work as you split into sub-problems
  - Do work only at the leaves
  - Do work as you recombine

Exercise: PI with tasks

- Go back to the original pi.c program
  - Parallelize this program using OpenMP tasks

```
#pragma omp parallel
#pragma omp task
#pragma omp taskwait
#pragma omp single
double omp_get_wtime()
int omp_get_thread_num();
int omp_get_num_threads();
```

- Hint: first create a recursive pi program and verify that it works. **Think about the computation you want to do at the leaves. If you go all the way down to one iteration per leaf-node, won't you just swamp the system with tasks?**

Program: OpenMP tasks

```
include <omp.h>
static long num_steps = 100000000;
#define MIN_BLK  10000000
double pi_comp(int Nstart,int Nfinish,double step)
{   int i,iblk;
    double x, sum = 0.0,sum1, sum2;
    if (Nfinish-Nstart < MIN_BLK){
      for (i=Nstart;i< Nfinish; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
      }
    }
    else{
      iblk = Nfinish-Nstart;
      #pragma omp task shared(sum1)
          sum1 = pi_comp(Nstart,        Nfinish-iblk/2,step);
      #pragma omp task shared(sum2)
          sum2 = pi_comp(Nfinish-iblk/2, Nfinish,      step);
      #pragma omp taskwait
        sum = sum1 + sum2;
   }return sum;
}
```

```
int main ()
{
  int i;
  double step, pi, sum;
   step = 1.0/(double) num_steps;
   #pragma omp parallel
   {
      #pragma omp single
          sum =
              pi_comp(0,num_steps,step);
    }
     pi = step * sum;
}
```

Results*: Pi with tasks

threads	1st SPMD	SPMD critical	PI Loop	Pi tasks
1	1.86	1.87	1.91	1.87
2	1.03	1.00	1.02	1.00
3	1.08	0.68	0.80	0.76
4	0.97	0.53	0.68	0.52

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Using Tasks

- Don’t use tasks for things already well supported by OpenMP
  - e.g. standard do/for loops
  - the overhead of using tasks is greater

- Don’t expect miracles from the runtime
  - best results usually obtained where the user controls the number and granularity of tasks

Outline



- Irregular Parallelism and tasks
- Other Systems for task-level parallelism: TBB
- Non-Uniform Memory Architectures (NUMA)
- Programming your GPU with OpenMP
- Recap

OpenMP The Good, the Bad, and the Ugly

- The Good:
  - Directives are easy to use and support incremental parallelism.
  - OpenMP exposes threads … so you ***can*** optimize how threads map to data and cores

- The Bad:
  - Exposes threads … so you ***must*** reason about threads to understand how a program executes.

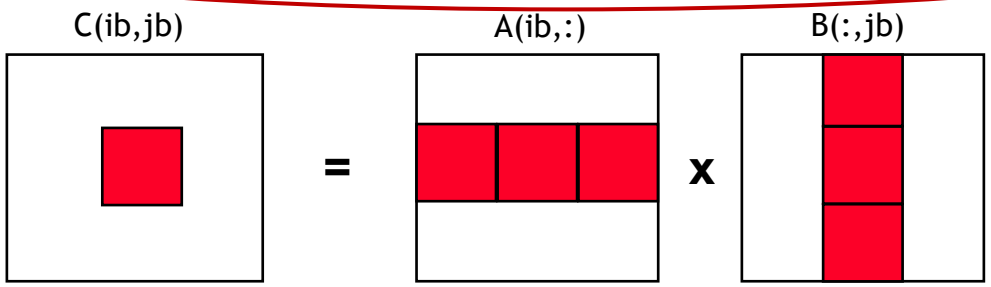
- The Ugly
  - Parallel Composition is broken since parallelism is expressed in terms of OS-managed threads.

What is parallel composition?

Matrix multiplication: blocked with level 3 BLAS

This is how you do a really fast dense matrix multiplication for three order N, square matrices … by processing over blocks, you can choose the block size to optimize use of the cache hierarchy

```
void mat_mul(int N, float *A, float *B, float *C)
{
 int NB=N/block_size; // assume N%block_size=0
 #pragma omp parallel for collapse(2)
 for (int ib = 0; ib < NB; ib++)
   for (int jb = 0; jb < NB; jb++)
     for (int kb = 0; kb < NB; kb++)
       sgemm(C, A, B, …)    // C_ib,jb = A_ib,kb * B_kb,jb
}
```



What happens if SGEMM uses OpenMP multithreading?

```
#pragma omp parallel for collapse(2)
for (i = ib*NB; i < (ib+1)*NB; i++)
 for (j = jb*NB; j < (jb+1)*NB; j++)
  for (k = kb*NB; k < (kb+1)*NB; k++)
    C[i*N+j] += A[i*N+k] * B[k*N+j];
```

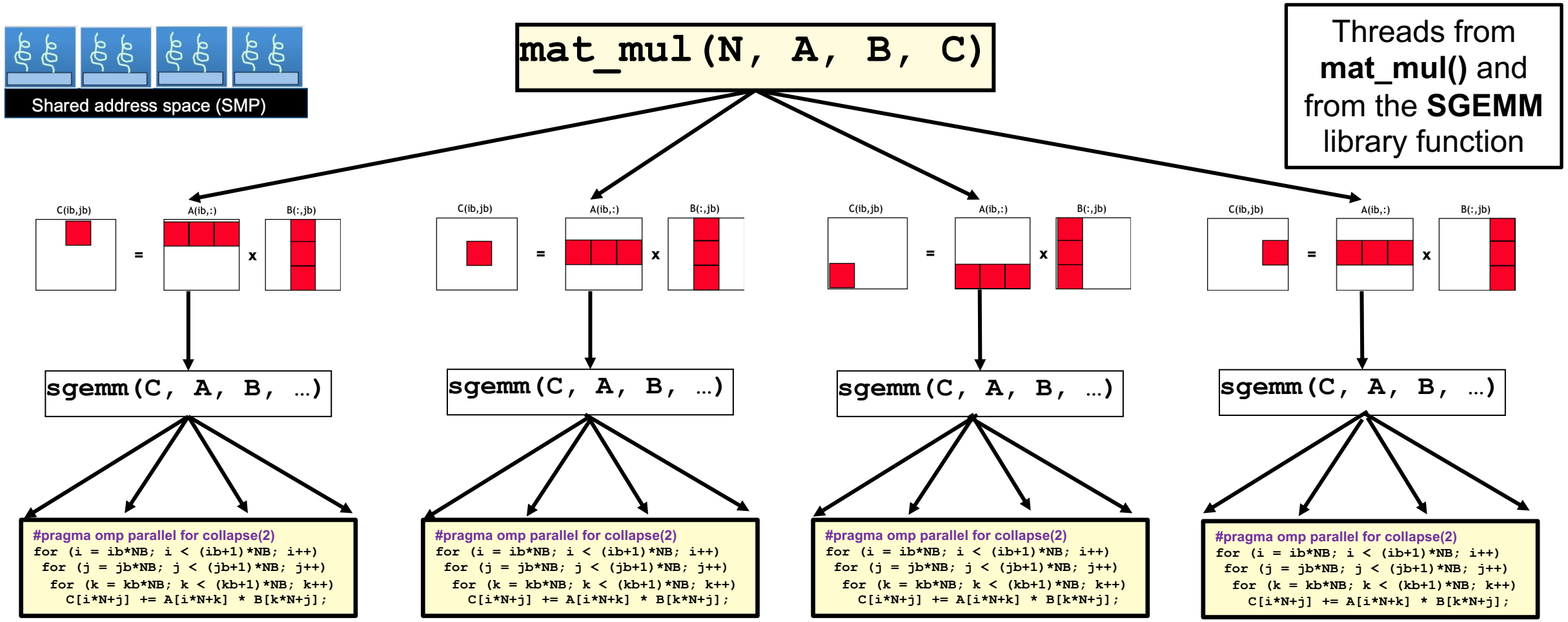
BLAS: Basic Linear Algebra Subprograms

sgemm is the name of the level three BLAS routine to multiply two matrices

SGEMM: Single Precision General Matrix Multiply

Matrix multiplication: blocked with level 3 BLAS

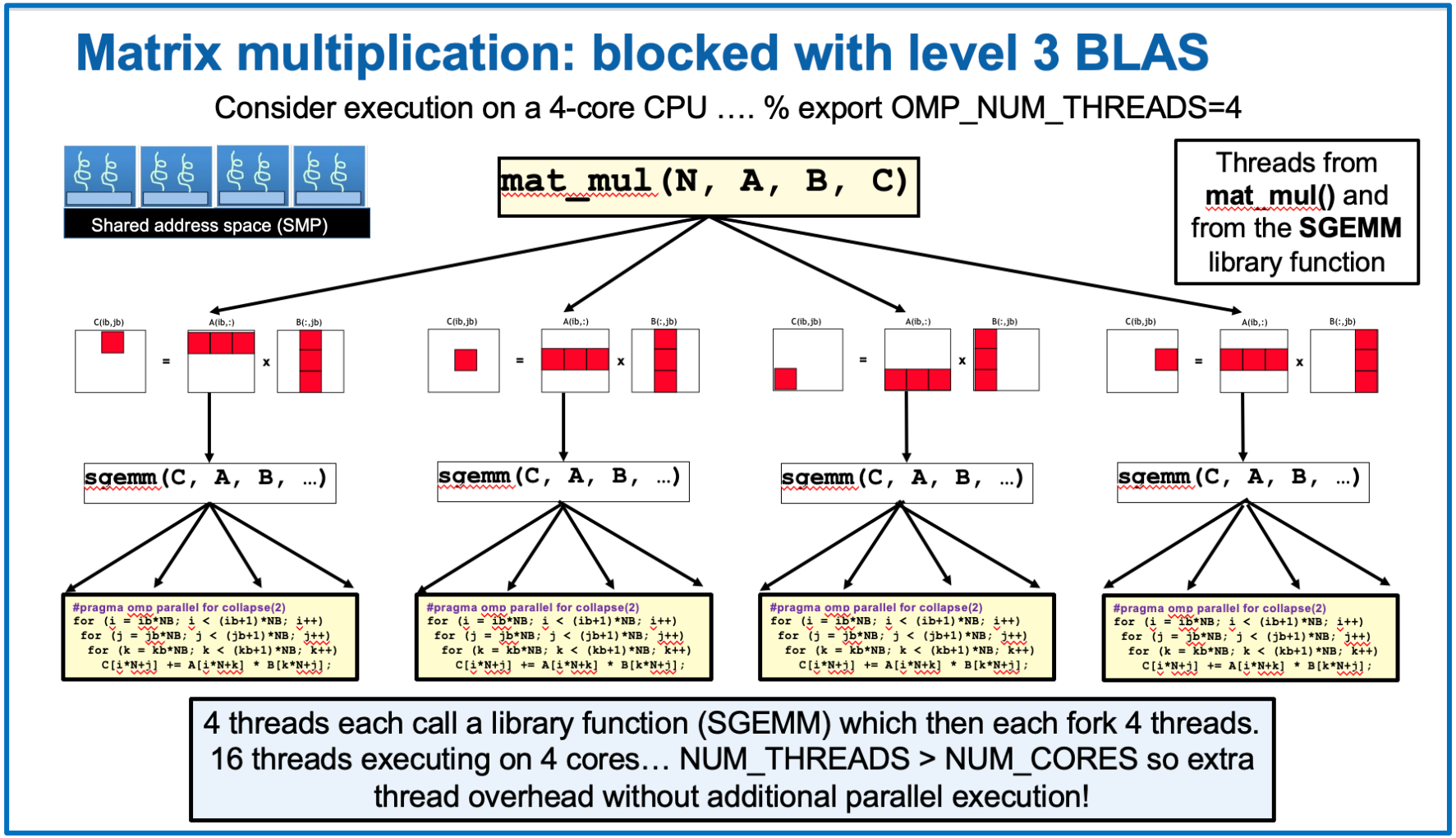
Consider execution on a 4-core CPU …. % export OMP_NUM_THREADS=4



4 threads each call a library function (SGEMM) which then each fork 4 threads. 16 threads executing on 4 cores… NUM_THREADS > NUM_CORES so extra thread overhead without additional parallel execution!

The Problem with OpenMP: Parallel Composition

- This is called the Parallel Composition problem.
- If concurrency is expressed in terms of threads, then as you call functions that use threads … you can get an explosion in the number of threads.
- When there are more threads than cores, you add thread management overhead without additional parallelism to support them.
- Performance can suffer dramatically



TBB: threading building blocks

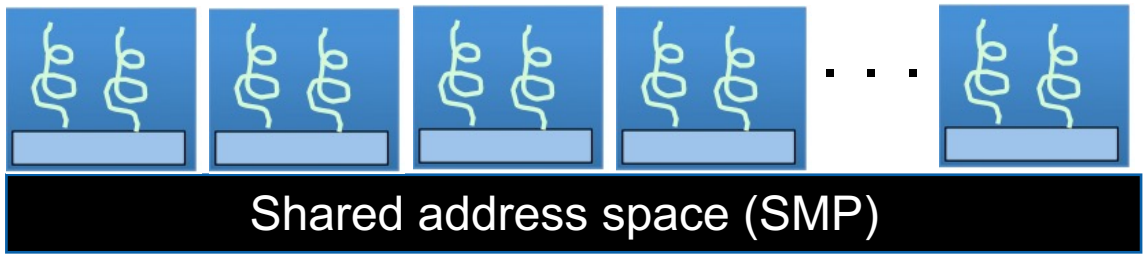


oneTBB

- TBB was created in the early 2000's by Arch Robison. First released (by Intel) in 2006.
  - Arch was involved with OpenMP in the early days of the standard, but was disappointed by how OpenMP mapped onto C++.
  - He created TBB as an API that would better fit with C++ perspective for parallel programming.

- The TBB execution model: Programmers think in terms of tasks with threads in the background,

A multicore CPU with P cores

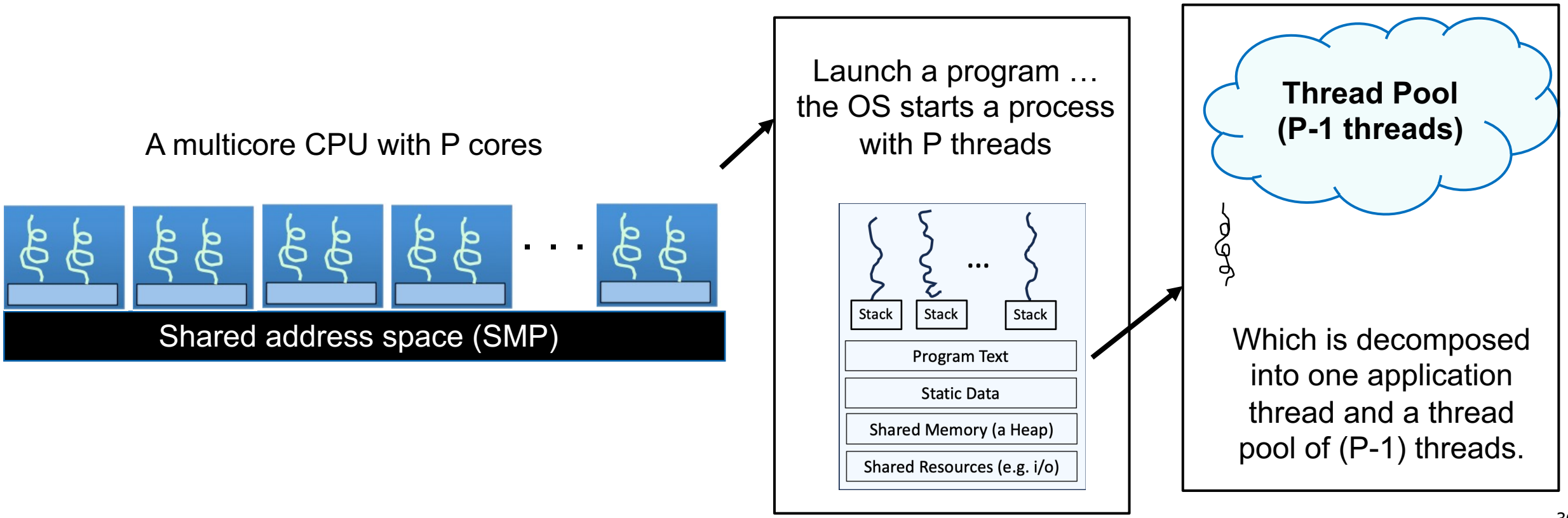


TBB: threading building blocks

oneTBB

- TBB was created in the early 2000's by Arch Robison. First released (by Intel) in 2006.
  - Arch was involved with OpenMP in the early days of the standard, but was disappointed by how OpenMP mapped onto C++.
  - He created TBB as an API that would better fit with C++ perspective for parallel programming.

- The TBB execution model: Programmers think in terms of tasks with threads in the background,



TBB: threading building blocks

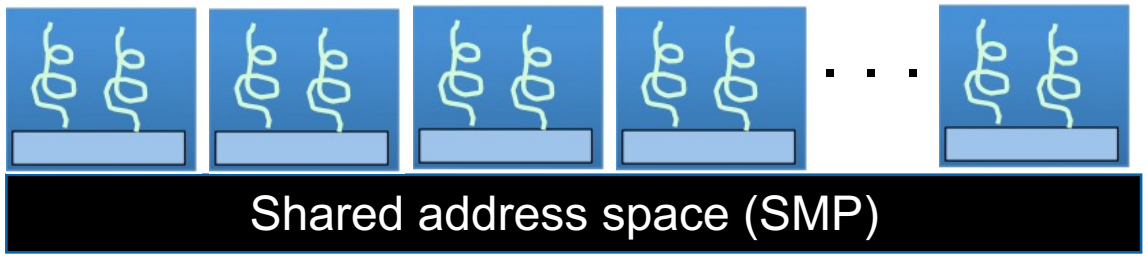


oneTBB

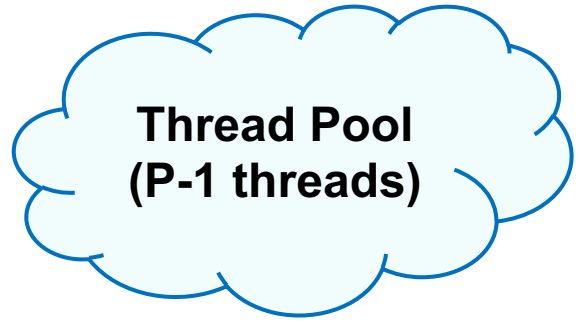
- TBB was created in the early 2000's by Arch Robison. First released (by Intel) in 2006.
  - Arch was involved with OpenMP in the early days of the standard, but was disappointed by how OpenMP mapped onto C++.
  - He created TBB as an API that would better fit with C++ perspective for parallel programming.

- The TBB execution model: Programmers think in terms of tasks with threads in the background,

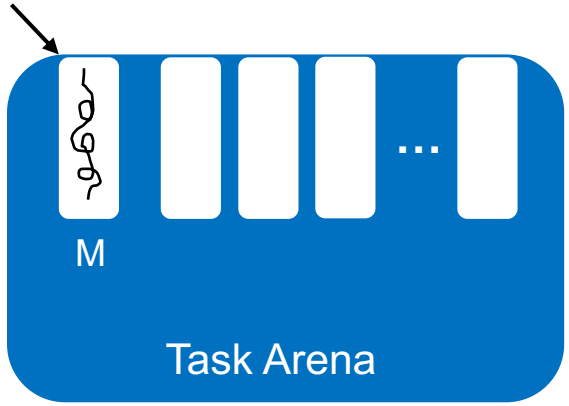
A multicore CPU with P cores



Threads in TBB execute through an arena of thread execution slots



The application thread goes in a master slot (M).



TBB: threading building blocks

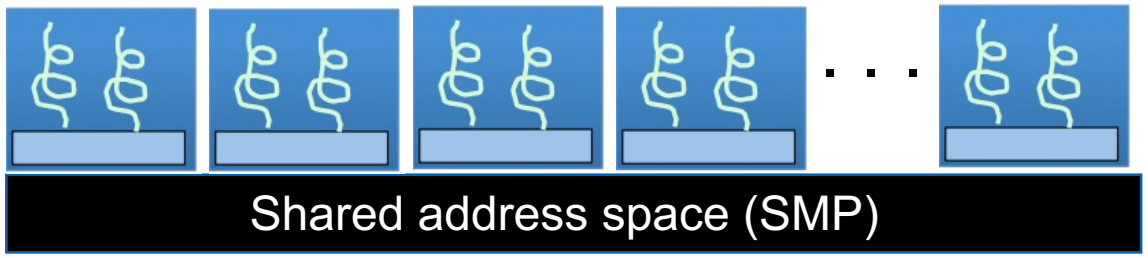


oneTBB

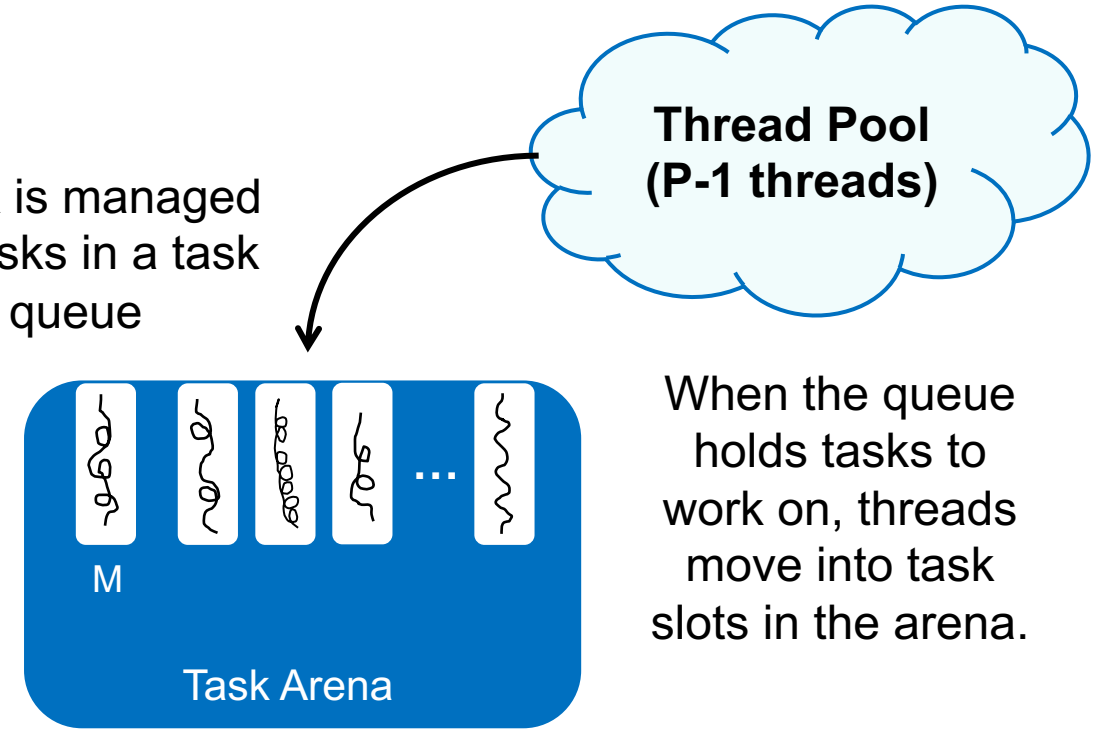
- TBB was created in the early 2000's by Arch Robison. First released (by Intel) in 2006.
  - Arch was involved with OpenMP in the early days of the standard, but was disappointed by how OpenMP mapped onto C++.
  - He created TBB as an API that would better fit with C++ perspective for parallel programming.

- The TBB execution model: Programmers think in terms of tasks with threads in the background,

A multicore CPU with P cores



Work is managed as tasks in a task queue



When the queue holds tasks to work on, threads move into task slots in the arena.

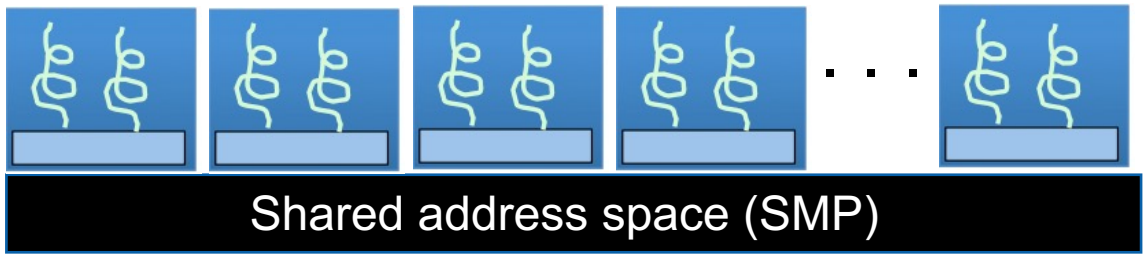
Load balancing and workstealing queues



oneTBB

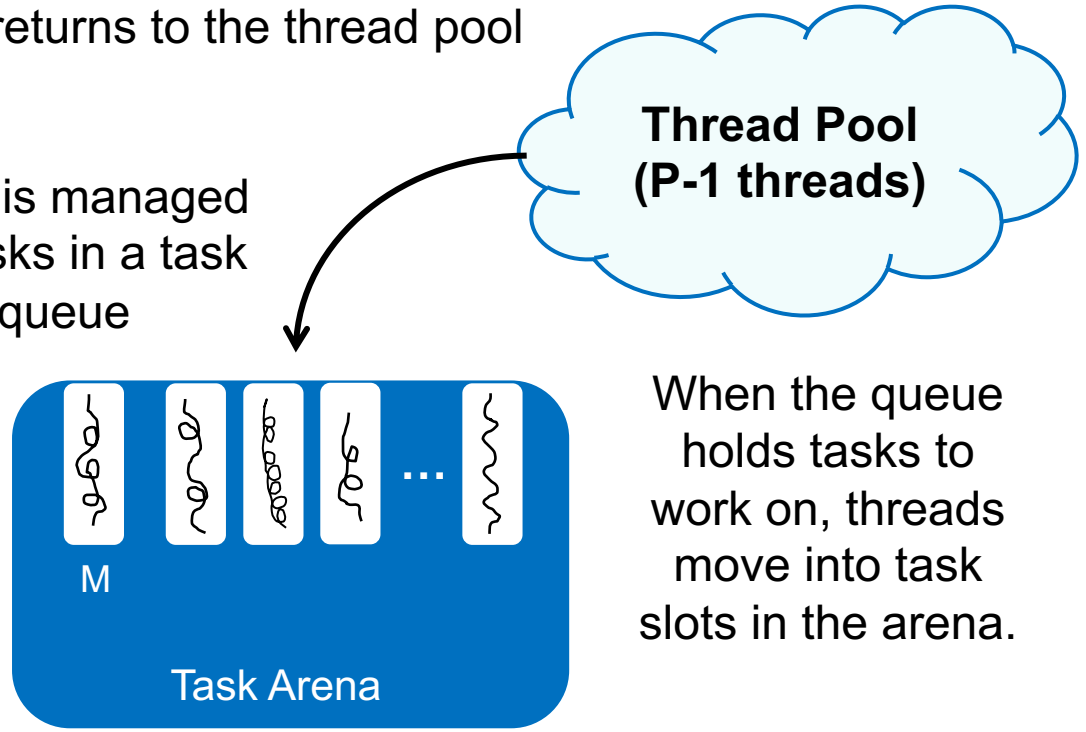
- TBB uses a provably optimal* strategy for balancing the load among the set of threads.
- Associated with each thread is a work-queue.
- If the work-queue is empty, it tries to steal work form the end of a randomly selected thread.
- If none of the threads have any more work to do, the thread returns to the thread pool

A multicore CPU with P cores



*This approach to load balancing in task-queue systems was developed by Charles Leiserson and his group at MIT in the context of the Cilk programming language.

Work is managed as tasks in a task queue



When the queue holds tasks to work on, threads move into task slots in the arena.

TBB: task groups

- As with OpenMP, you can create a group of tasks and manipulate them directly. Consider the Fibonacci number program with the divide and conquer algorithm.

```
long fib (int n) {
   long x,y;
   if (n < 2) return n;

#pragma omp task shared(x)
   x = fib(n-1);
#pragma omp task shared(y)
   y = fib (n-2);
#pragma omp taskwait
   return (x+y);
}

Int main() {
   int NW = 42;
   #pragma omp parallel
      #pragma omp single
           fib(NW);
   return 0;
}
```

Algorithmically, the TBB and OpenMP programs are VERY similar.

The TBB task_group is explicit and has a collection of methods for manipulating groups and how they execute

OpenMP makes the threads explicit. TBB "hides" them.

```
#include <tbb/tbb.h>
#include <tbb/task_group.h>
long fib (int n)
  long fnm1, fnm2;
  if(n<2)  return n;

  tbb::task_group group;

  group.run([&fnm1,n]{fnm1=fib(n-1);});
  group.run([&fnm2,n]{fnm2=fib(n-2);});

  group.wait();
  return fnm1+fnm2;
}
int main(){
  long NW = 42;

  long fib_comp = fib(num);
  return 0;
}
```

TBB Algorithms

- The TBB execution model is based on tasks, but most programming in TBB is done in terms of a collection of high-level parallel algorithms. TBB supports a range of these algorithms which include:
  - parallel_for.
  - parallel_reduction
  - parallel_scan
  - parallel_pipeline
  - parallel_sort
  - Flow_graph
- For example, consider parallel_for. At its simplest level you replace the sequential loop as follows:

```
#define N 100
int main()
{
    float A[N], B[N], C[N];

    for (int i=0; i<N; i++)
      A[i] = 1;   B[i]=i; C[i]=0;

    for (int i=0; i<N; i++)
      C[i] = A[i] + B[i];
}
```

This is the simplest case making use of basic, default behavior

TBB includes ways to control how loop iterations are partitioned, the granularity of units of scheduling, and many other details

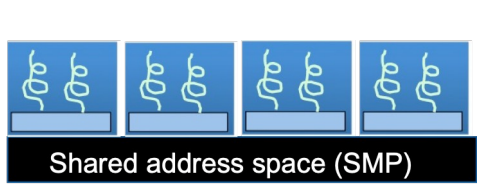
```
#include <tbb/tbb.h>
#define N 100
int main()
{
    float A[N], B[N], C[N];

    for (int i=0; i<N; i++)
      A[i] = 1;   B[i]=i; C[i]=0;

    tbb::parallel_for<int>(0,N,1,[&](int i) {
      C[i] = A[i] + B[i];
    });
}
```

Matrix multiplication: blocked with level 3 BLAS

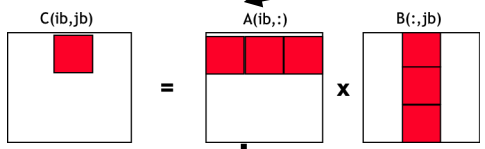
Consider execution on a 4-core CPU ….

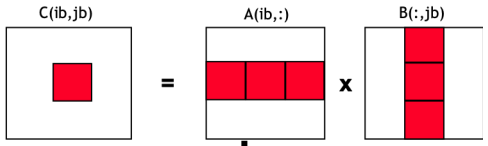


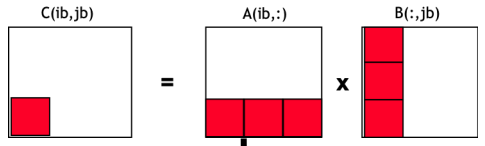

```
mat_mul(N, A, B, C)
```

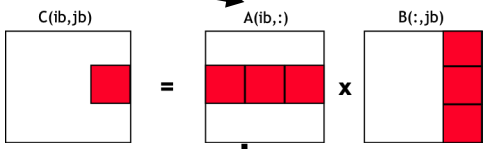
tasks from **mat_mul()** and from the **SGEMM** library function

One task for each block of the product matrix, C










```
sgemm(C, A, B, …)
```

```
sgemm(C, A, B, …)
```

```
sgemm(C, A, B, …)
```

```
sgemm(C, A, B, …)
```

One task-loop per SGEMM

One task-loop per SGEMM

One task-loop per SGEMM

One task-loop per SGEMM

```
#pragma omp parallel for collapse(2)
for (i = ib*NB; i < (ib+1)*NB; i++)
 for (j = jb*NB; j < (jb+1)*NB; j++)
  for (k = kb*NB; k < (kb+1)*NB; k++)
   C[i*N+j] += A[i*N+k] * B[k*N+j];
```

```
#pragma omp parallel for collapse(2)
for (i = ib*NB; i < (ib+1)*NB; i++)
 for (j = jb*NB; j < (jb+1)*NB; j++)
  for (k = kb*NB; k < (kb+1)*NB; k++)
   C[i*N+j] += A[i*N+k] * B[k*N+j];
```

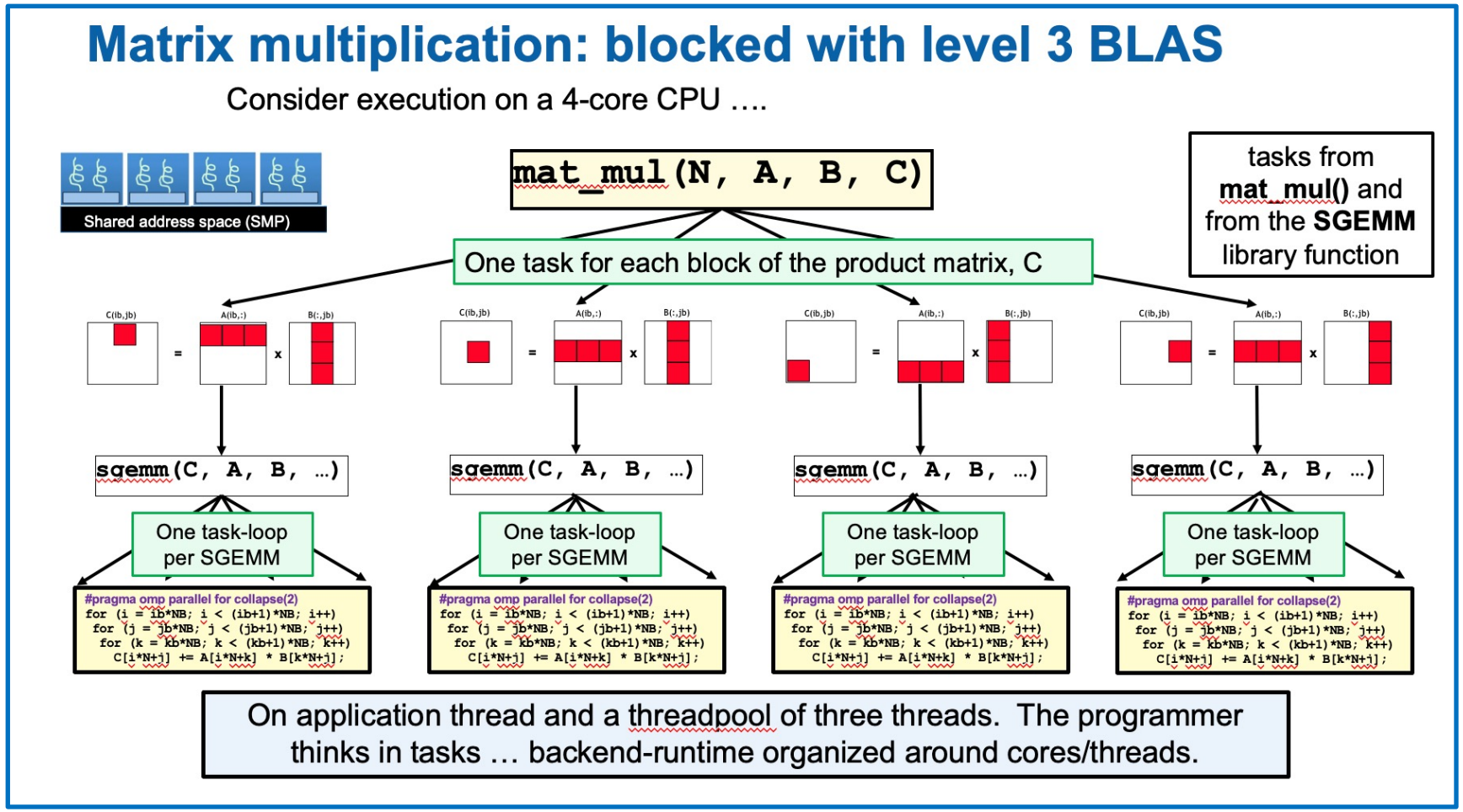
```
#pragma omp parallel for collapse(2)
for (i = ib*NB; i < (ib+1)*NB; i++)
 for (j = jb*NB; j < (jb+1)*NB; j++)
  for (k = kb*NB; k < (kb+1)*NB; k++)
   C[i*N+j] += A[i*N+k] * B[k*N+j];
```

```
#pragma omp parallel for collapse(2)
for (i = ib*NB; i < (ib+1)*NB; i++)
 for (j = jb*NB; j < (jb+1)*NB; j++)
  for (k = kb*NB; k < (kb+1)*NB; k++)
   C[i*N+j] += A[i*N+k] * B[k*N+j];
```

On application thread and a threadpool of three threads. The programmer thinks in tasks … backend-runtime organized around cores/threads.

TBB and Parallel Composition

- By separating the algorithm concept (tasks) from the execution context (threads) you can nest parallel functions without the number of threads blowing up.
- This means you can safely embed parallel functions inside parallel applications.
- Parallel composition ceases to be a problem if you use a task-based execution model such as TBB.



TBB: There is MUCH we did not cover.



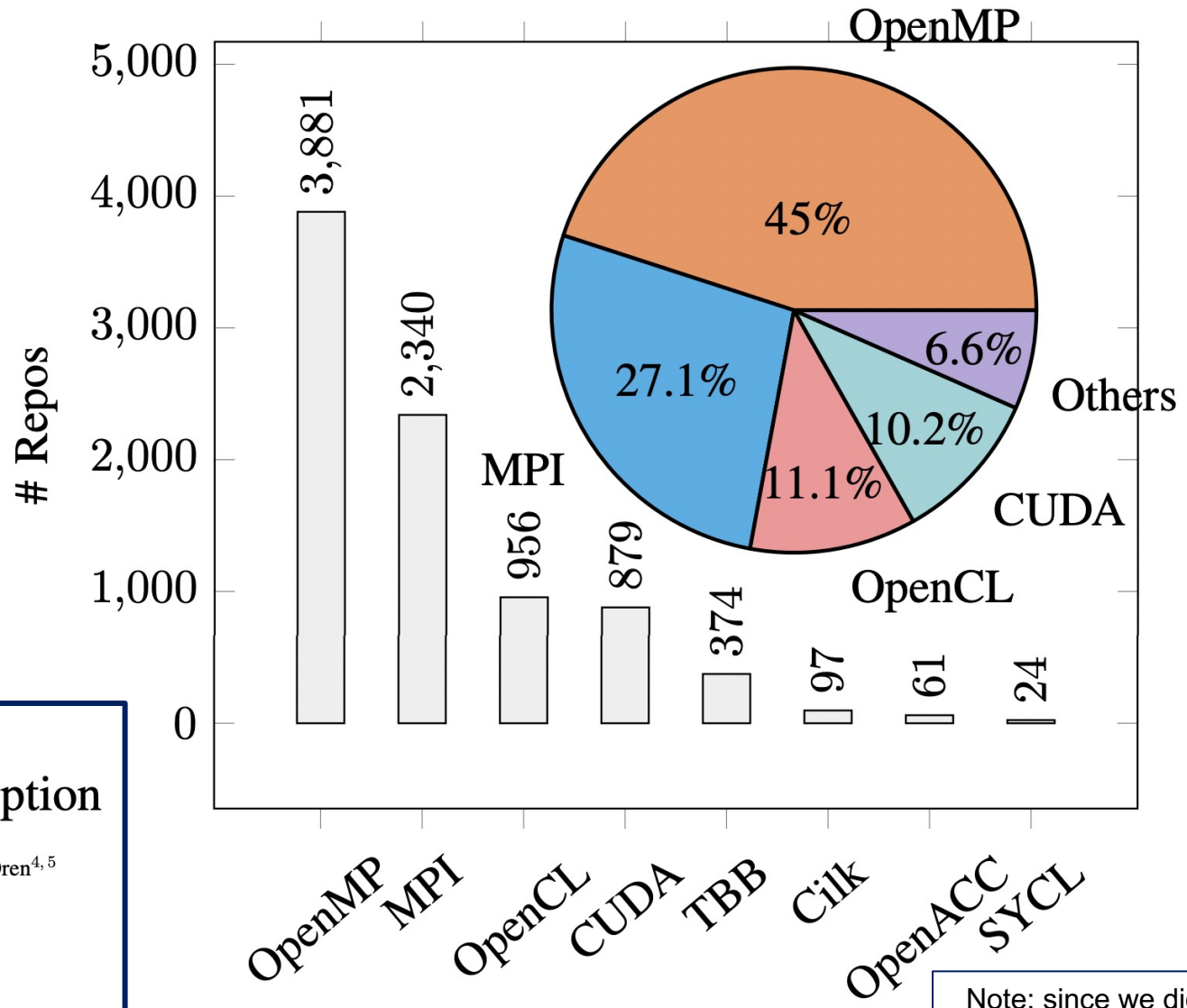
oneTBB

- TBB is over 20 years old and as with any older API, it has grown into a very large and full featured API. We have covered the basic execution model and just a few of the ways it can be used.

- In addition to parallell algorithms, TBB includes:
  - Detailed control over how algorithms are partitioned and mapped onto task queues
  - Detailed control over task groups and managing complex applications through multiple arenas.
  - A collection of concurrent containers (painfully difficult to implement … so it is GREAT that TBB provides them).
  - Mechanisms to control the execution details (numbers or threads, how threads map onto the system, and more).
  - Ways to map onto system utilities for managing execution in a NUMA environment.
  - ... And much more!!

Programming model usage 2013 to 2023

OpenMP is by far the most popular parallel programming model across Github repositories!!



Aggregate numbers over all repositories from 2013 to 2023

Quantifying OpenMP:
Statistical Insights into Usage and Adoption

Tal Kadosh[1,2], Niranjan Hasabnis[3], Timothy Mattson[3], Yuval Pinter[1] and Gal Oren[4,5]

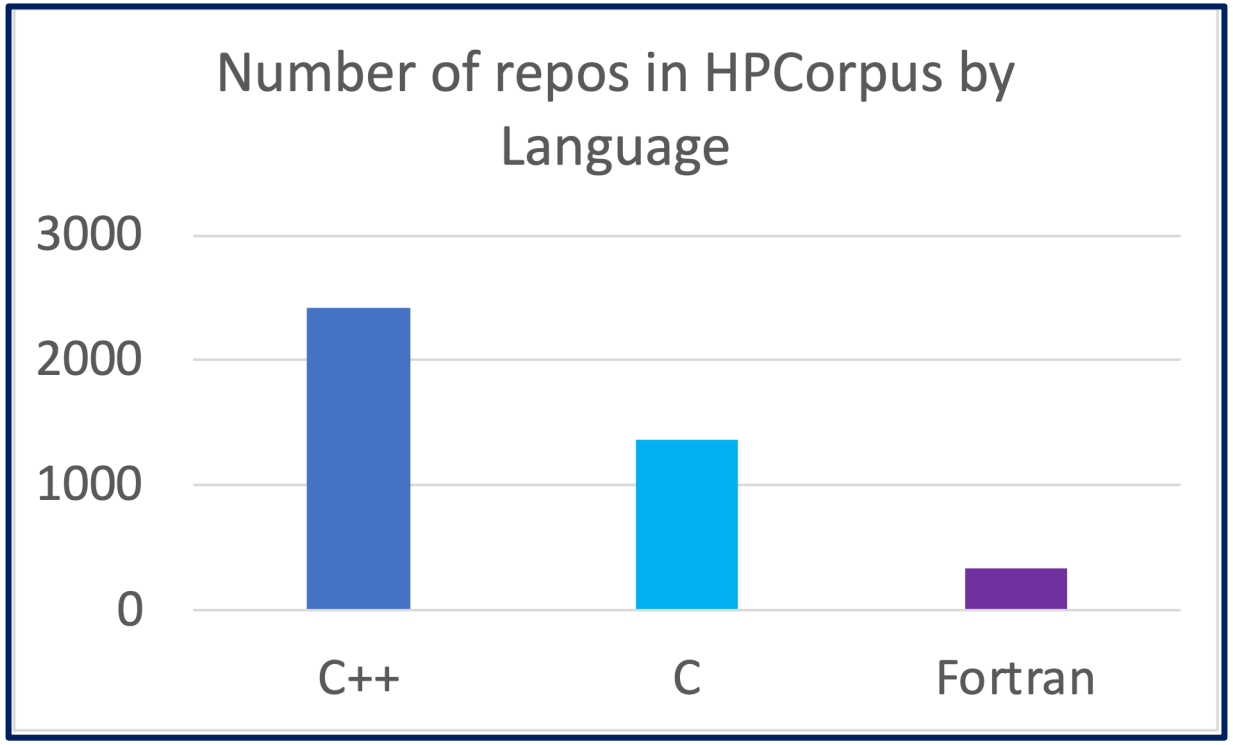
[1]Department of Computer Science, Ben-Gurion University, Israel
[2]Israel Atomic Energy Commission
[3]Intel Labs, United States
[4]Scientific Computing Center, Nuclear Research Center – Negev, Israel
[5]Department of Computer Science, Technion – Israel Institute of Technology, Israel

IEEE HPEC 2023

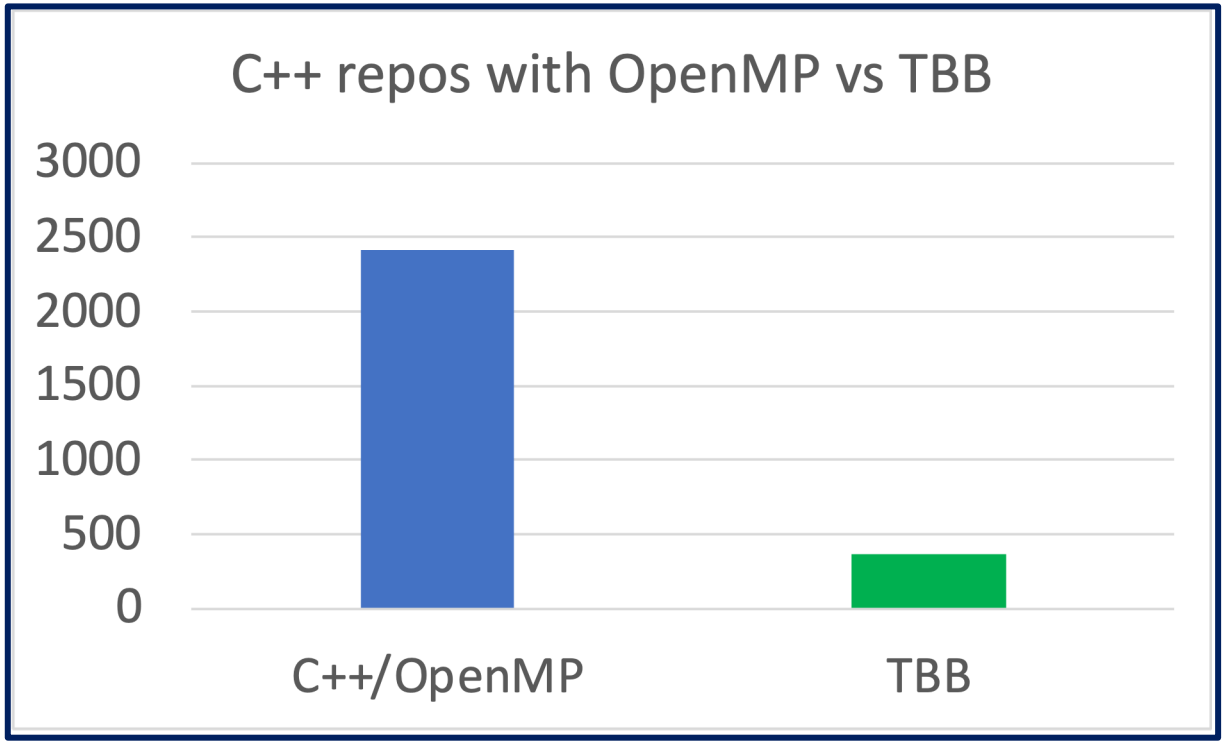
Note: since we did not collect files with .cu or .cuf suffices, we may have undercounted CUDA usage in HPCorpus.

C++ parallelism: OpenMP vs TBB

C++ is the most popular language for Parallel Programming

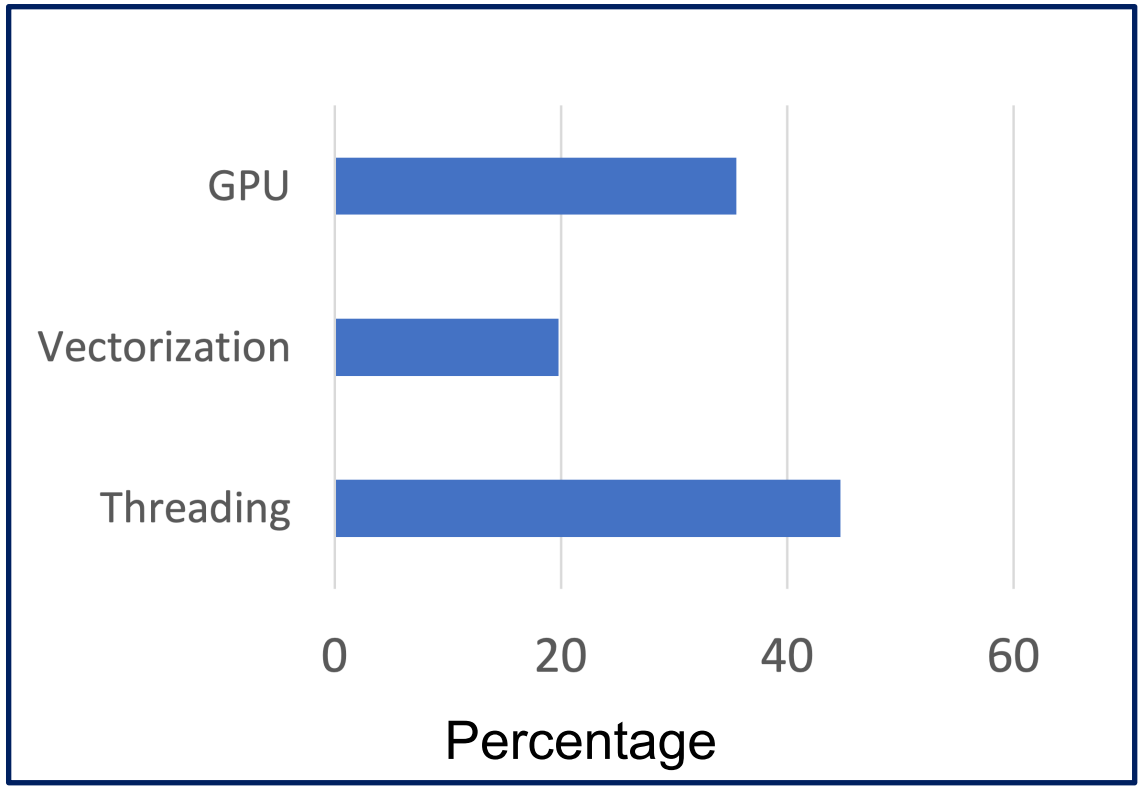


OpenMP is 6.5 times more likely than TBB by C++ parallel programmers

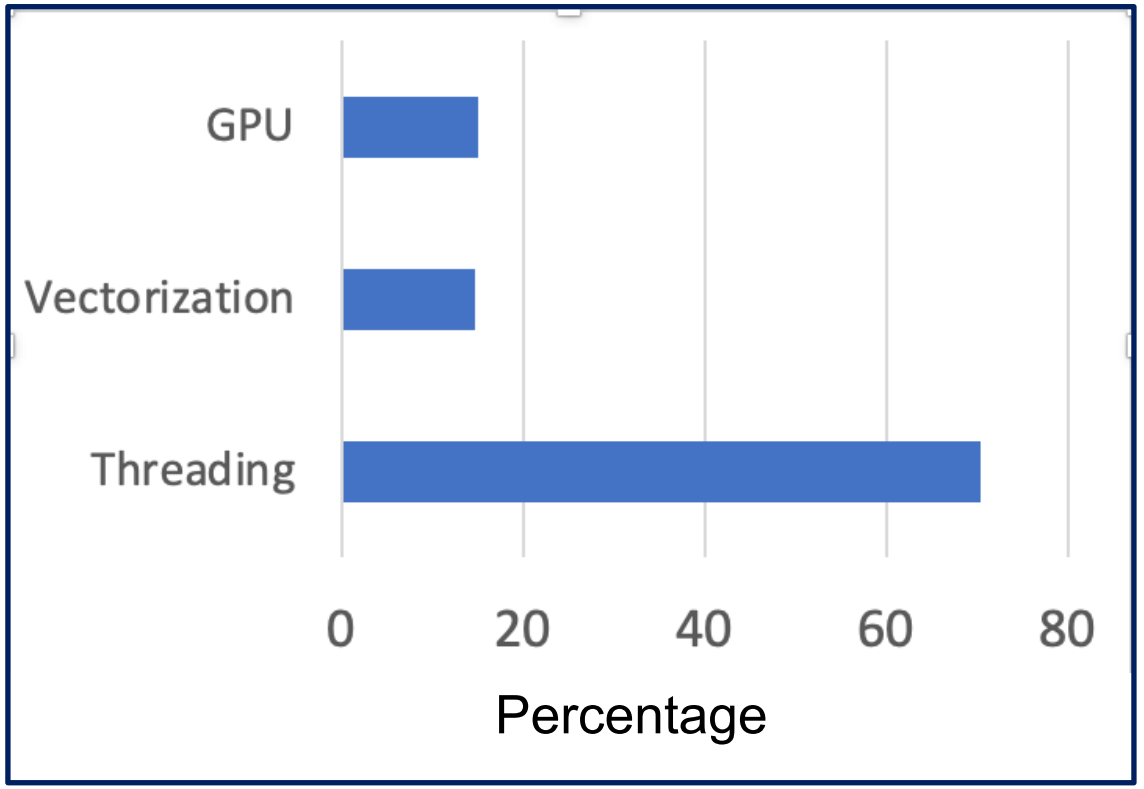


C++ vs C OpenMP programs in HPCorpus

Type of parallelism across full HPCorpus (C++)



Type of parallelism across full HPCorpus (C)



C++ programmers are MUCH more aggressive about the newer types of parallelism in OpenMP

We suspect a big reason OpenMP is more popular than TBB is that C++ programmers are highly likely to mix different styles of parallelism (something OpenMP does very well)

TBB Conclusion



oneTBB

- TBB has some real advantages
  - It was designed from the beginning to fit in with C++ programming styles
  - It solves the parallel composition problem

- The work steeling queues are optimal for load balancing task level parallelism. If you know your problems map nicely onto task level parallelism and you use C++, TBB is an excellent match.

- However, it is not as heavily used as OpenMP and unlike OpenMP, is not readily available everywhere you are likely to run your software.

Working with TBB

- oneTBB is Open Source under an Apache license. Source code is available on github:
  - https://github.com/uxlfoundation/oneTBB.git
  - oneTBB documentation is available at https://github.com/Apress/pro-TBB.git

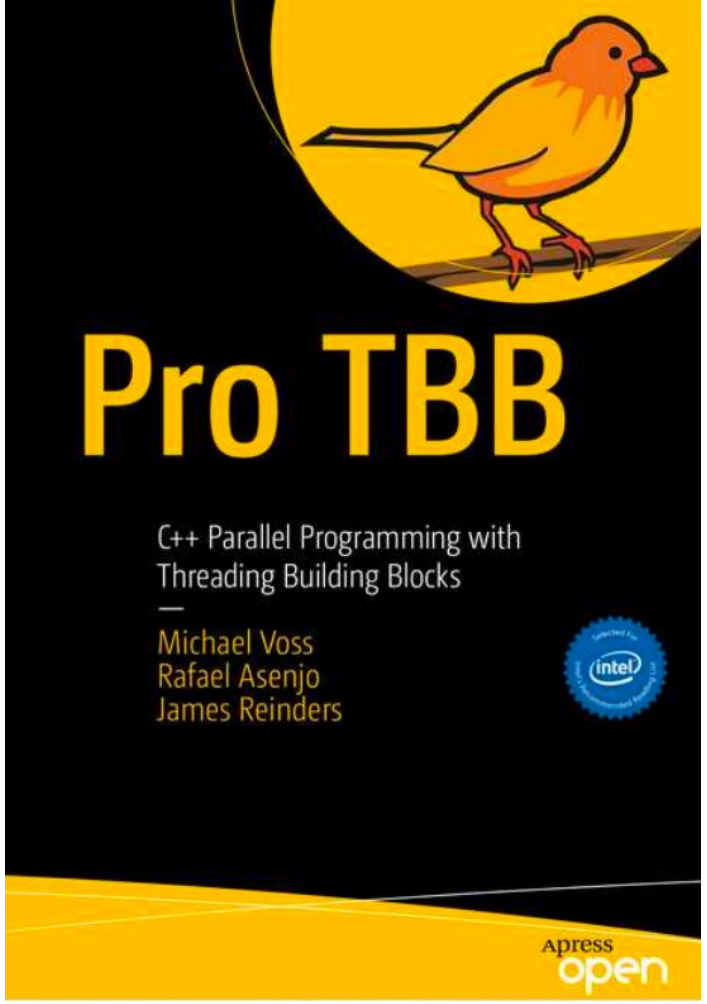
- Load on an Apple laptop with homebrew

```
% brew install tbb

% brew –prefix tbb
  /opt/homebrew/opt/tbb

% export TBB=/opt/homebrew/opt/tbb

% g++ -L$TBB/lib –I$TBB/include –ltbb file.cc
```



Open access book
https://link.springer.com/book/10.1007/978-1-4842-4398-5

Examples online at https://github.com/Apress/pro-TBB.git

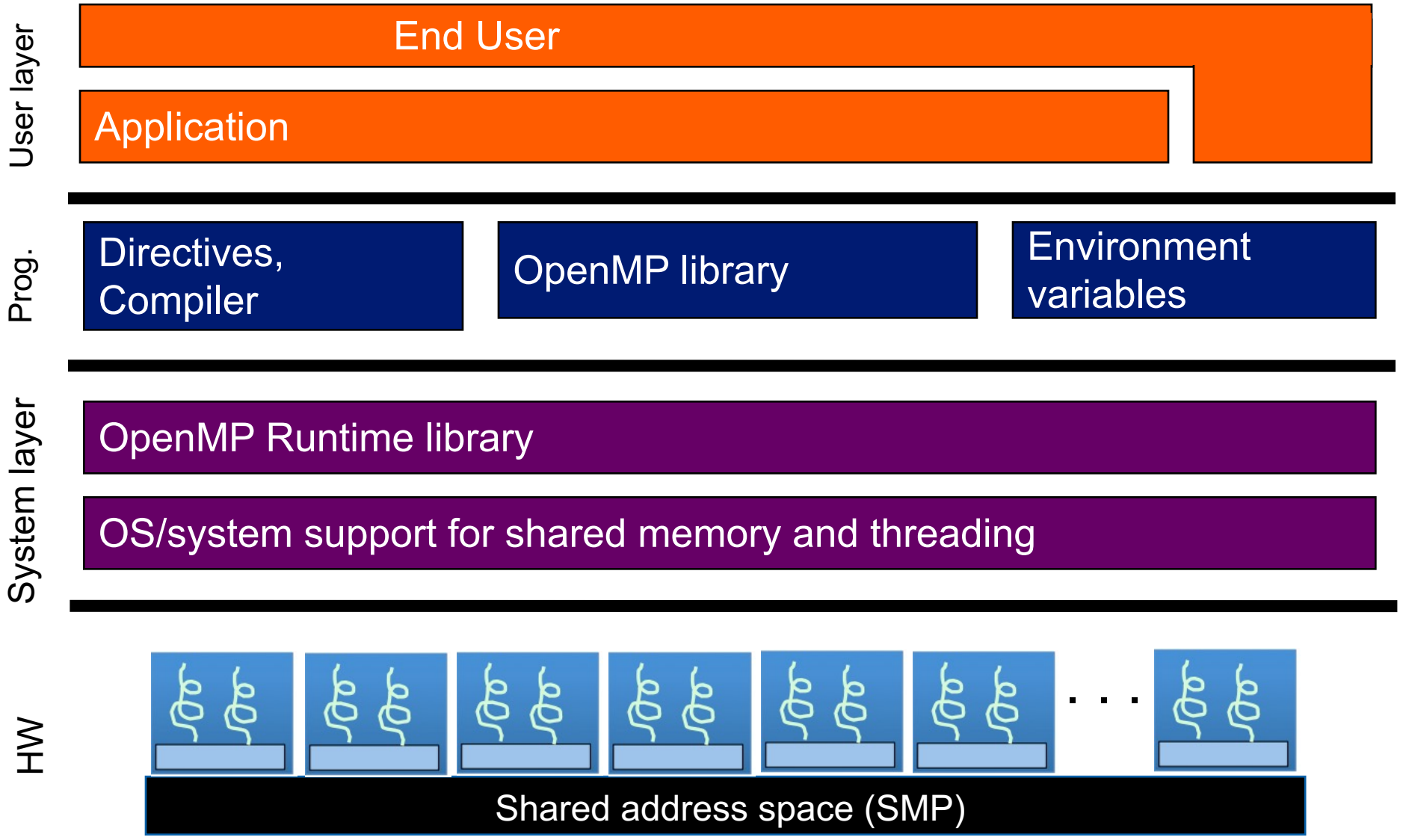
Note: the API is slightly out of date, but it is still valuable for the detailed background information on TBB.

Outline



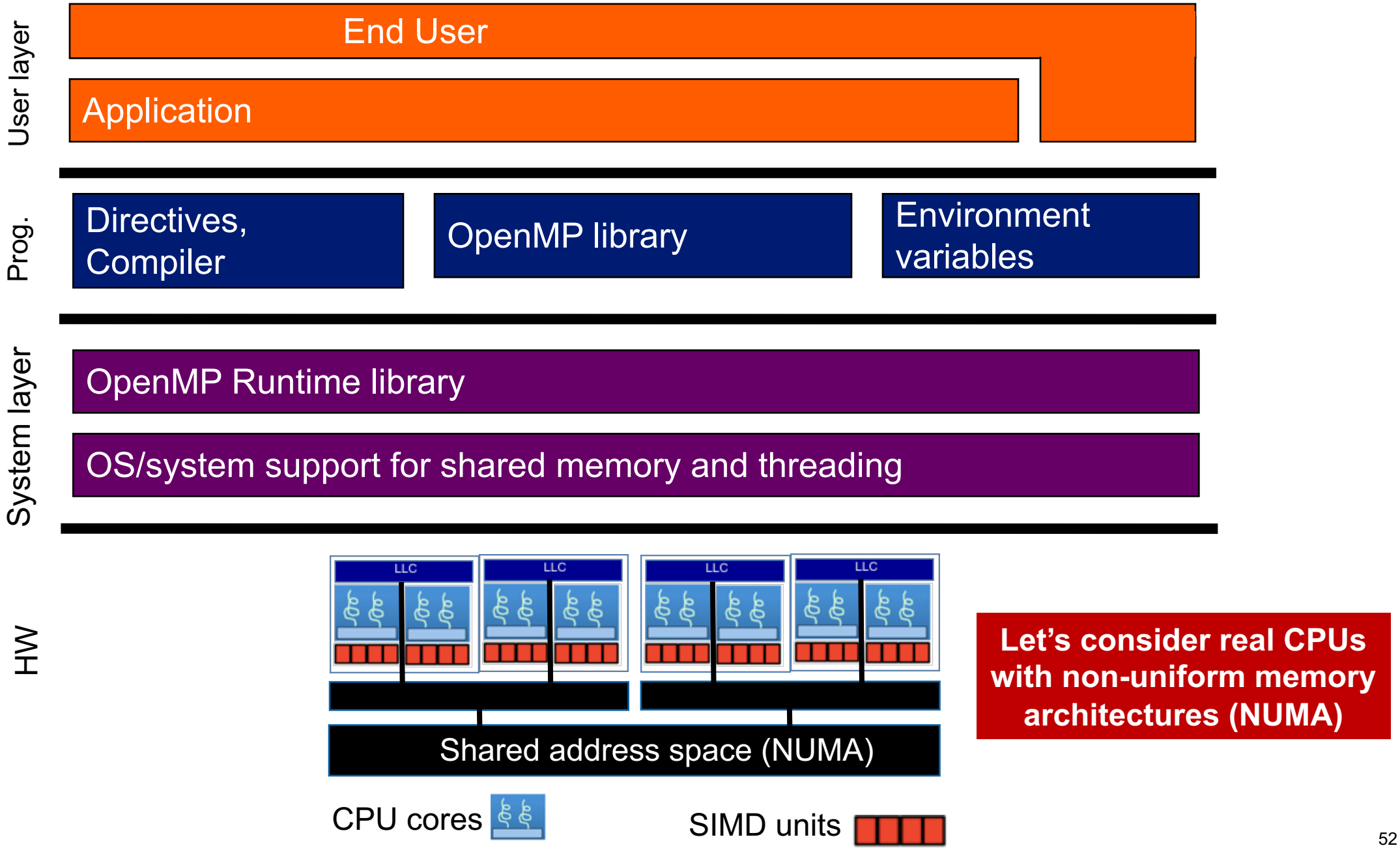
- Irregular Parallelism and tasks
- Other Systems for task-level parallelism: TBB
- Non-Uniform Memory Architectures (NUMA)
- Programming your GPU with OpenMP
- Recap

OpenMP Basic Definitions: Basic Solution Stack



For the OpenMP Common Core, we focus on Symmetric Multiprocessor Case ….
i.e., lots of threads with “equal cost access” to memory

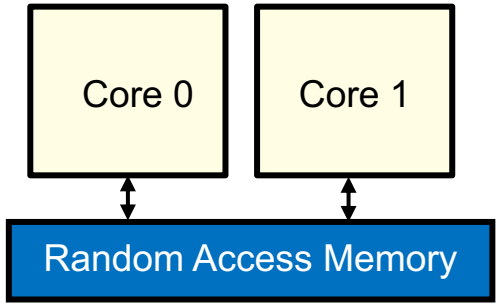
OpenMP Basic Definitions: Basic Solution Stack

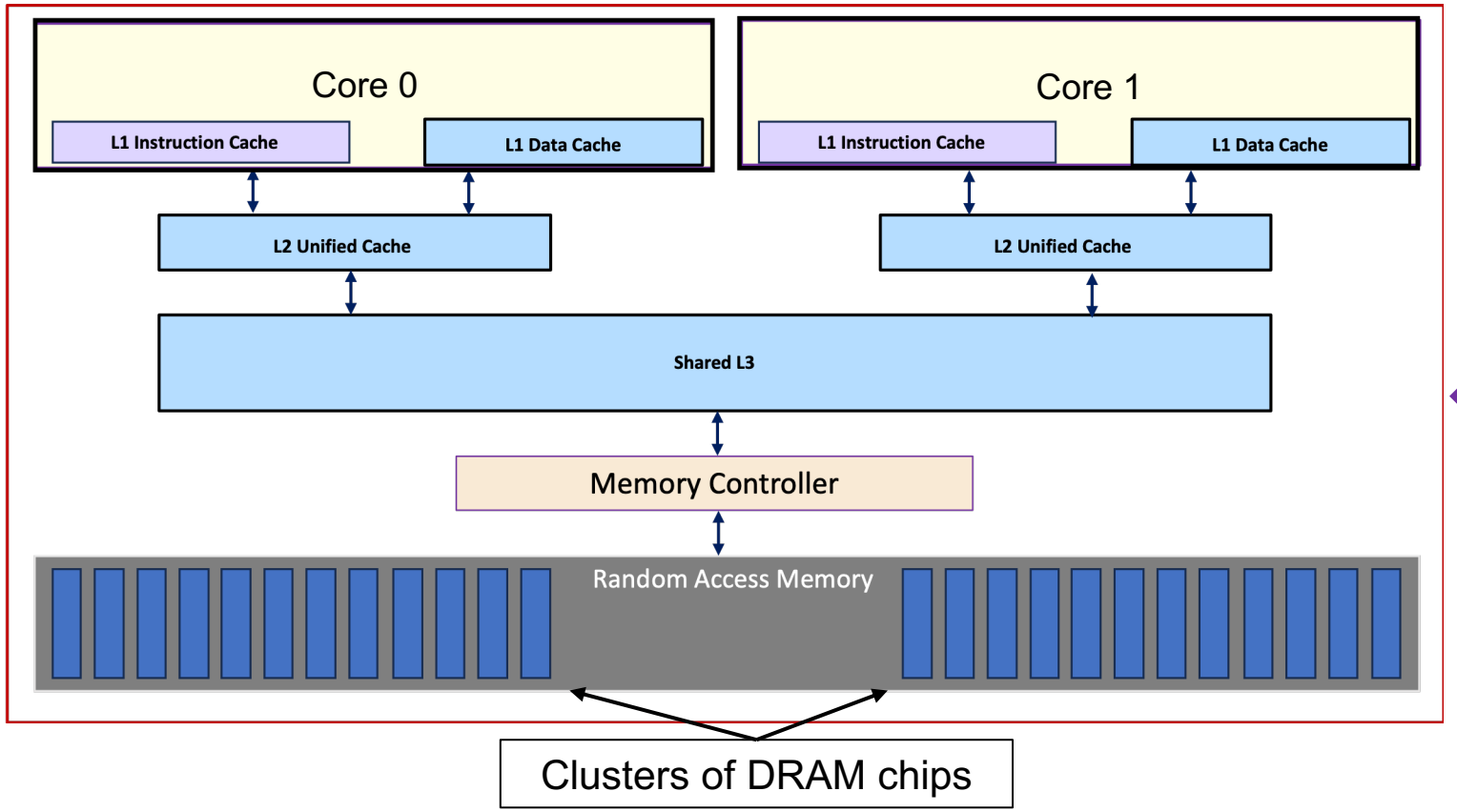


Let's consider real CPUs with non-uniform memory architectures (NUMA)

SMP is a lie … The memory is Non-Uniform

We like to draw pictures like this





But reality is much more complex

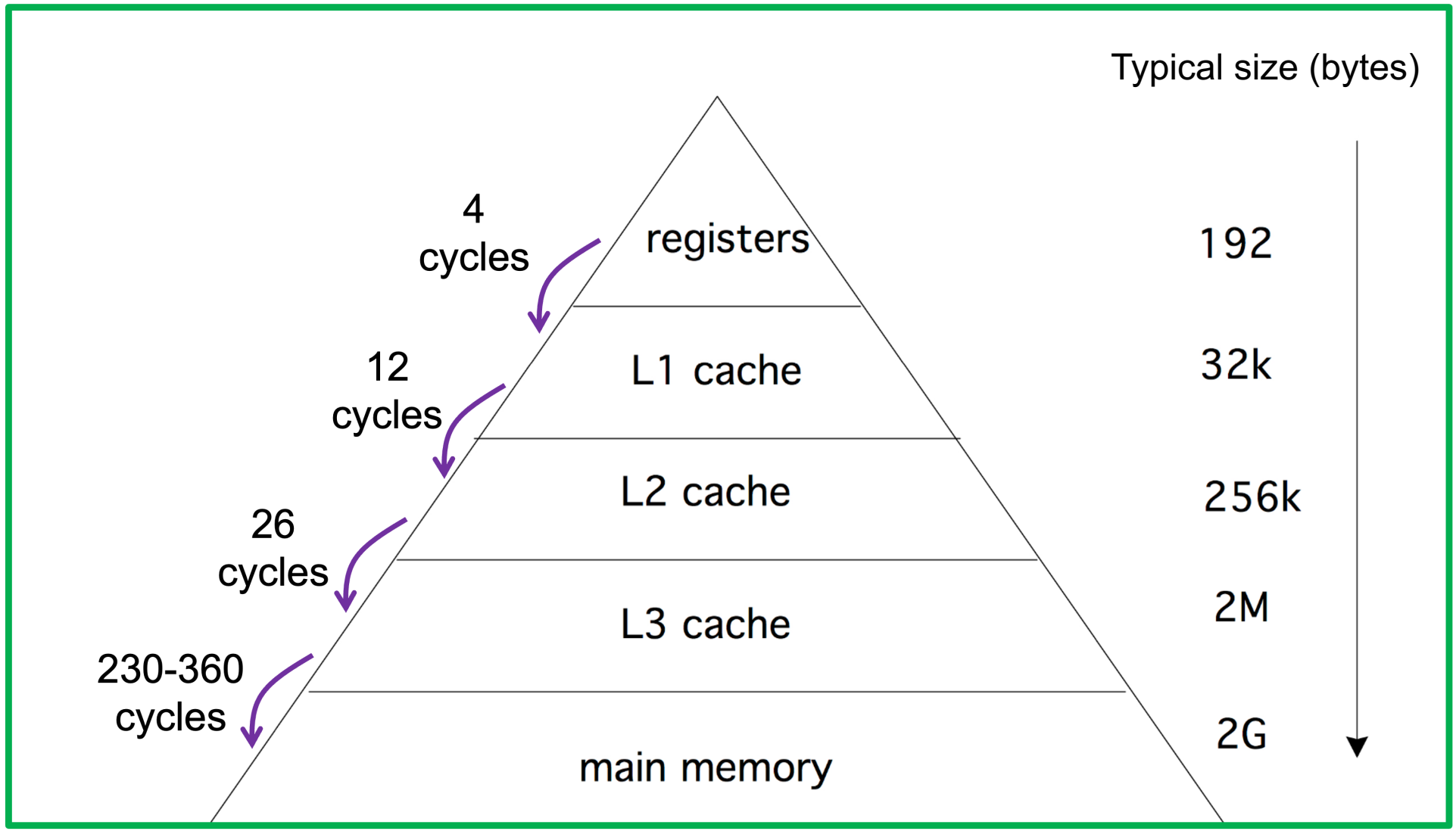
SMP: Symmetric Multiprossesor

DRAM: Dynamic Random Access Memory

L1, L2, L3 pronounced Level 1, Level 2, Level 3. Also written sometimes as $L1, $L2, $L3

L2 is called a unified cache since it holds both instructions and data

Latencies across the Memory Hierarchy

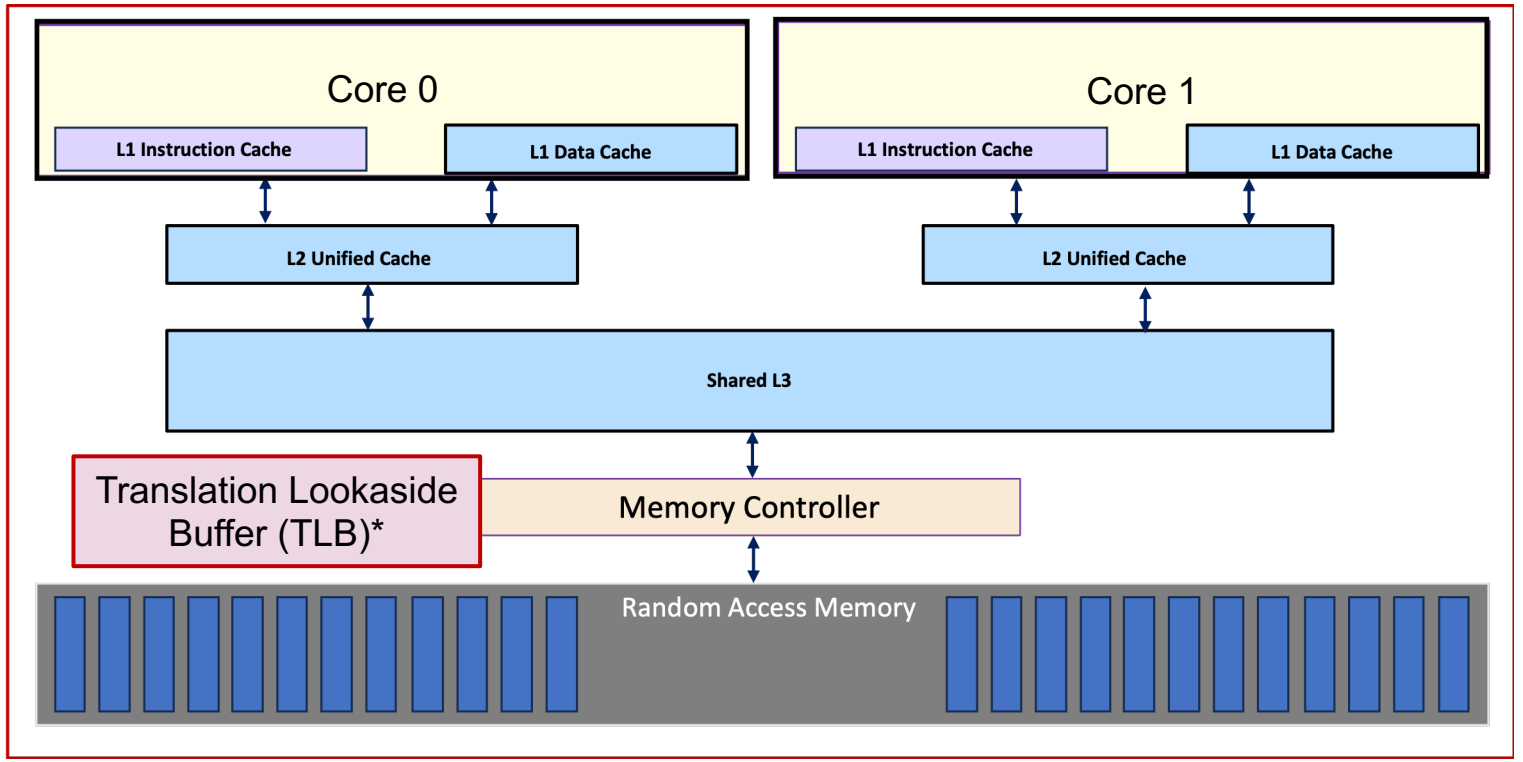


Source: Victor Eijkhout – Processor Architecture – Fall'2023

Memory Hierarchies are even more complex: the TLB

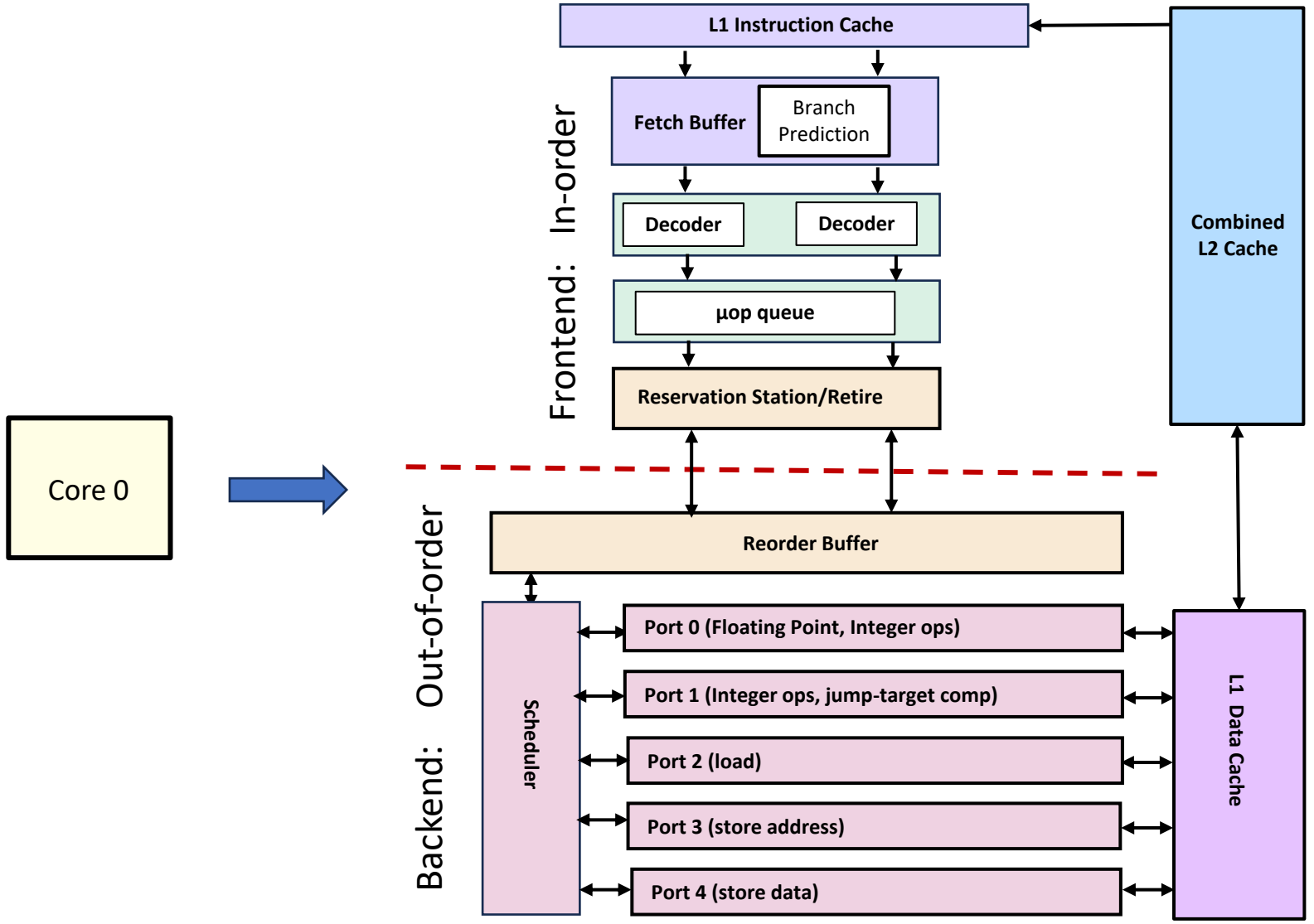
- A 64-bit address space can address 16 exabytes of memory. Clearly, that exceeds even the largest RAMs.
- A systems maps the full address space (virtual memory) onto **pages** of physical memory.

- The **TLB (Translation Lookaside Buffer)** implements virtual memory and caches the mapping from virtual addresses to physical addresses
  - If the entry is in the TLB page table, a small overhead of reading the entry and computed the physical address is incurred.
  - If the entry is not in the page table, a page fault exception will be raised. Requires expensive OS operations and stalls the CPU.



*Note: we show just one simple configuration of the TLB where it is associated with the memory controller. The TLB can be hierarchical with separate TLB tied to the cache hierarchy and different TLBs for data and instructions

Once you get past Memory Hierarchy issues, consider the complexity of the cores



Frontend processes instructions for execution.

Maps them into micro-operations (**µop**) and manages them (and the register file) so they retire "in order"

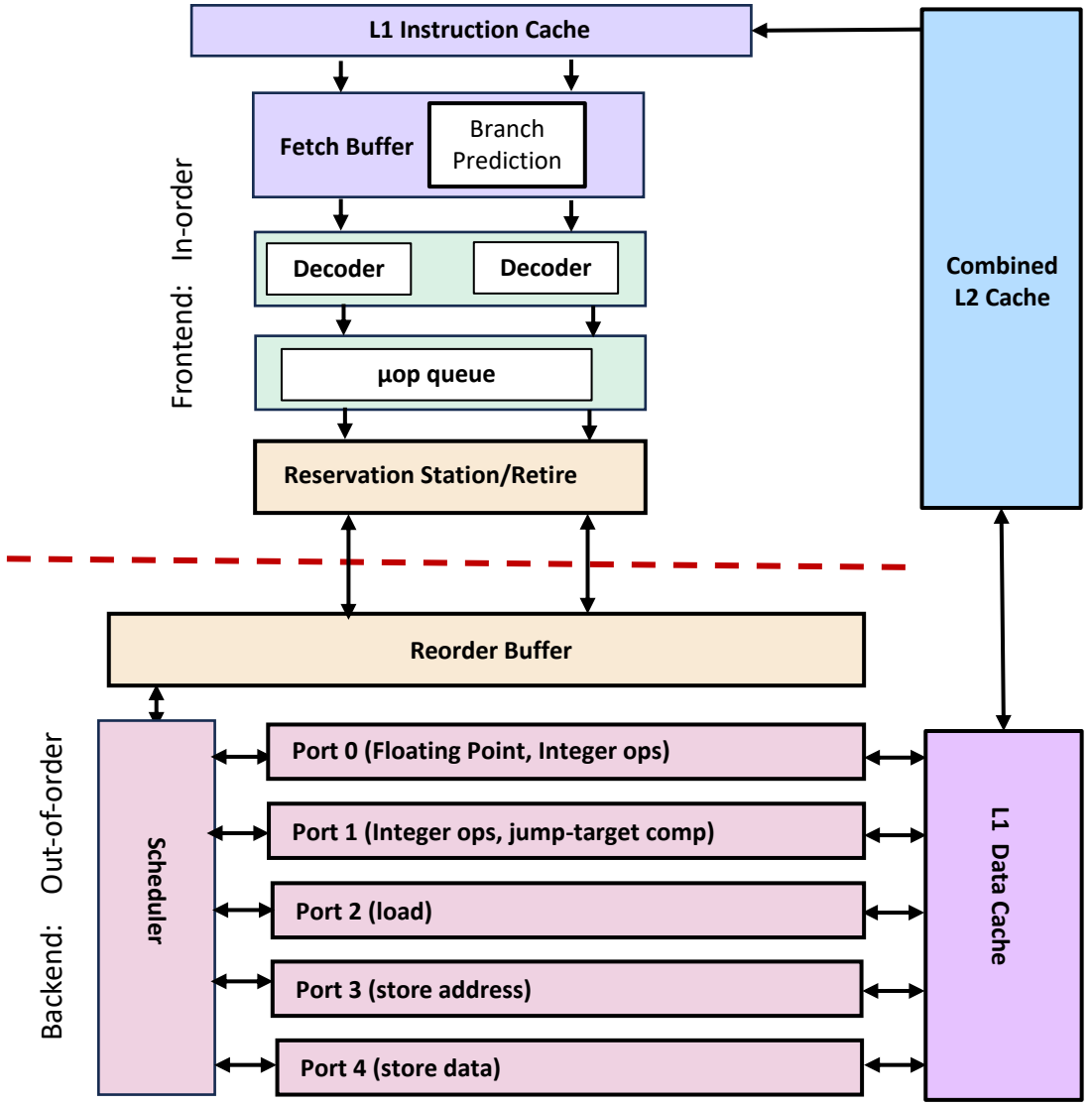
Backend executes decoded instructions.

Instructions are pipelined and include multiple ports and SIMD execution units.

Execution is "out of order" to give the scheduler maximum flexibility to maximize instruction-level parallelism

And its even worse ….

A single frontend typically can't generate enough decoded instructions to keep all the resource in the backend busy.

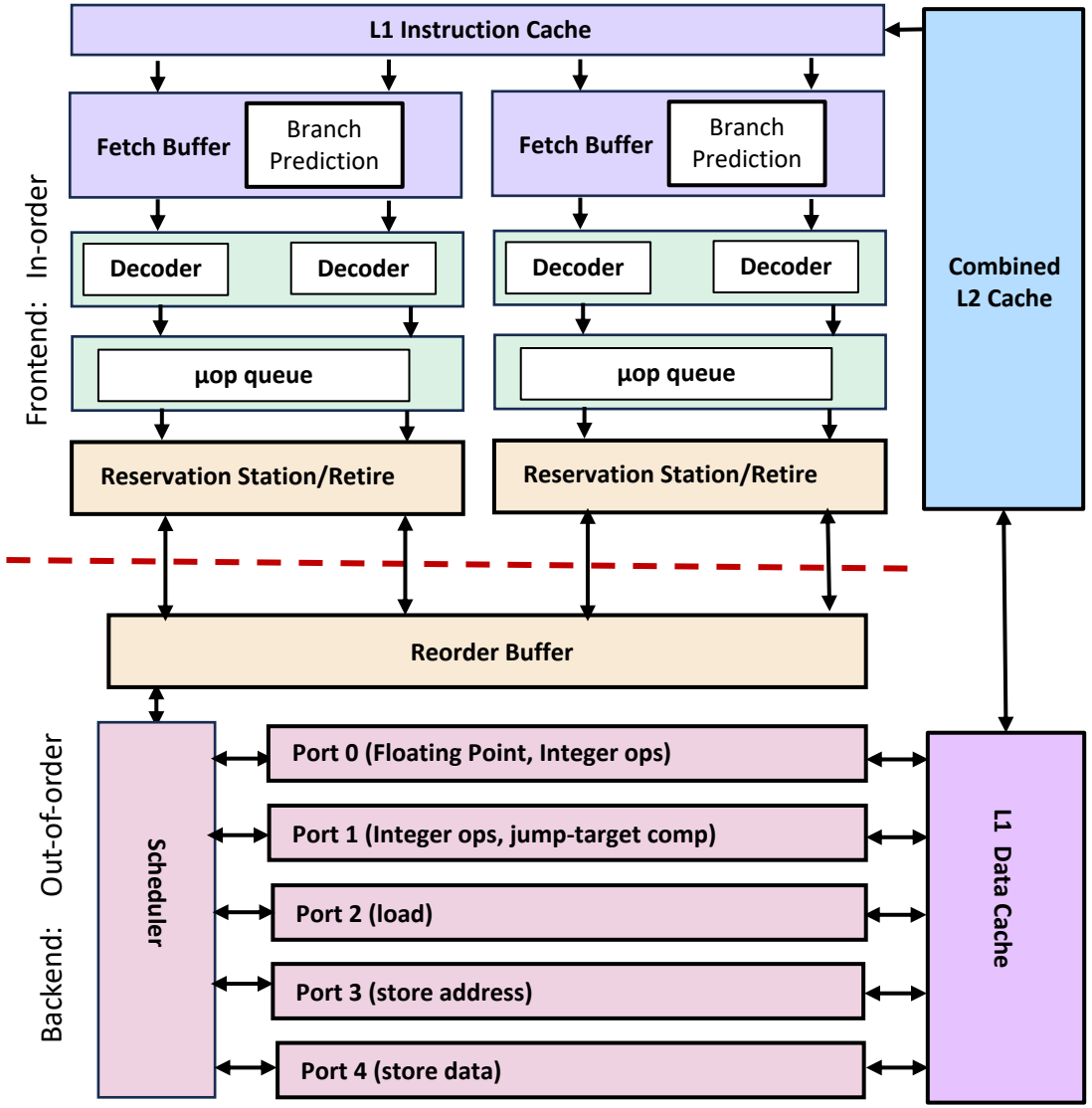


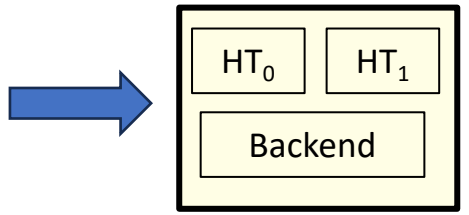
And its even worse ….

So, we have multiple frontends (in this case, 2) feeding a single back end.

These appear to the OS as an additional core … or a virtual core.

They maintain state of a thread in hardware so these are often called **Hardware threads.**





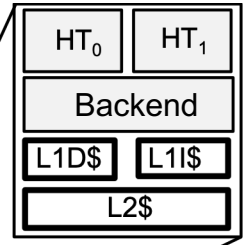
HT: hardware thread

And then we put all this into a real two-socket node

2 Intel® Xeon™ E5-2698 v3 CPUs (Haswell) per node (launched Q3'14)

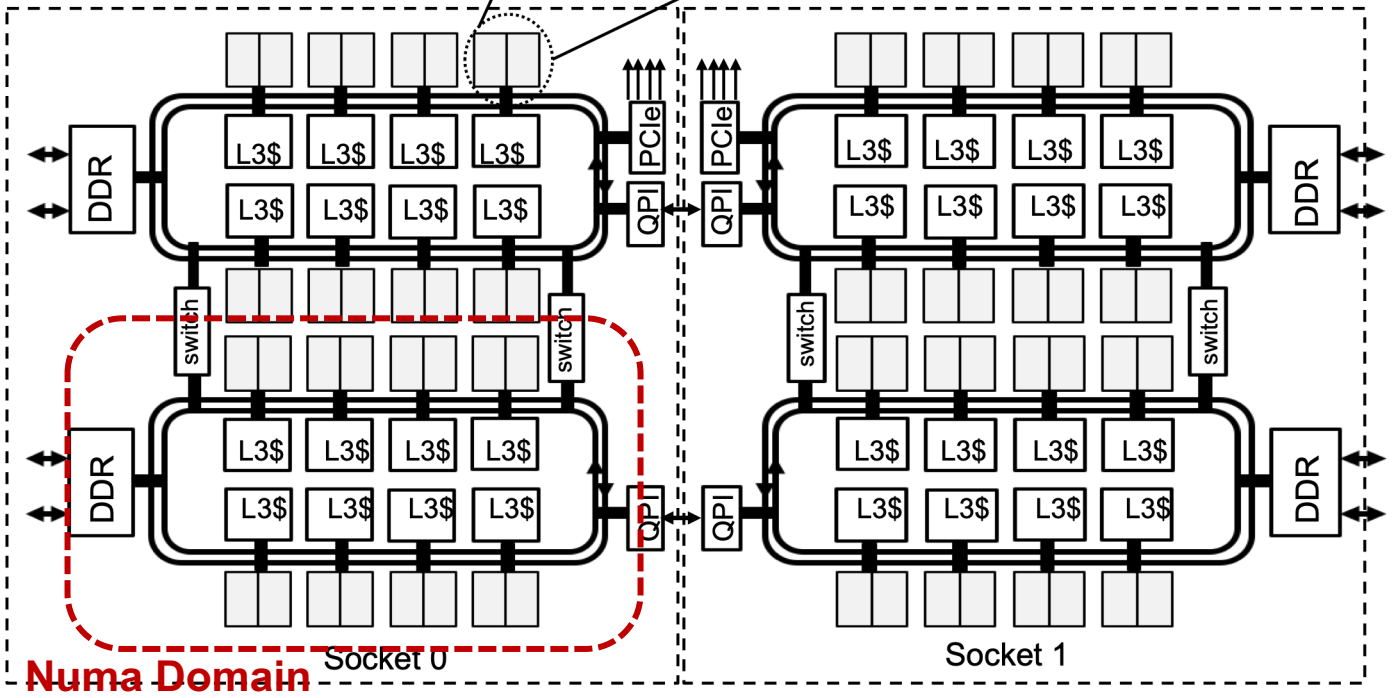
4 blocks of 8 core units connected by an on-chip-network with a DDR memory controller.

Each block is a NUMA domain since memory access from a core to its "own" DDR is less expensive.



2 Hardware threads (HT) per core
Intel® AVX2 (256 bit Vector unit)
L1$ instruction and data: 32 KB
Unified L2$ 256 KB

40 MB shared L3$



DDR: Double Data Rate memory controller

PCIe is the connection from the CPU to other devices in a node.

QPI: Quick Path Interconnect. A coherent interconnect between CPUs. Makes it easy to build multi-CPU nodes.

As configured for Cori at NERSC: CPUs at 2.3 GHz, 2 16 GB DIMMs per DDR memory controller, 16 cores per CPU. 2 CPUs connected by a high-speed interconnect (QPI)

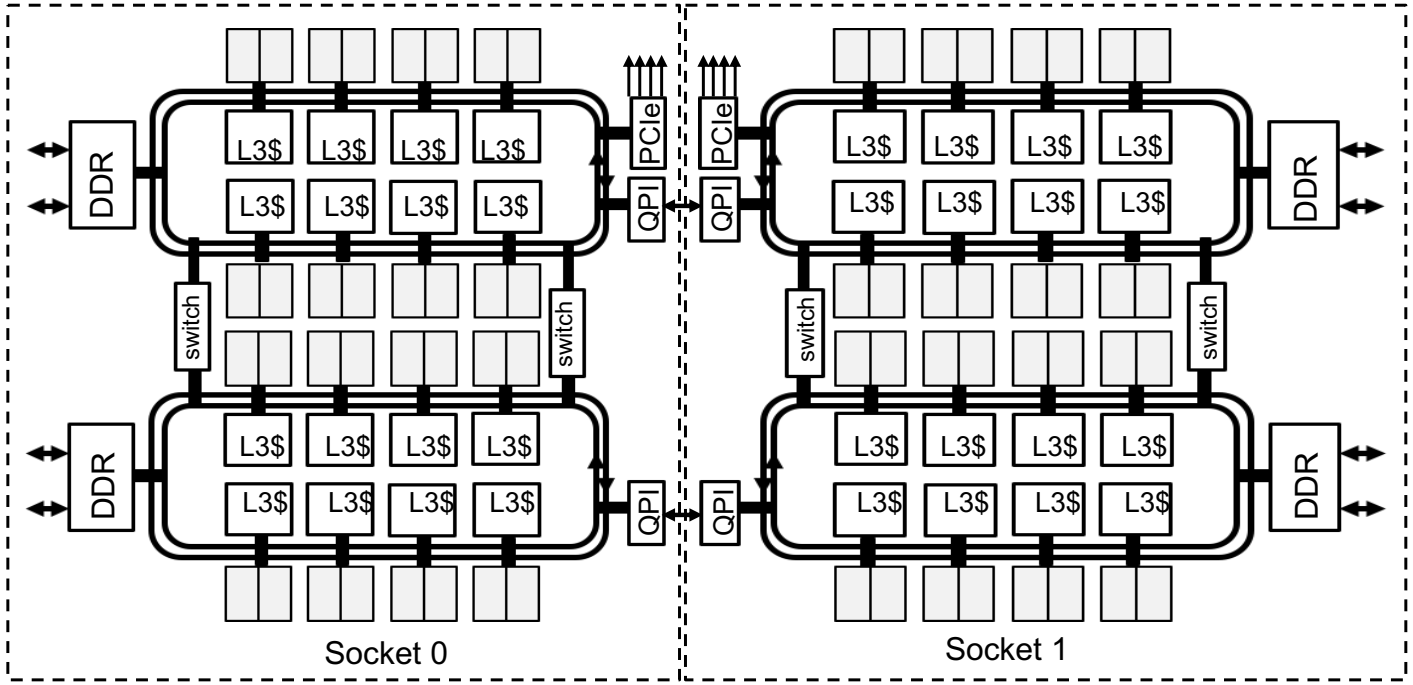
And then we put all this into a real two-socket node

2 Intel® Xeon™ E5-2698 v3 CPUs (Haswell) per node (launched Q3'14)

- A floating-point operation takes O(~1 ns).
  - L1 Cache ~1.5 ns
  - L2 Cache reference ~5 ns
  - L3 Cache reference ~25 ns
  - Near memory DRAM access ~100ns
  - Near memory DRAM access ~100ns
  - Far memory DRAM access ~200 ns

- **The key to performance is to minimize memory movement:** get the memory movement right and the "rest" is easy

40 MB shared L3$



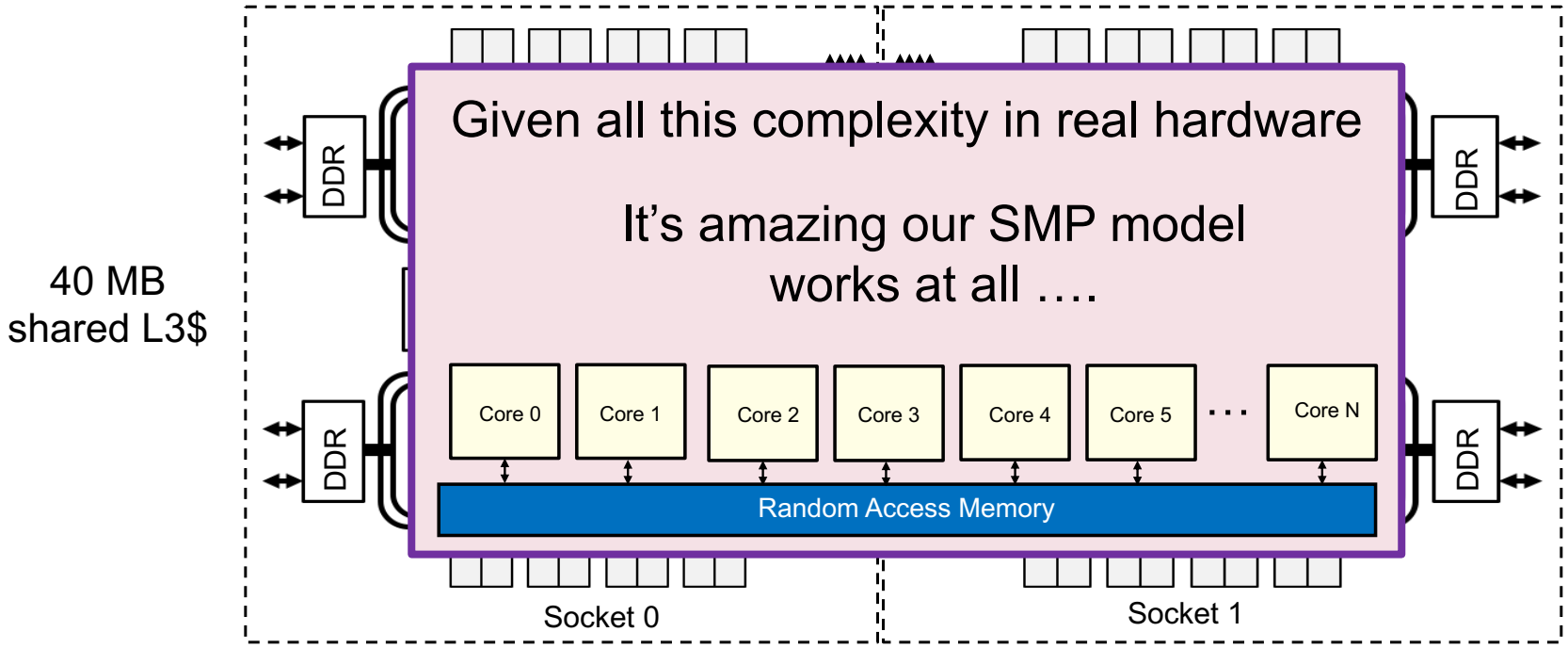
As configured for Cori at NERSC: CPUs at 2.3 GHz, 2 16 GB DIMMs per DDR memory controller, 16 cores per CPU. 2 CPUs connected by a high-speed interconnect (QPI)

And then we put all this into a real two-socket node

2 Intel® Xeon™ E5-2698 v3 CPUs (Haswell) per node (launched Q3'14)

- A floating-point operation takes O(~1 ns).
  - L1 Cache ~1.5 ns
  - L2 Cache reference ~5 ns
  - L3 Cache reference ~25 ns
  - Near memory DRAM access ~100ns
  - Near memory DRAM access ~100ns
  - Far memory DRAM access ~200 ns

- **The key to performance is to minimize memory movement:** get the memory movement right and the "rest" is easy

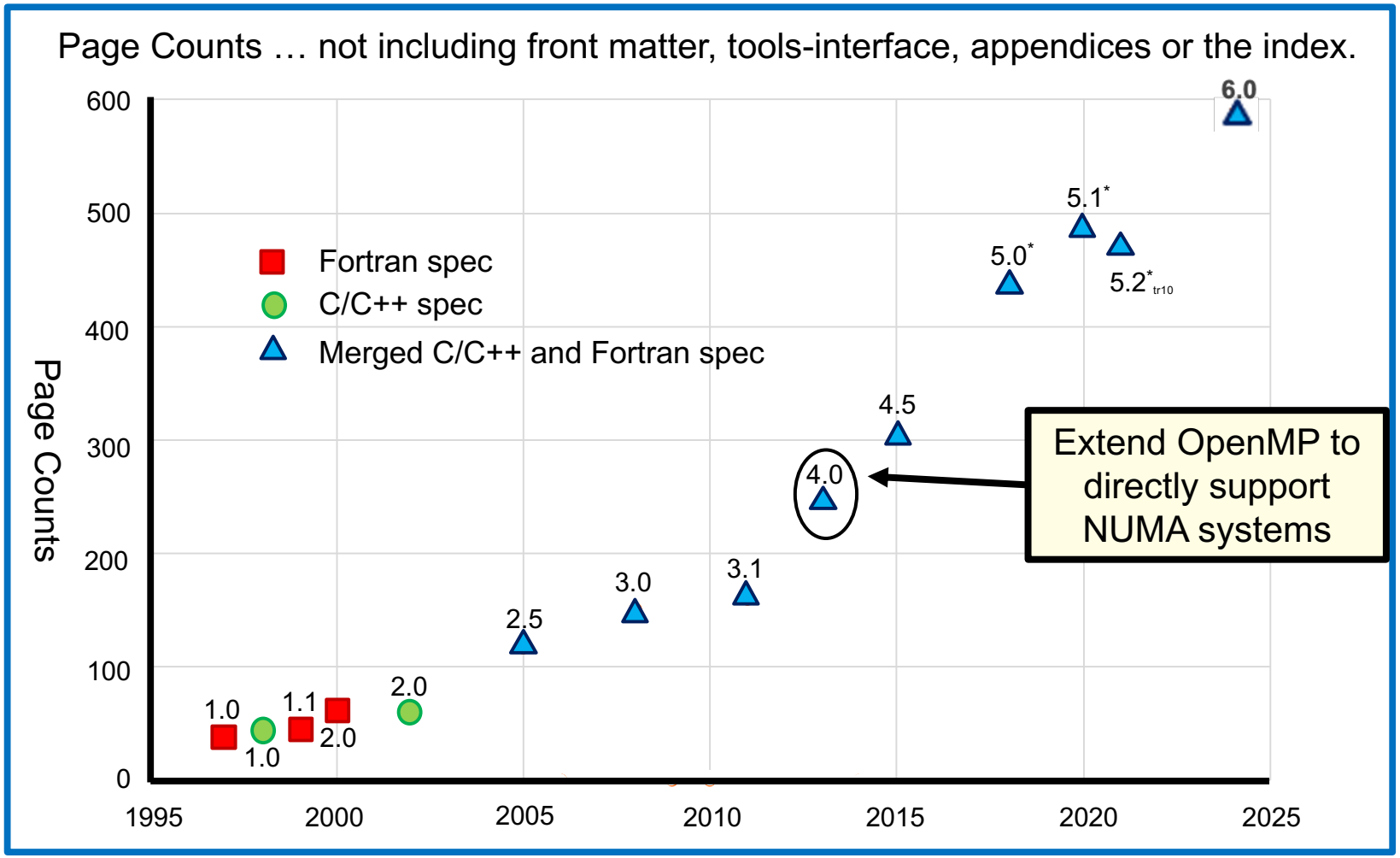


As configured for Cori at NERSC: CPUs at 2.3 GHz, 2 16 GB DIMMs per DDR memory controller, 16 cores per CPU. 2 CPUs connected by a high-speed interconnect (QPI)

Optimizing code for NUMA systems

The Growth of Complexity in OpenMP

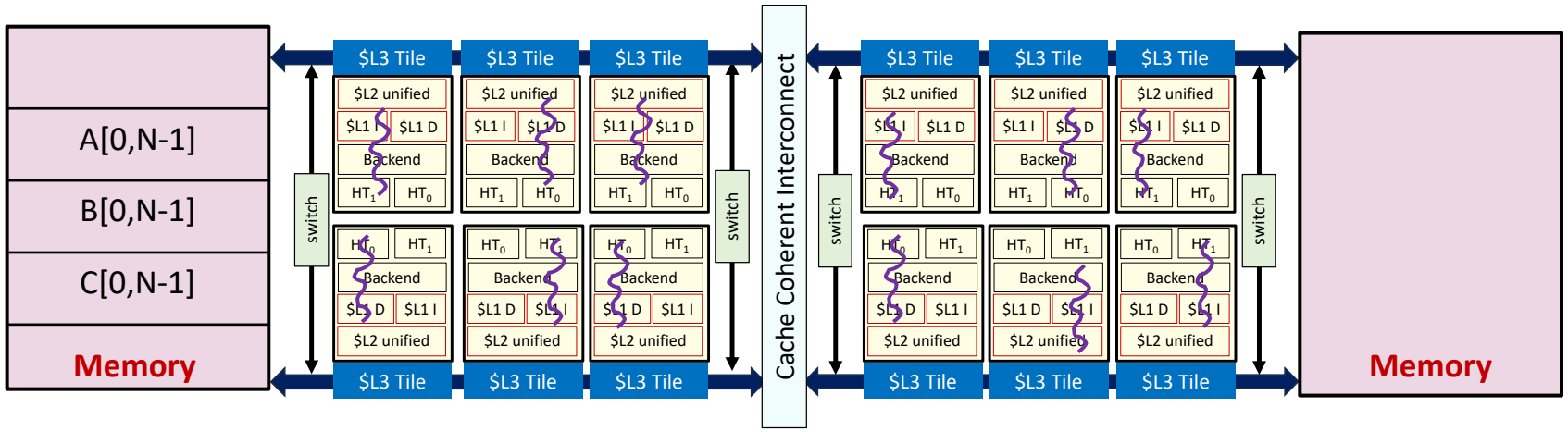
Our goal in 1997 … A simple interface for application programmers



Consider the Stream Triad Benchmark

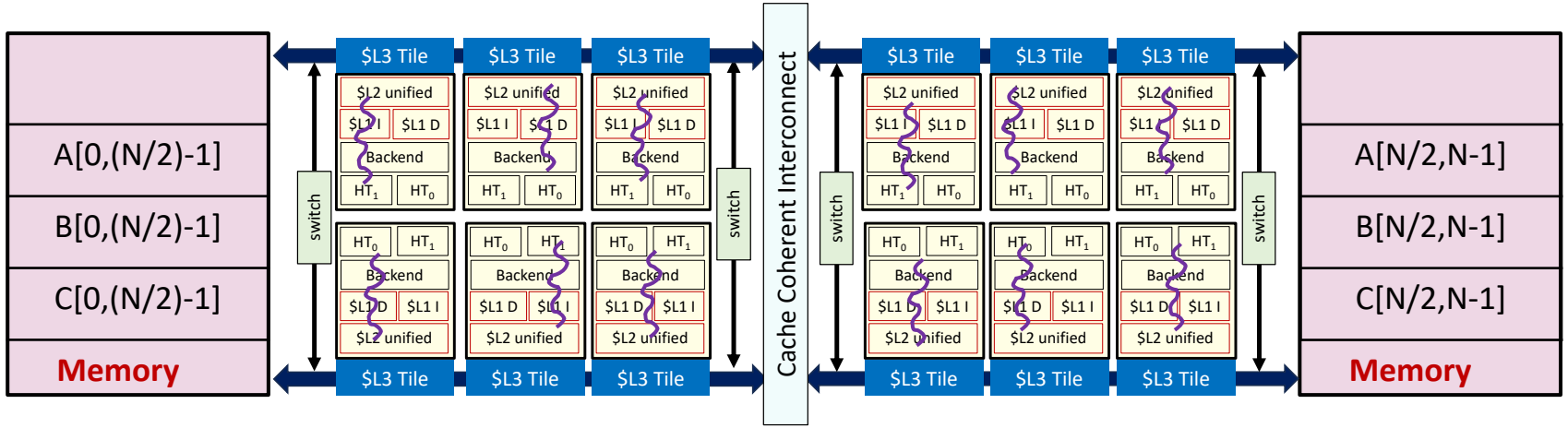
- Stream triad memory-bandwidth benchmark* running on a two-socket Intel® Xeon™ X5675 with 12 threads on 12 cores

```
// Stream Triad benchmark
#pragma omp parallel for
for (int i=0; i<N; i++)
    a[i] = b[i] + scalar * c[i];
```



3 arrays in one NUMA domain

triad
18.2 GB/s



Arrays split between both NUMA domains

triad
40.4 GB/s

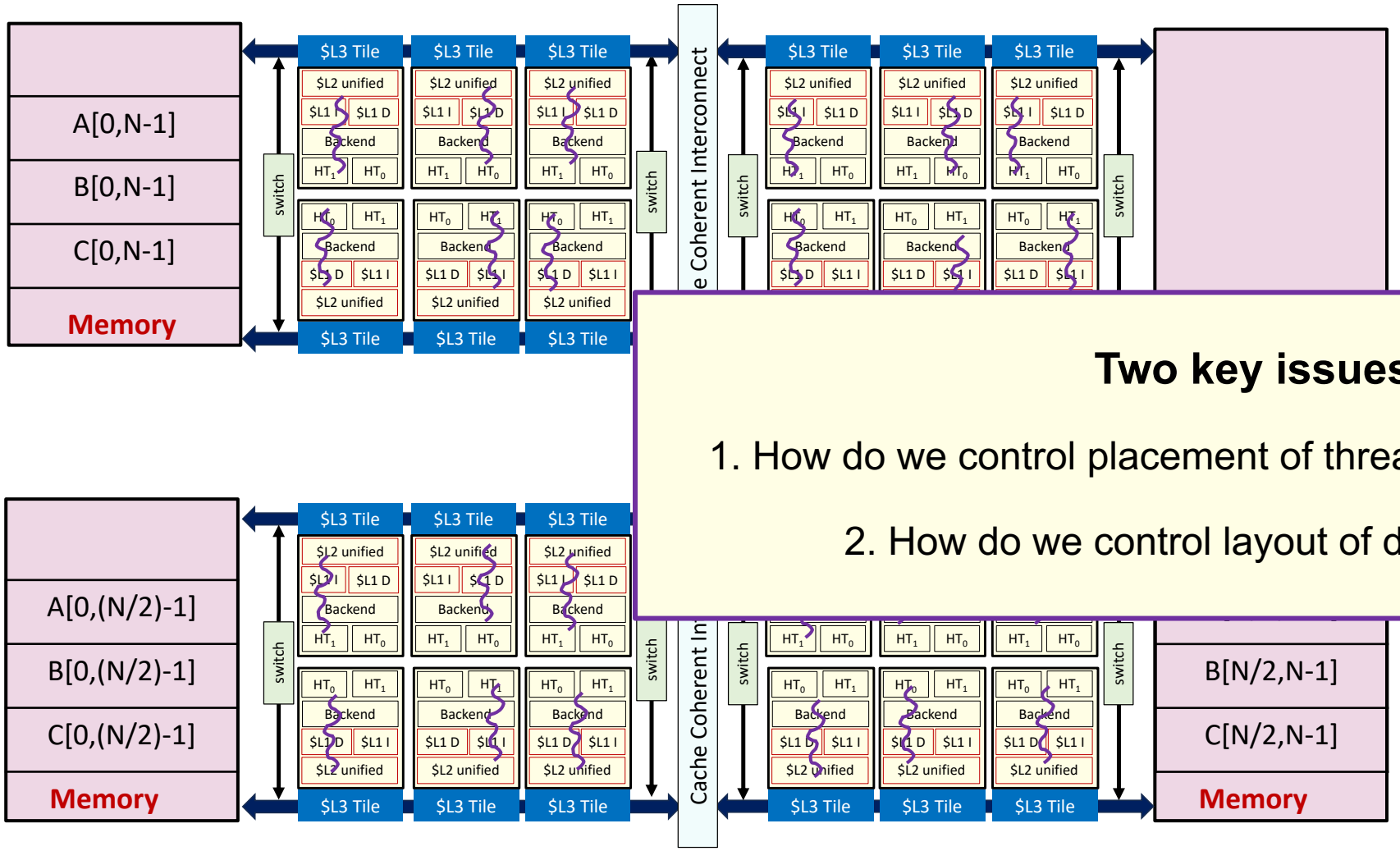
Based on content from Christian Terboven (RWTH/AACHEN) and Michael Klemm (OpenMP)

* We focus only on triad since results for the other benchmark components (copy, add, scale) matched Triad.

Consider the Stream Triad Benchmark

- Stream triad memory-bandwidth benchmark* running on a two-socket Intel® Xeon™ X5675 with 12 threads on 12 cores

```
// Stream Triad benchmark
#pragma omp parallel for
for (int i=0; i<N; i++)
    a[i] = b[i] + scalar * c[i];
```



3 arrays in one NUMA domain

triad
18.2 GB/s

Two key issues:

1. How do we control placement of threads in a NUMA system?

2. How do we control layout of data in memory?

Arrays split between both NUMA domains

triad
40.4 GB/s

Based on content from Christian Terboven (RWTH/AACHEN) and Michael Klemm (OpenMP)

* We focus only on triad since results for the other benchmark components (copy, add, scale) matched Triad.

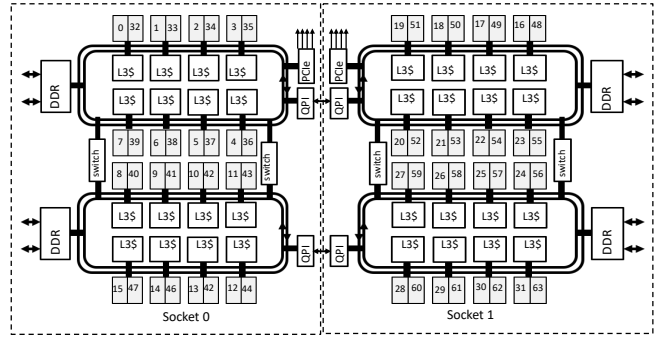
Controlling thread layout on a NUMA system:

1. How does the OS view components of processors?

2. Where can threads go (places) and how do you line up threads to places (affinity)?

Discover the OS view of virtual cores

- Hardware Locality tools …. hwloc-ls, lstopo, Numactl and others depending on the system. Generates text or graphical output depending on how the tools are configured on your system.



Machine (126GB total)

NUMANode P#0 (63GB)

Package P#0

L3 (40MB)

L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)
L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)
L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)
Core P#0	Core P#1	Core P#2	Core P#3	Core P#4	Core P#5	Core P#6	Core P#7	Core P#8	Core P#9	Core P#10	Core P#11	Core P#12	Core P#13	Core P#14	Core P#15
PU P#0	PU P#1	PU P#2	PU P#3	PU P#4	PU P#5	PU P#6	PU P#7	PU P#8	PU P#9	PU P#10	PU P#11	PU P#12	PU P#13	PU P#14	PU P#15
PU P#32	PU P#33	PU P#34	PU P#35	PU P#36	PU P#37	PU P#38	PU P#39	PU P#40	PU P#41	PU P#42	PU P#43	PU P#44	PU P#45	PU P#46	PU P#47

NUMANode P#1 (63GB)

Package P#1

L3 (40MB)

L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)	L2 (256KB)
L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)	L1d (32KB)
L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)	L1i (32KB)
Core P#0	Core P#1	Core P#2	Core P#3	Core P#4	Core P#5	Core P#6	Core P#7	Core P#8	Core P#9	Core P#10	Core P#11	Core P#12	Core P#13	Core P#14	Core P#15
PU P#16	PU P#17	PU P#18	PU P#19	PU P#20	PU P#21	PU P#22	PU P#23	PU P#24	PU P#25	PU P#26	PU P#27	PU P#28	PU P#29	PU P#30	PU P#31
PU P#48	PU P#49	PU P#50	PU P#51	PU P#52	PU P#53	PU P#54	PU P#55	PU P#56	PU P#57	PU P#58	PU P#59	PU P#60	PU P#61	PU P#62	PU P#63

Graphical output for a dual Socket node with Intel® Xeon™ E5-2698v3 CPUs

PU: processor unit. The smallest physical execution unit that hwloc recognizes.

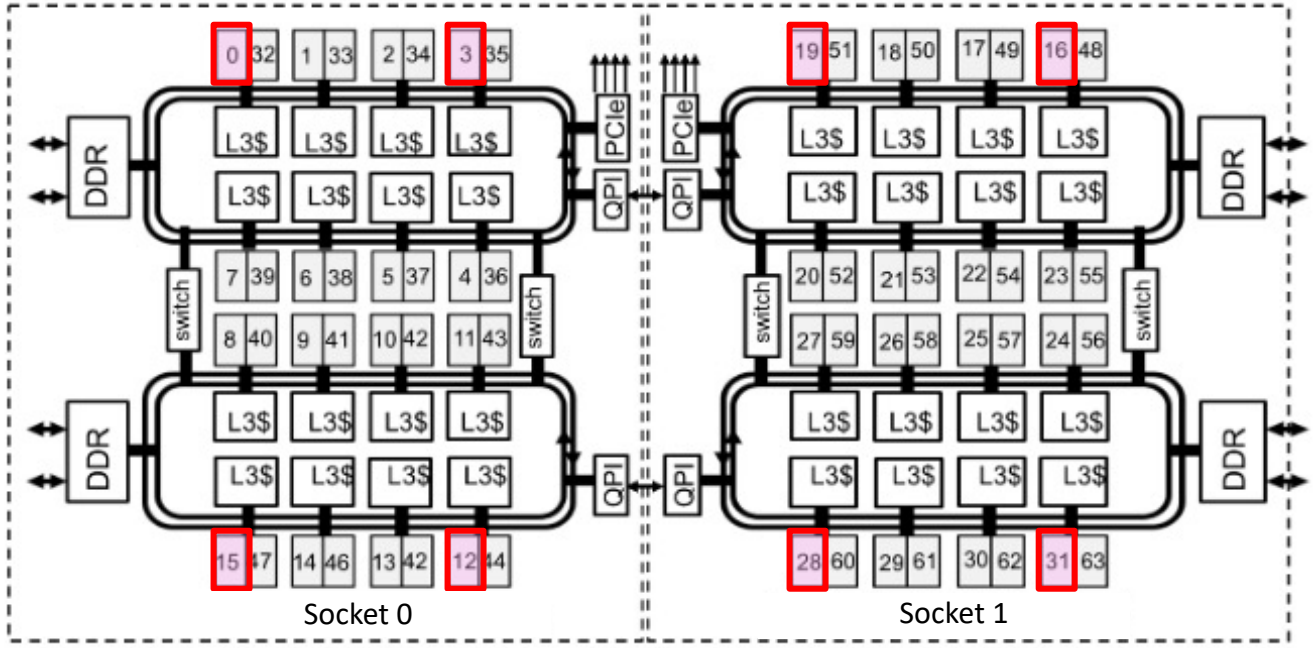
Based on content from Yun (Helen) He from NERSC)

Using OMP_PLACES to select where to map threads

- Use a tool to learn the logical core IDs (processor units or PUs) as understood by the OS.
- OpenMP defines an environment variable (OMP_PLACES) to define where threads can be scheduled to execute.

```
> export OMP_PLACES="{0, 3, 15, 12, 19,  16, 28, 31}"
> export NUM_THREADS= 6
```

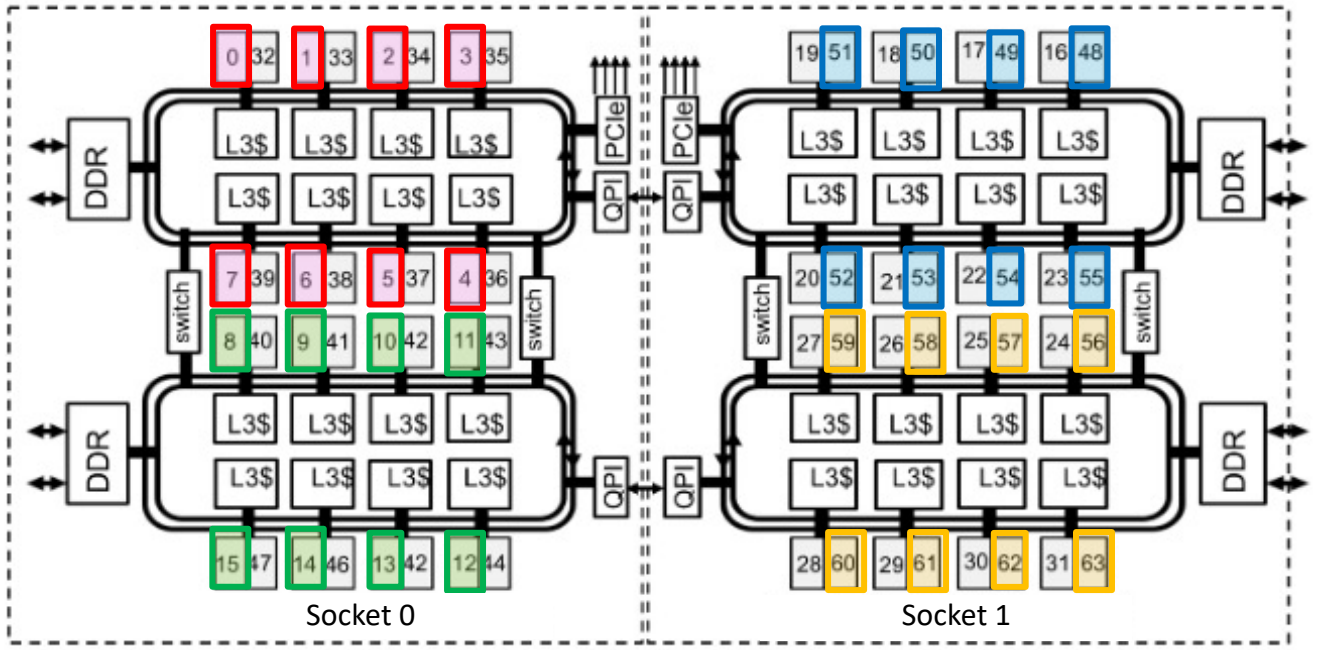
You can't control where the first thread goes, but the others will start at that first thread and wrap around the set of places



Using OMP_PLACES to select where to map threads

- Use a tool to learn the logical core IDs (processor units or PUs) as understood by the OS.
- OpenMP defines an environment variable (OMP_PLACES) to define where threads can be scheduled to execute.
- We can organize places into partitions to capture the structure of the NUMA system.
- In the following, we have one places partition per NUMA domain*

> export OMP_PLACES="{0:7:1},{8:15:1},{48:55:1},{56:63:1}"



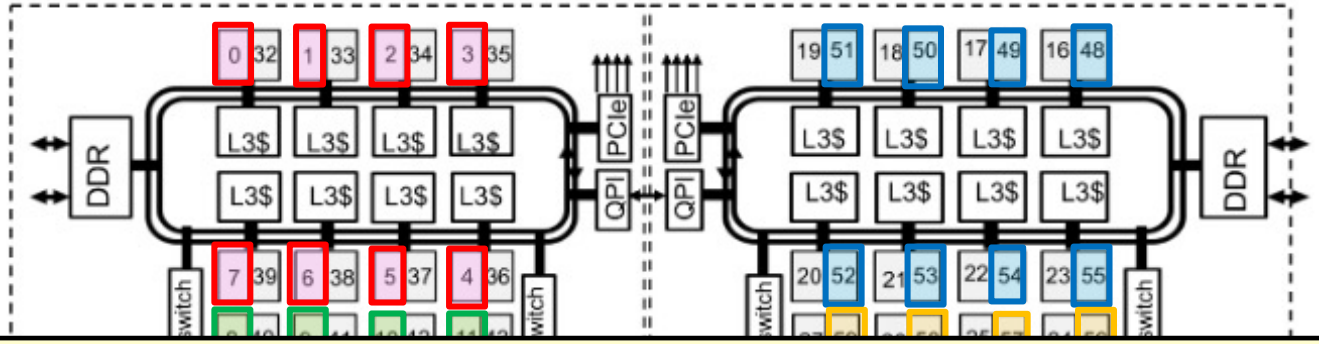
* Note that {0:7:1}. is equivalent to {0, 1, 2, 3, 4, 5, 6, 7}.

The details of what constitutes a NUMA domain is OS dependent … it depends on how the OS maps memory pages onto DRAM and how caches are managed.

Using OMP_PLACES to select where to map threads

- Use a tool to learn the logical core IDs (processor units or PUs) as understood by the OS.
- OpenMP defines an environment variable (OMP_PLACES) to define where threads can be scheduled to execute.
- We can organize places into partitions to capture the structure of the NUMA system.
- In the following, we have one places partition per NUMA domain*

> export OMP_PLACES="{0:7:1},{8:15:1},{48:55:1},{56:63:1}"



Programmers can use OMP_PLACES for detailed control over the execution-units threads utilize. BUT …

- The rules for mapping onto physical execution units are complicated.
- PLACES expressed as numbers is non-portable

There has to be an easier and more portable way to describe places

* Note that {0:7:1}. is equivalent to {0, 1, 2, 3, 4, 5, 6, 7}.

The details of what constitutes a NUMA domain is OS dependent … it depends on how the OS maps memory pages onto DRAM and how caches are managed.

Hardware Abstraction and OMP_PLACES

- OMP_PLACES can use an abstract name instead of a list of numbers
  - threads: each place corresponds to a single hardware thread
  - cores: each place corresponds to a single core (which may have one or more hardware threads)
  - sockets: each place corresponds to a single socket (consisting of one or more cores)

- Examples:
  - export OMP_PLACES=threads
  - export OMP_PLACES=cores

Mapping threads to places: Thread affinity with OMP_PROC_BIND

- OMP_PROC_BIND controls thread affinity within and between OpenMP places
- Allowed values:
  - true: the runtime will not move threads around between processors
  - false: the runtime may move threads around between processors
  - close: bind threads close to the primary* thread
  - spread: bind threads as evenly distributed as possible (i.e., spread them out)
  - primary: bind threads to the same place as the primary thread
- The values **primary***, **close,** and **spread** imply the value **true**

Example:

```
export OMP_PROC_BIND=spread
```

*Primary thread: this is the thread with ID=0 that encountered the parallel construct and created the team of threads

Mapping threads to places: Thread affinity with proc_bind() clause

- The proc_bind() clause controls thread affinity within and between OpenMP places
- Allowed values:
  - true: the runtime will not move threads around between processors
  - false: the runtime may move threads around between processors
  - close: bind threads close to the primary* thread
  - spread: bind threads as evenly distributed as possible (i.e., spread them out)
  - primary: bind threads to the same place as the primary thread
- The values **primary***, **close,** and **spread** imply the value **true**

Example … using clauses on a parallel construct:

```
#pragma omp parallel num_threads(4) proc_bind(spread)
```

*Primary thread: this is the thread with ID=0 that encountered the parallel construct and created the team of threads

Note: You **cannot** set places as a clause on a directive since the OS defines places when it launches the process to run a program

Examples: OMP_PROC_BIND

Assume 4 cores, 2 hardware threads per core, and 4 OpenMP threads

Set 4 place partitions: OMP_PLACES={0,1},{2,3},{4,5},{6,7}

- none: no affinity setting
- close: Bind threads as close to each other as possible

Node	Core 0		Core 1		Core 2		Core 3	
	HT_0	HT_1	HT_0	HT_1	HT_0	HT_1	HT_0	HT_1
	PU 0	PU 1	PU 2	PU 3	PU 4	PU 5	PU 6	PU 7
Thread	0	1	2	3				

- spread: Bind threads as far apart as possible

Node	Core 0		Core 1		Core 2		Core 3	
	HT_0	HT_1	HT_0	HT_1	HT_0	HT_1	HT_0	HT_1
	PU 0	PU 1	PU 2	PU 3	PU 4	PU 5	PU 6	PU 7
Thread	0		1		2		3	

- primary: bind threads to the same place as the primary thread

We have no control of where the initial thread is placed, but for these examples, we assume it is placed on HT_0 of Core 0.

With close, threads placed in consecutive locations

With spread, threads placed in first place of each partition

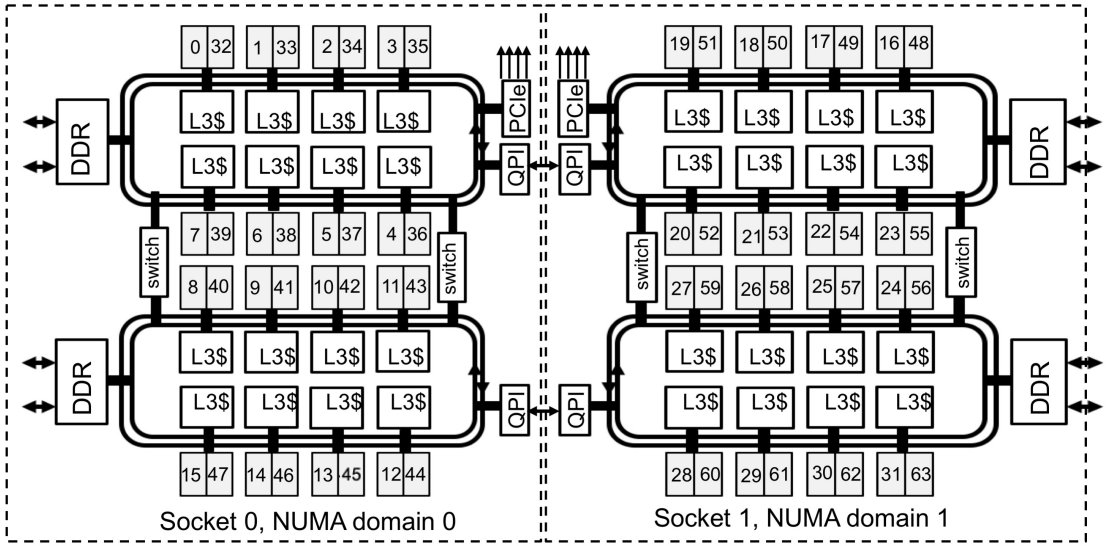
PU: processor unit. The smallest physical execution unit that hwloc recognizes.

OMP_PROC_BIND Choices for STREAM Benchmark

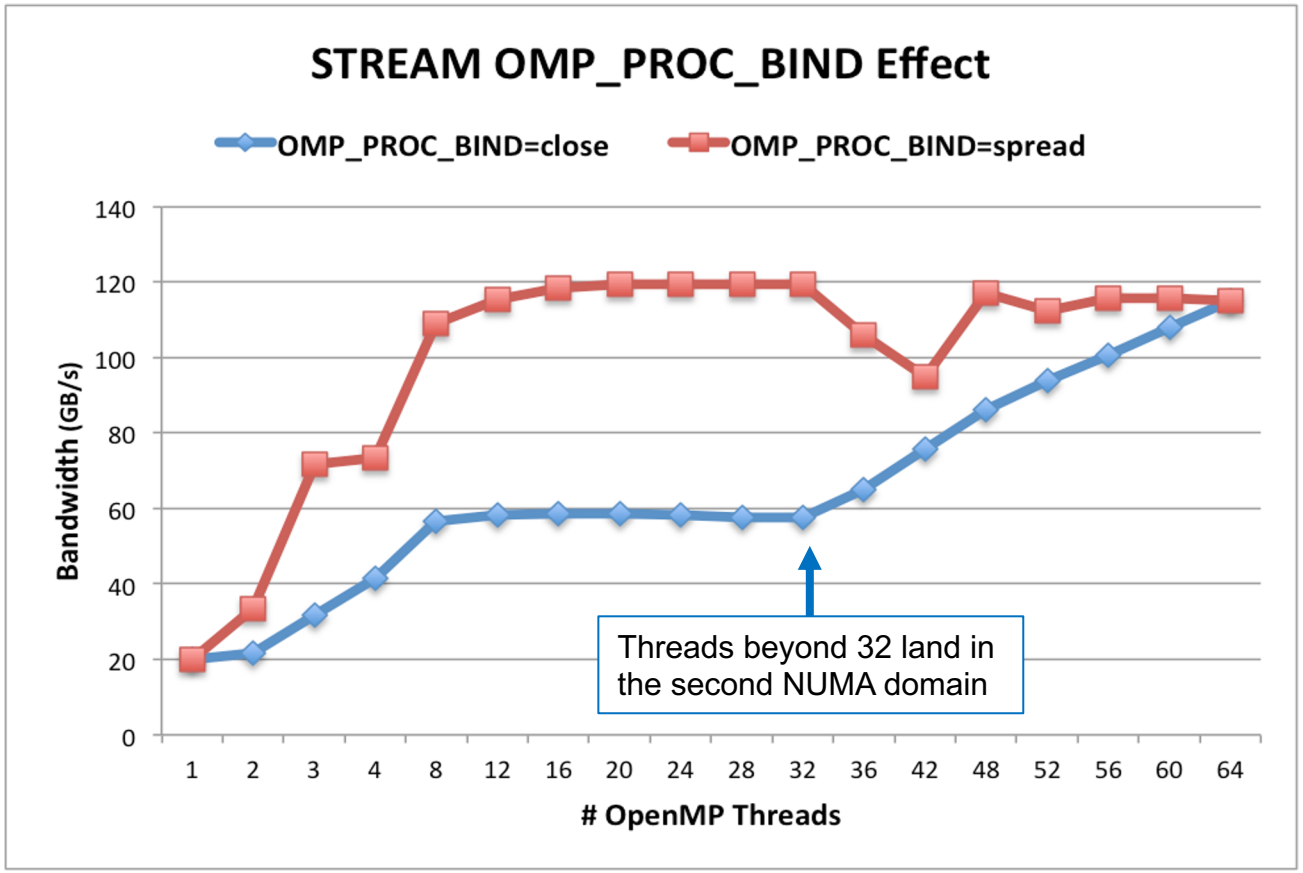
OMP_NUM_THREADS=32
OMP_PLACES=threads

OMP_PROC_BIND=close
Threads 0 to 31 bind to cores (0,32),(1,33),(2,34),…(15,47). All threads are in the first socket. The second socket is idle. Not optimal.

OMP_PROC_BIND=spread
Threads 0 to 31 bind to cores 0,1,2,… to 31. Both sockets and memory are used to maximize memory bandwidth.



Blue: OMP_PROC_BIND=close
Red: OMP_PROC_BIND=spread
In both cases, assume the arrays are spread out across all memory domains (a topic we'll address soon)



Stream is a well-known memory bandwidth benchmark based on simple vector operations on huge vectors

Based on content from Yun (Helen) He from NERSC)

Aligning memory to threads … First touch

Memory Affinity: Exploiting "First Touch" page mapping policy

Step 1.1 Initialization by primary thread only

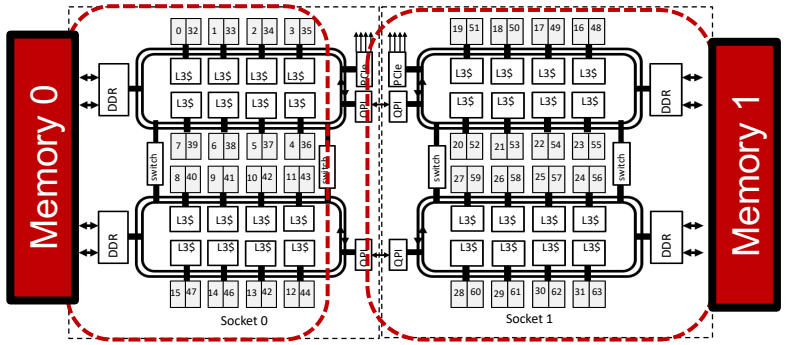
```
for (j=0; j<VectorSize; j++) {
      a[j] = 1.0; b[j] = 2.0; c[j] = 0.0;}
```

Step 1.2 Initialization by all threads

```
#pragma omp parallel for
for (j=0; j<VectorSize; j++) {
      a[j] = 1.0; b[j] = 2.0; c[j] = 0.0;}
```

Step 2 Compute

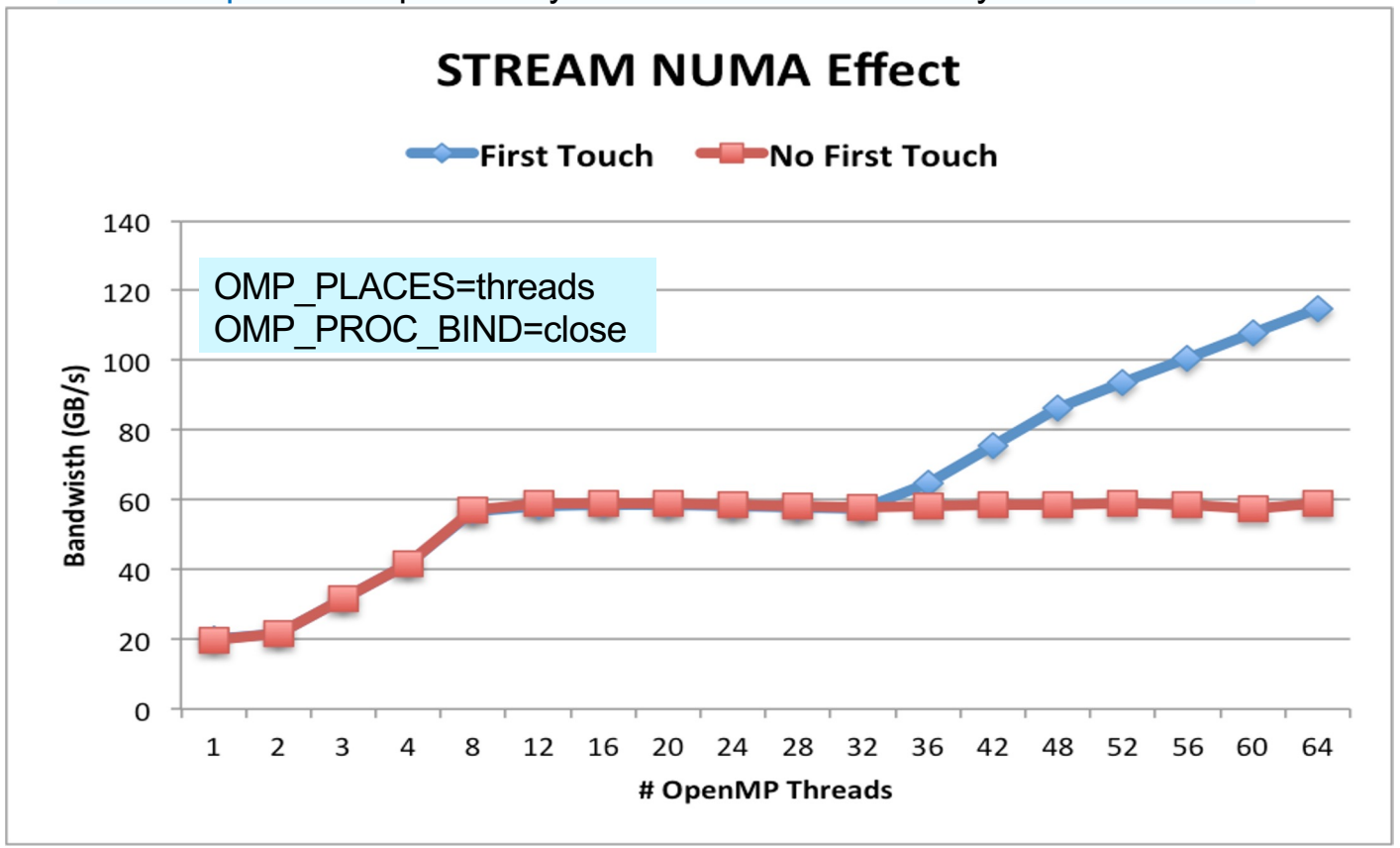
```
#pragma omp parallel for
for (j=0; j<VectorSize; j++)
      a[j]=b[j]+d*c[j];
```



A dual Socket node with Intel® Xeon™ E5-2698v3 CPUs

- The OS maps pages of memory based on a **first touch** policy.
- Hence, Affinity to memory is not defined when memory is allocated … it is defined when the memory is initialized.
- The result: memory is local to the thread that initializes it.

Red: step 1.1 + step 2. Arrays come from Memory 0 only
Blue: step 1.2 + step 2. Arrays come from both Memory domains



Based on content from Yun (Helen) He from NERSC)

Example: working with the First Touch Policy

Rember this slide?

OMP_PLACES="cores"

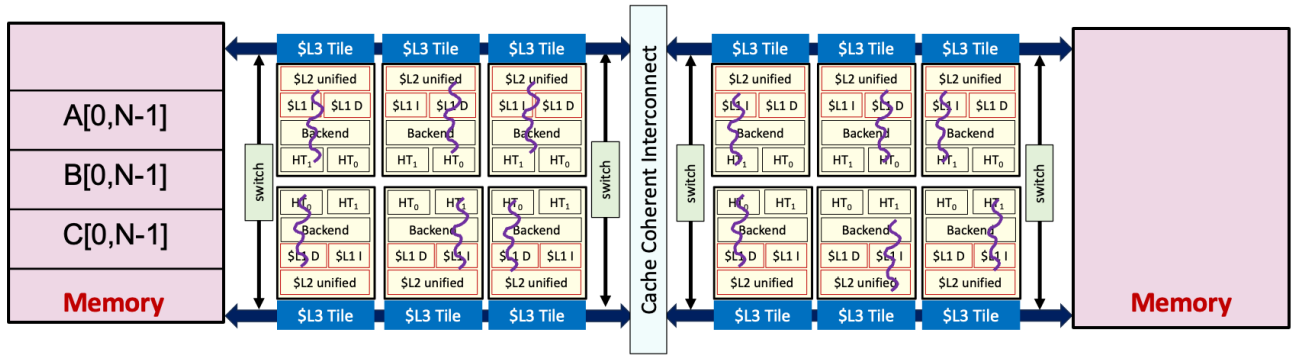
OMP_PROC_BIND="spread"

Consider the Stream Triad Benchmark

- Stream triad memory-bandwidth benchmark* running on a two-socket Intel® Xeon™ X5675 with 12 threads on 12 cores

```
// Stream Triad benchmark
#pragma omp parallel for
for (int i=0; i<N; i++)
    a[i] = b[i] + scalar * c[i];
```

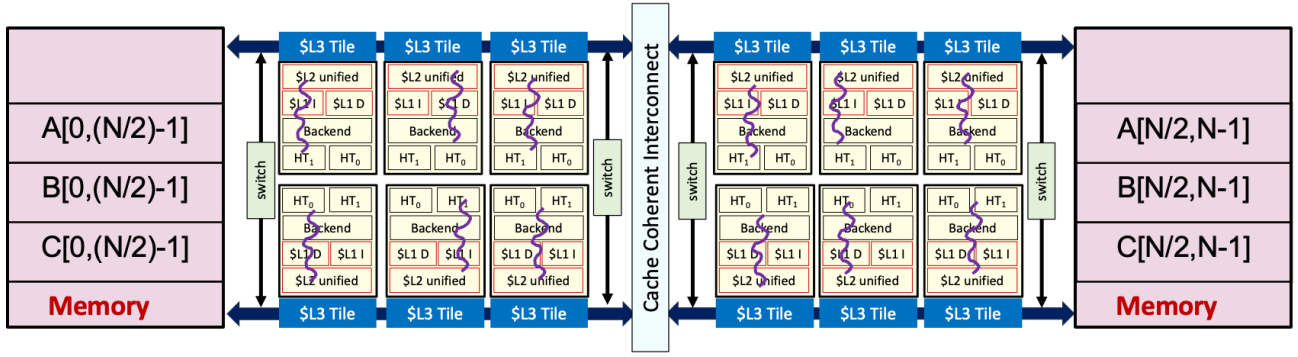
Arrays A, B, and C initialized on primary thread



3 arrays in one NUMA domain

triad
18.2 GB/s

Arrays A, B, and C initialized with the same "parallel for schedule" as used for the computation



Arrays split between both NUMA domains

triad
40.4 GB/s

Exercise: Matrix Transpose

- Consider the program transpose.c
  - Generates a square matrix, transposes the matrix and tests the result.
- This is all memory movement. Optimize the transpose operation:

```
export OMP_PROC_BIND=spread
#pragma omp parallel proc_bind(spread)
export OMP_PLACES="{0,21, 42, 64}"
export OMP_PLACES="{0:7:1},{8:15:1},{48:55:1},{56:63:1}"
export OMP_PLACES=threads

#pragma omp parallel
#pragma omp for
#pragma omp parallel for
#pragma omp for reduction(op:list)
#pragma omp critical
int omp_get_num_threads();     int omp_get_thread_num();   int omp_set_num_threads();
double omp_get_wtime();
```

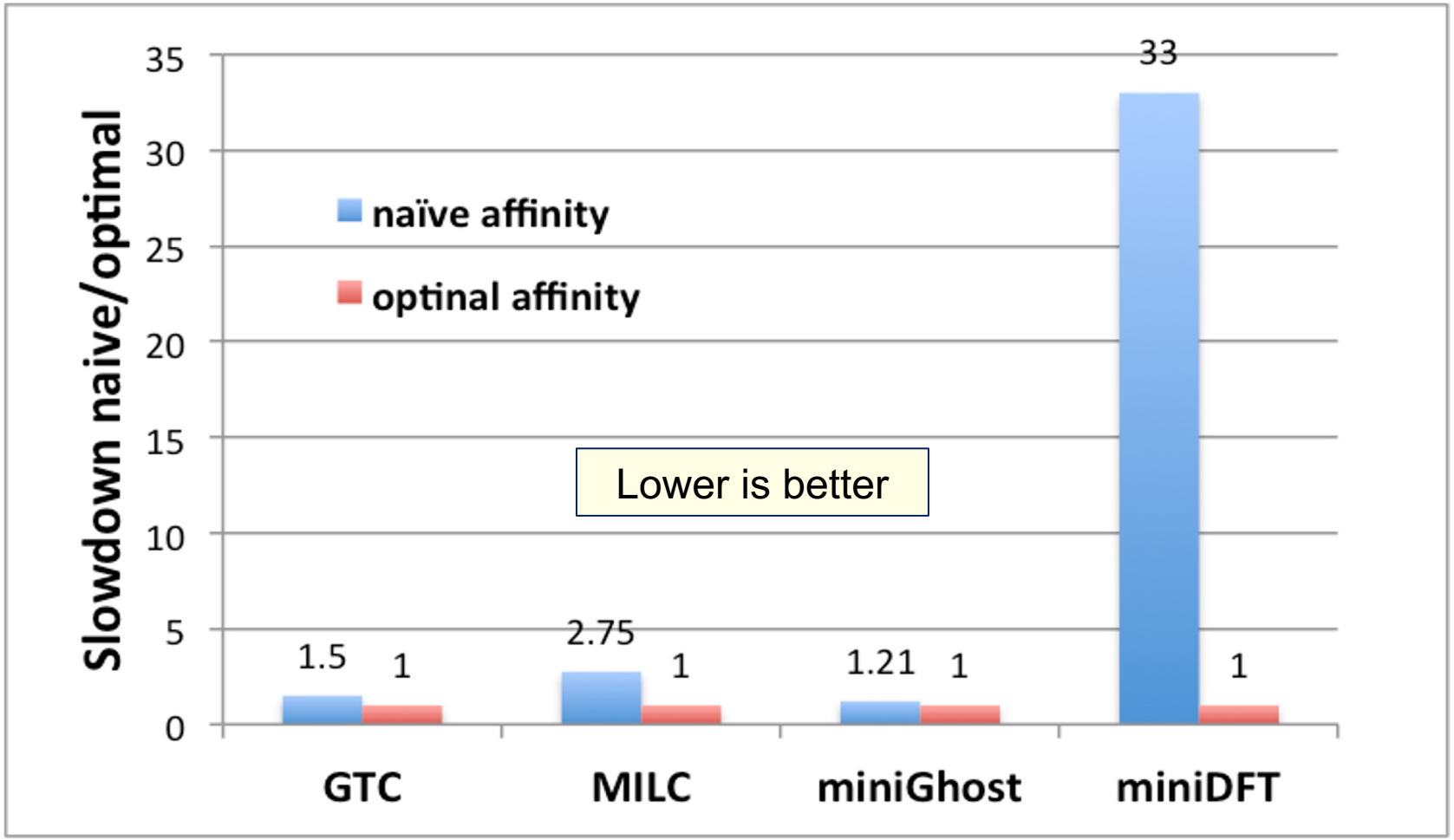
true, false, close, spread, primary

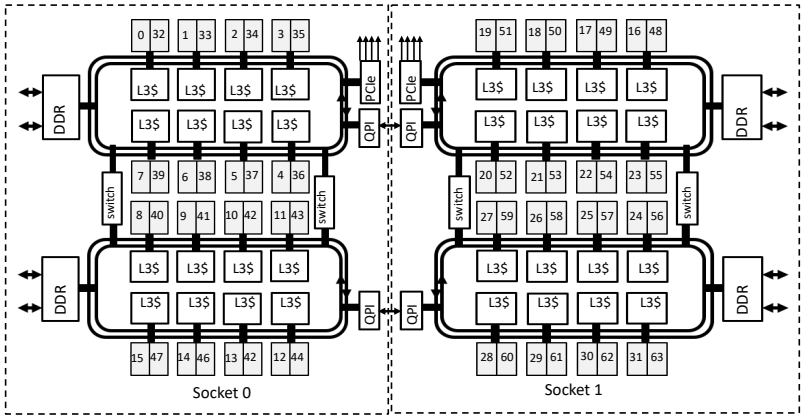
threads, cores, sockets

Wrapping up our discussion of taking NUMA features of a system into account in your multithreaded programs …

Getting the affinity right can have serious impacts on performance

Application Benchmark Performance for a number of benchmarks at NERSC





Results running on the Cori system at NERSC which has dual Socket nodes with Intel® Xeon™ E5-2698v3 CPUs

miniDFT: a DFT mini-app based on quantum expresso, miniGhost: 3D stencil, halo-exchange mini-app, MILC: MIMD Lattice QCD Computation

Based on content from Yun (Helen) He from NERSC

Finding the best strategy for thread affinity

- Experiment to find the best combinations of OMP_PLACES and OMP_PROC_BIND.
  - Using the environment variables makes it easy to try many options

- The best approach depends on the system but also on the features of an application
  - Putting threads for apart … on different sockets
    - May improve aggregate memory bandwidth available to an application
    - May improve combined cache size for the application
    - May increase synchronization overhead
  - Putting threads close together … on adjacent cores that may share some caches
    - May reduce synchronization overhead
    - May decrease memory bandwidth and total cache size

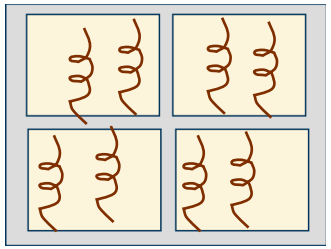
- Vendors have their own constructs for controlling NUMA features of a system.
  - Avoid vendor-specific constructs if you can … use portable OMP_PLACES and OMP_PROC_BIND

Outline



- Irregular Parallelism and tasks
- Other Systems for task-level parallelism: TBB
- Non-Uniform Memory Architectures (NUMA)
- Programming your GPU with OpenMP
- Recap

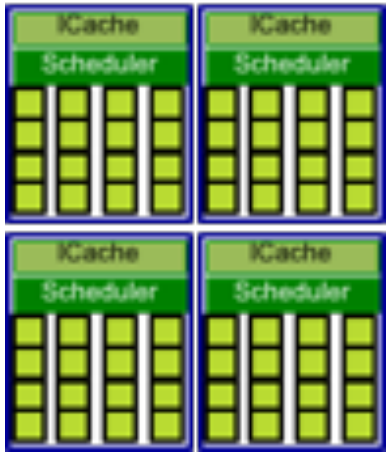
Hardware is diverse



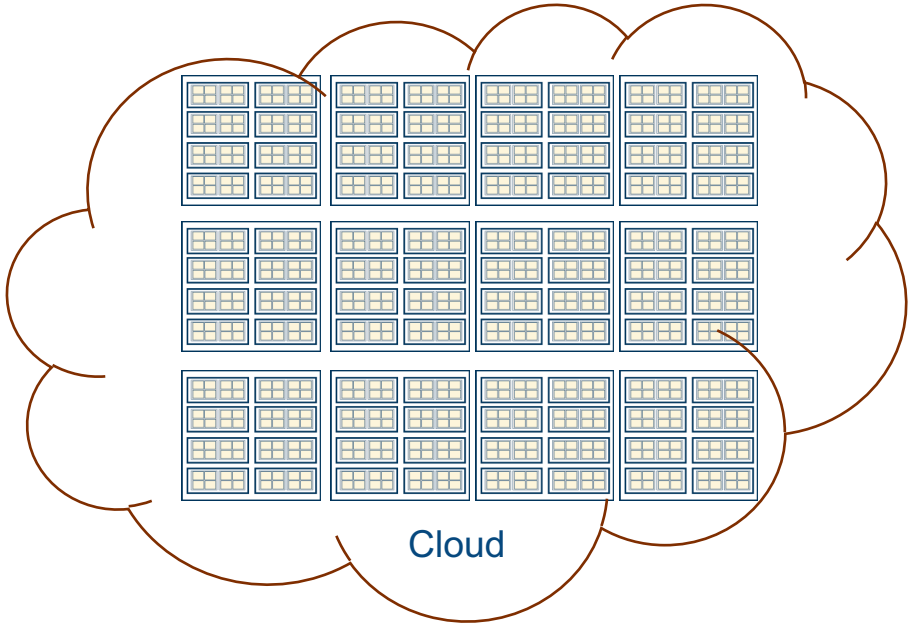
CPU



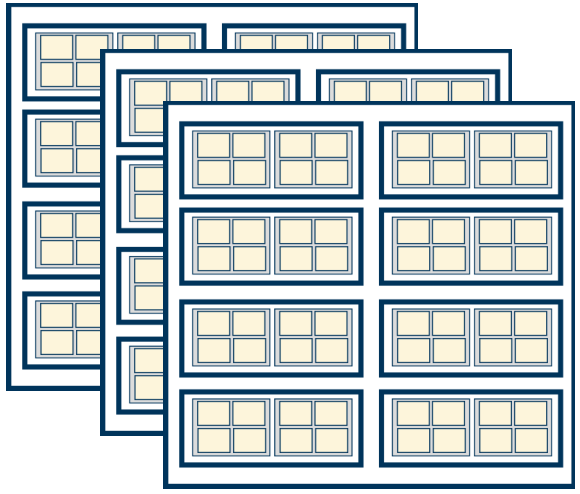
SIMD/Vector



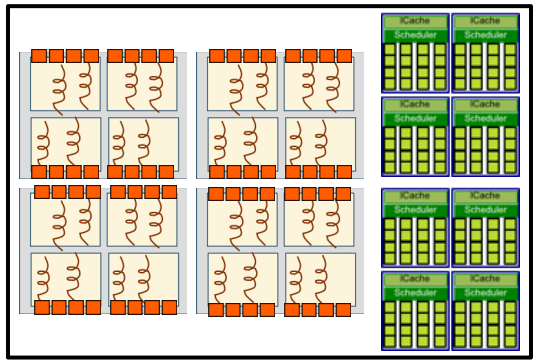
GPU



Cloud



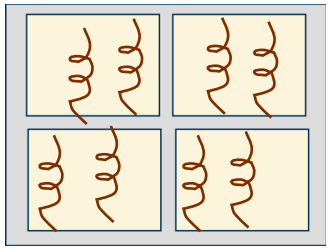
Cluster



Heterogeneous node

Hardware is diverse

You know all about programming with threads and a shared memories

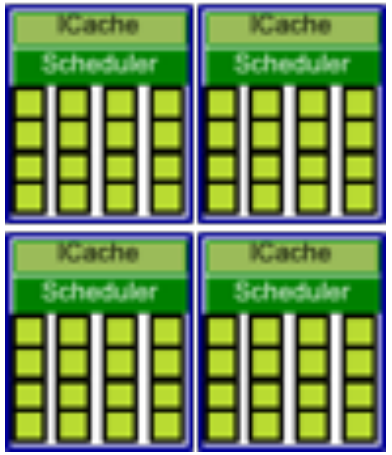


CPU

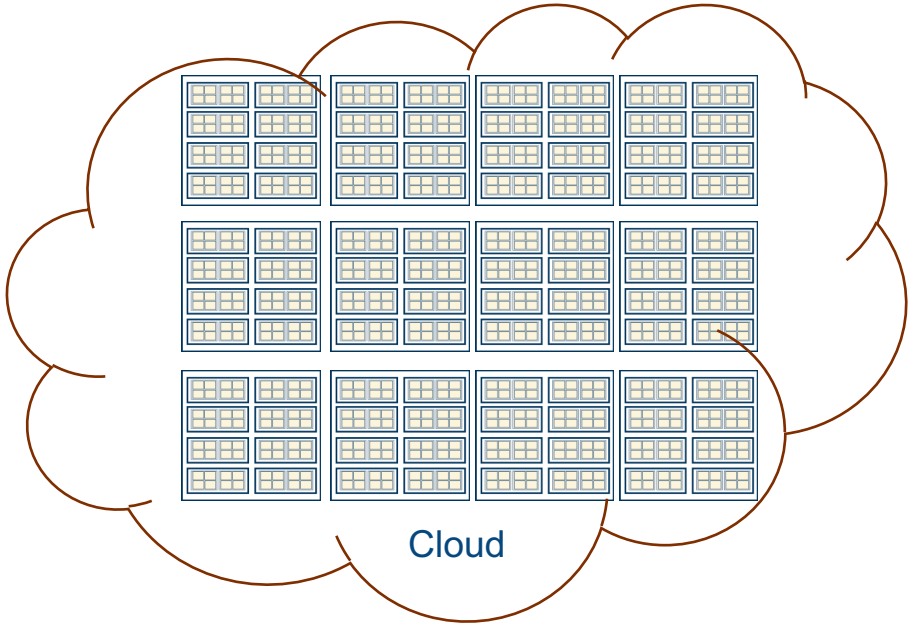
We'll leave vectorization to the compiler (use –O3)



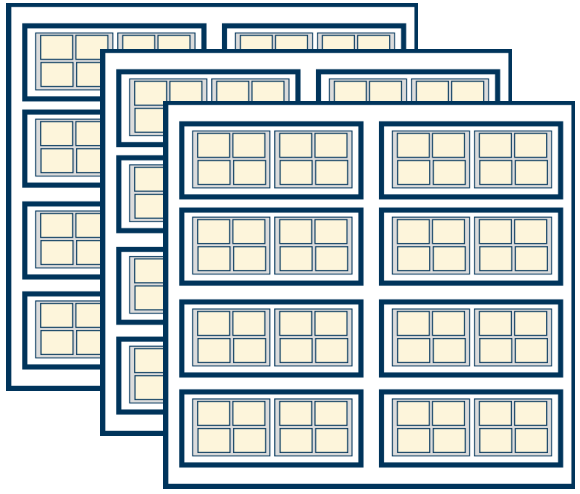
SIMD/Vector



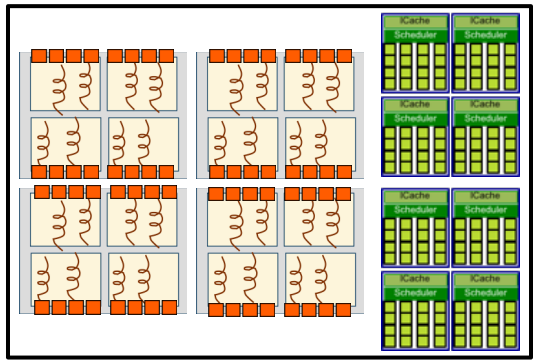
GPU



Cloud

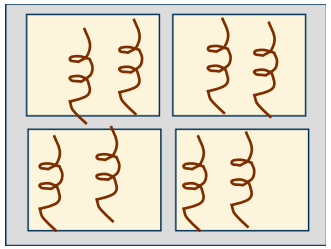


Cluster



Heterogeneous node

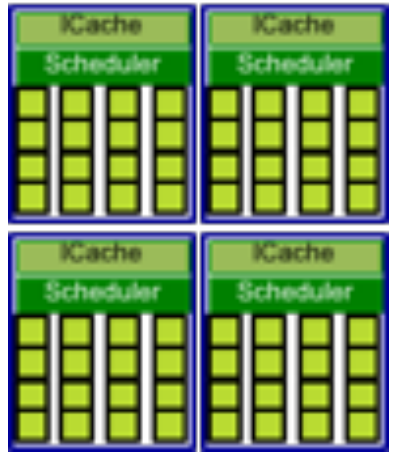
Hardware is diverse



CPU

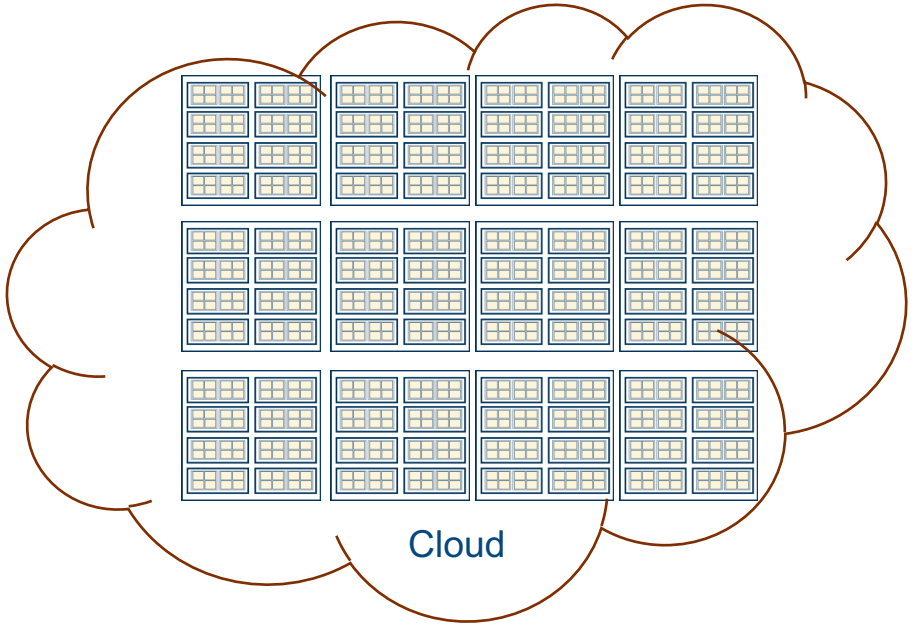


SIMD/Vector

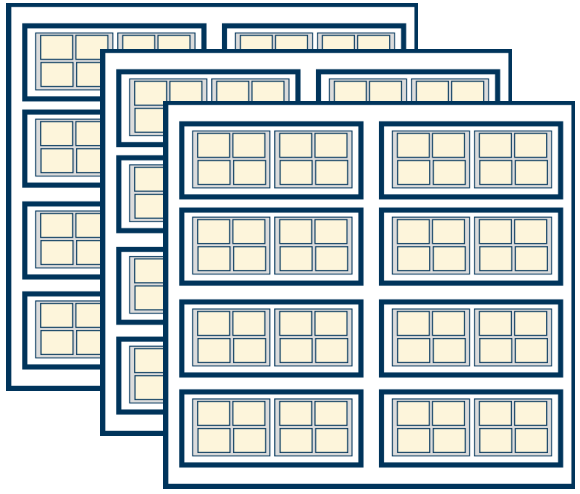


GPU

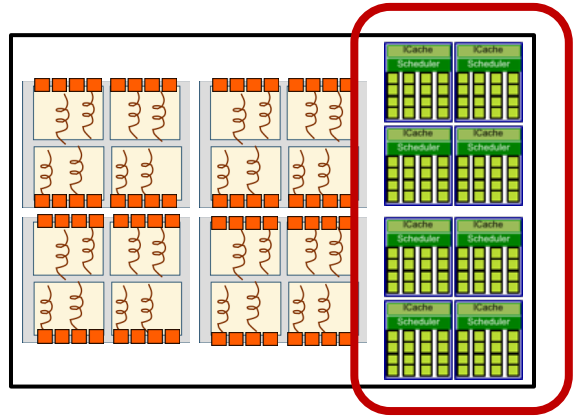
A very brief introduction to GPU programming



Cloud

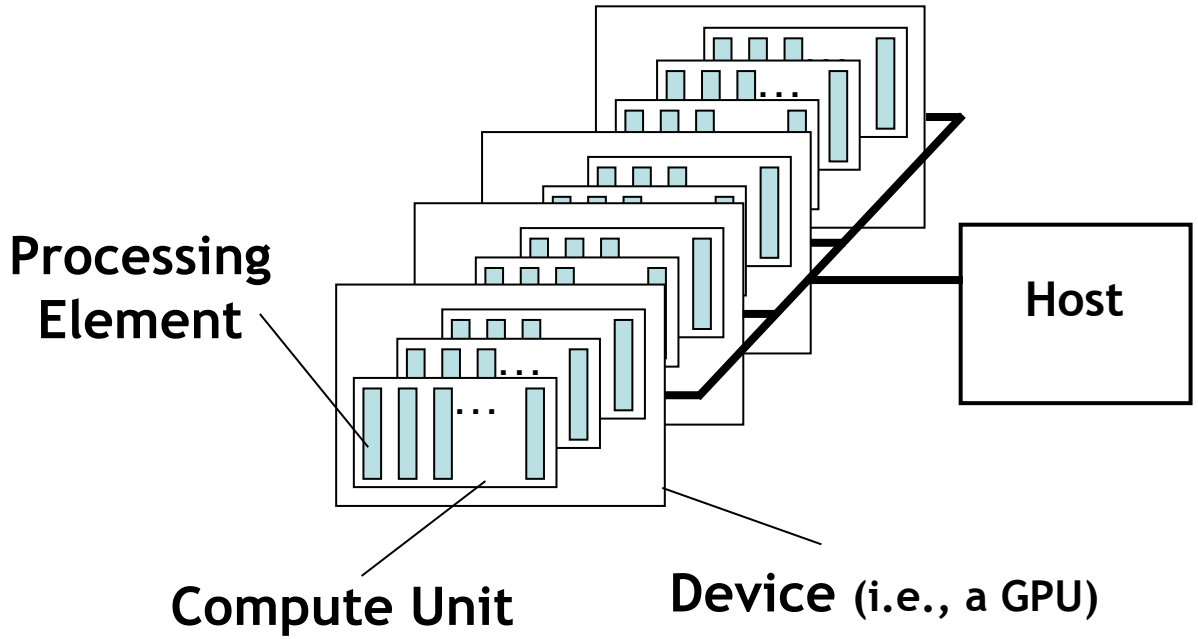


Cluster



Heterogeneous node

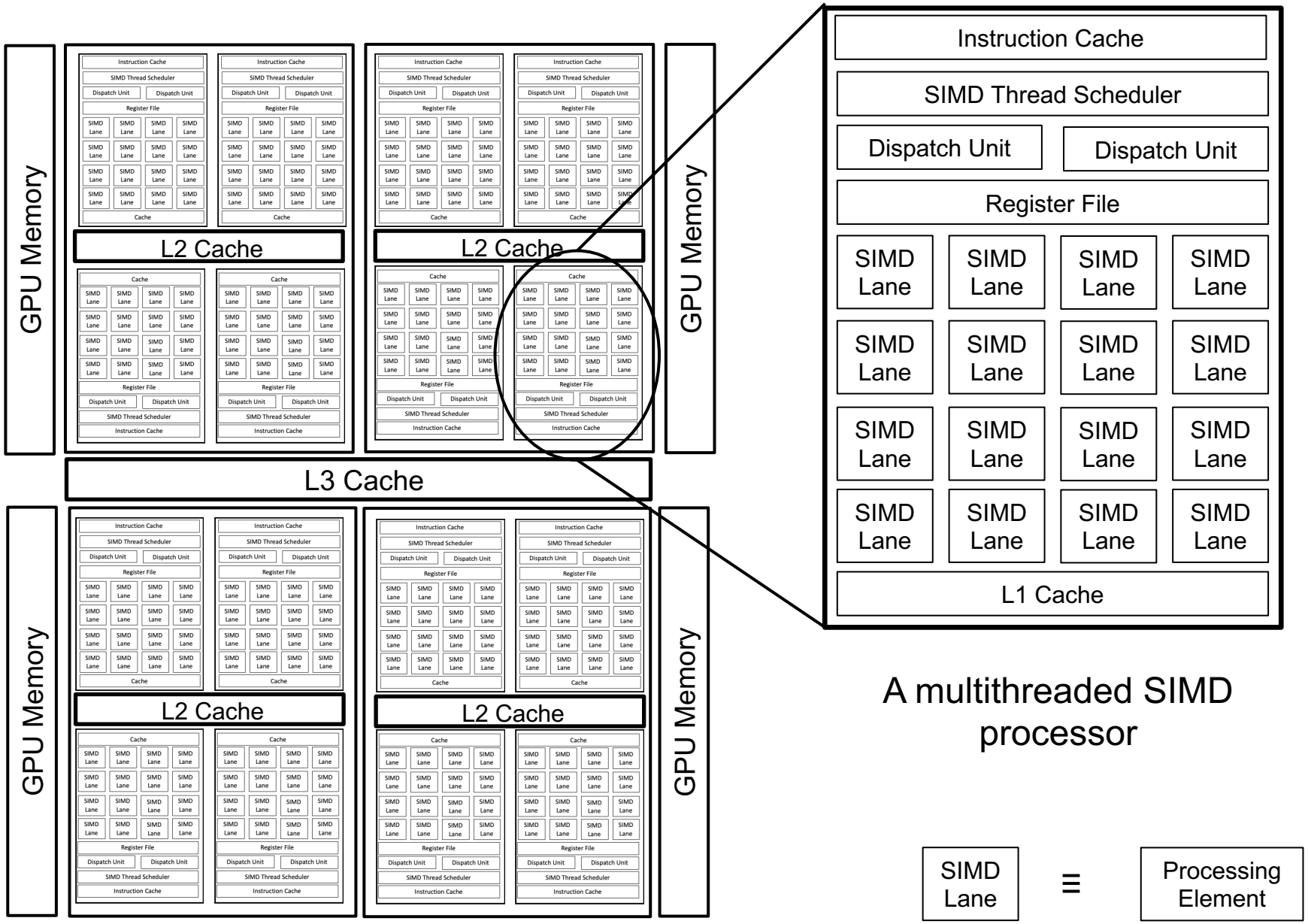
GPU Programming: A Host/Device Platform Model



- One ***Host*** and one or more ***Devices***
  - Each Device is composed of one or more Compute Units
  - Each Compute Unit is divided into one or more ***Processing Elements***

- Memory divided into ***host memory*** and ***device memory***

A Generic GPU (following Hennessey and Patterson)



A multithreaded SIMD processor

SIMD Lane ≡ Processing Element ≡ CUDA Core

Computer Architecture: A Quantitative Approach, John L. Hennessy and David A. Patterson.

The "BIG idea" Behind GPU programming

Traditional Loop based vector addition (vadd)

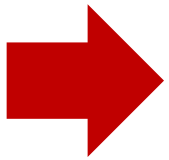
```
int main() {
   int N = . . . ;
   float *a, *b, *c;

   a* =(float *) malloc(N * sizeof(float));

   // ... allocate other arrays (b and c)
   // and fill with data

   for (int i=0;i<N; i++)
      c[i] = a[i] + b[i];

}
```



Data Parallel vadd with CUDA

```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c)
  // and fill with data

  // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

Assume a GPU with unified shared memory … allocate on host, visible on device too

How do we execute code on a GPU: The SIMT model (Single Instruction Multiple Thread)

1. Write kernel code for the scalar work-items

```
// Compute sum of order-N matrices: C = A + B
void __global__
matAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j<N) c[i][j] == a[i][j] + b[i][j];
}

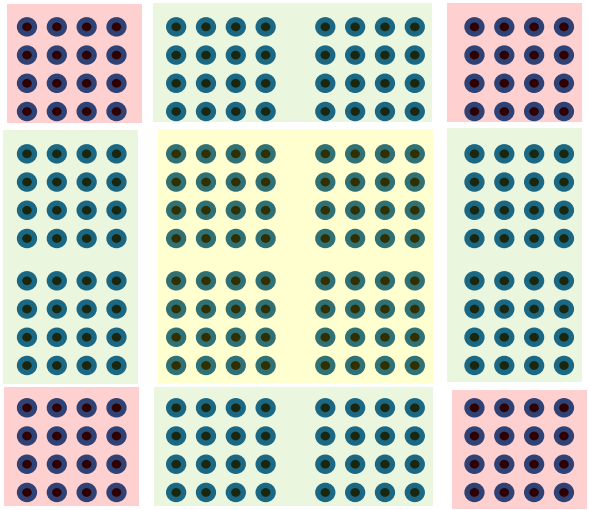
int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c)
  // and fill with data

  // define threadBlocks and the Grid
   dim3 dimBlock(4,4);
   dim3 dimGrid(4,4);

  // Launch kernel on Grid
    matAdd <<< dimGrid,dimBlock>>> (a, b, c, N);
}
```

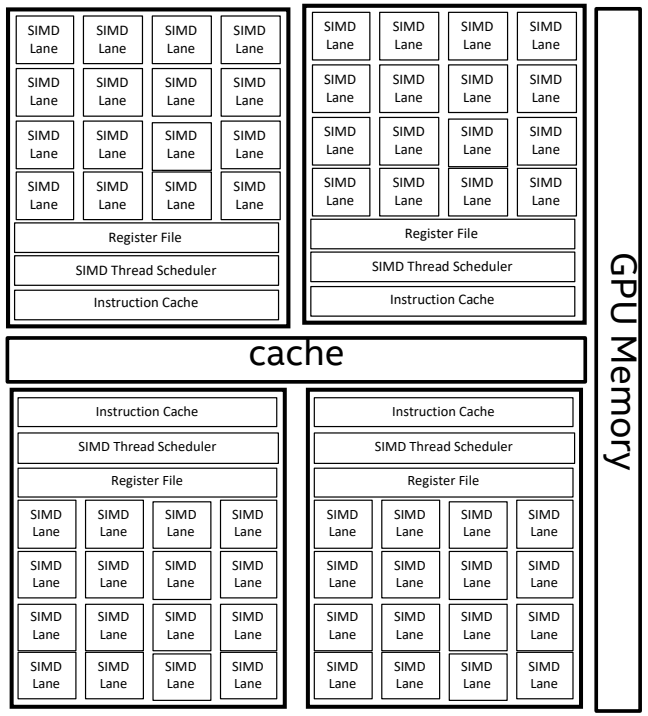
This is CUDA code

2. Map work-items onto an N dim index space.

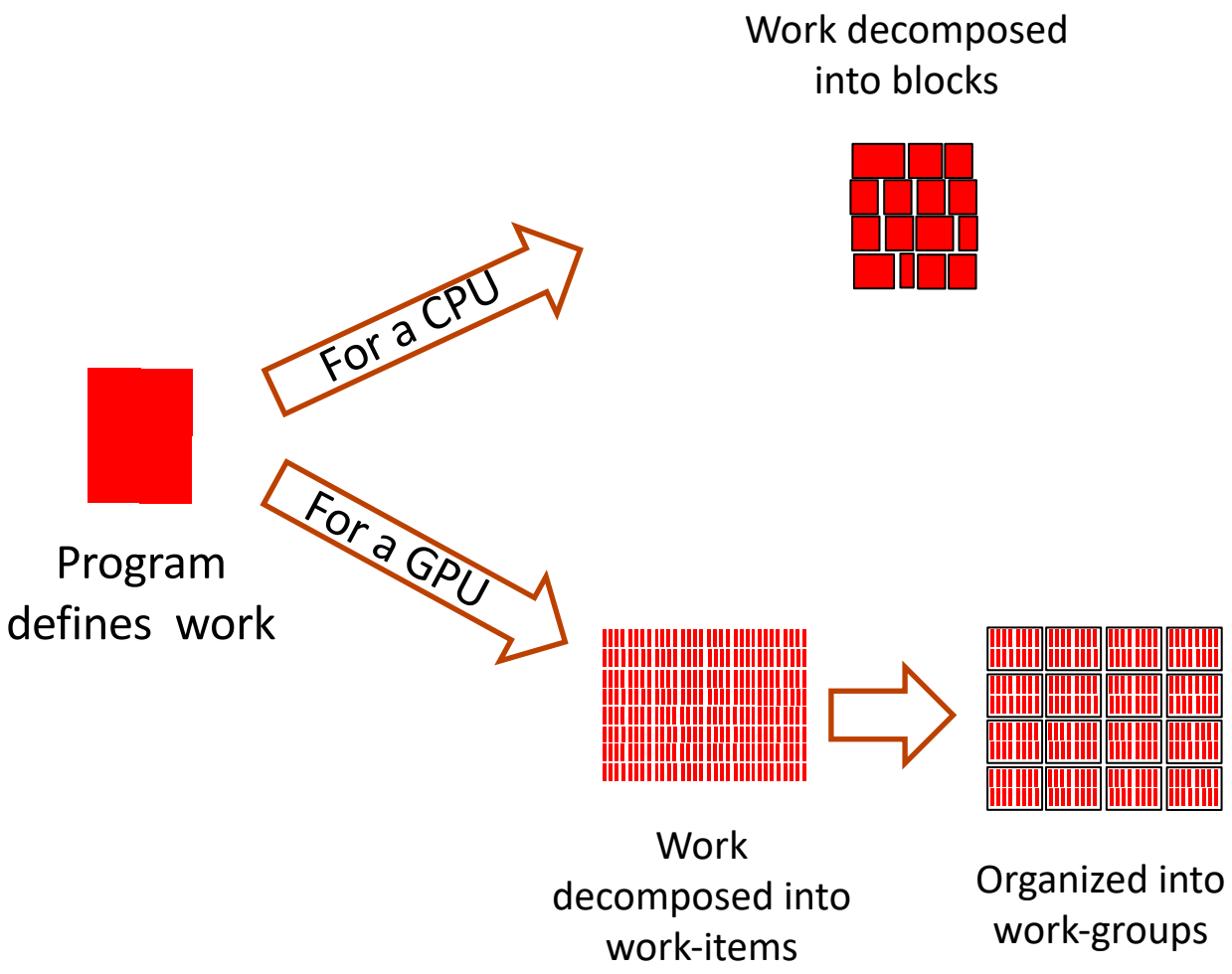


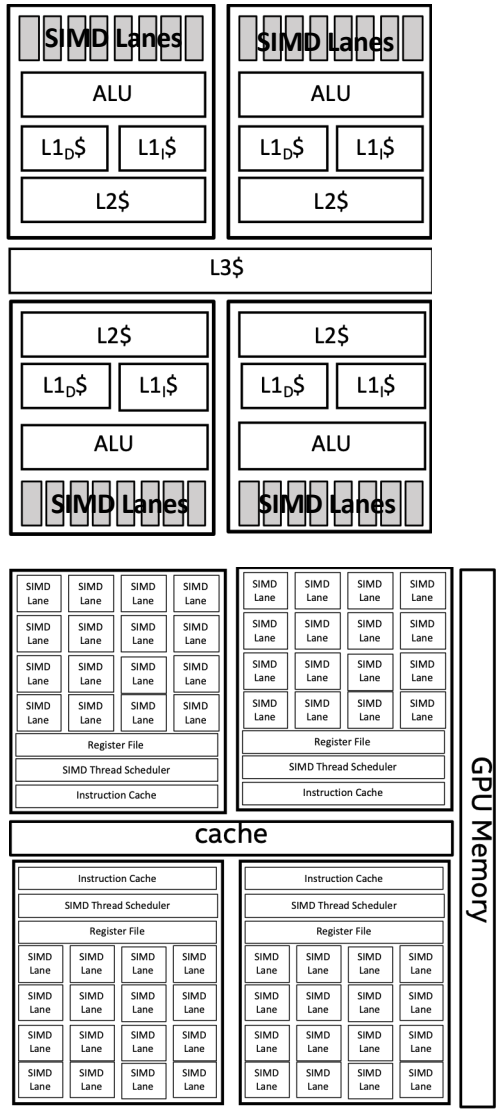
3. Map data structures onto the same index space

4. Run on hardware designed around the same SIMT execution model



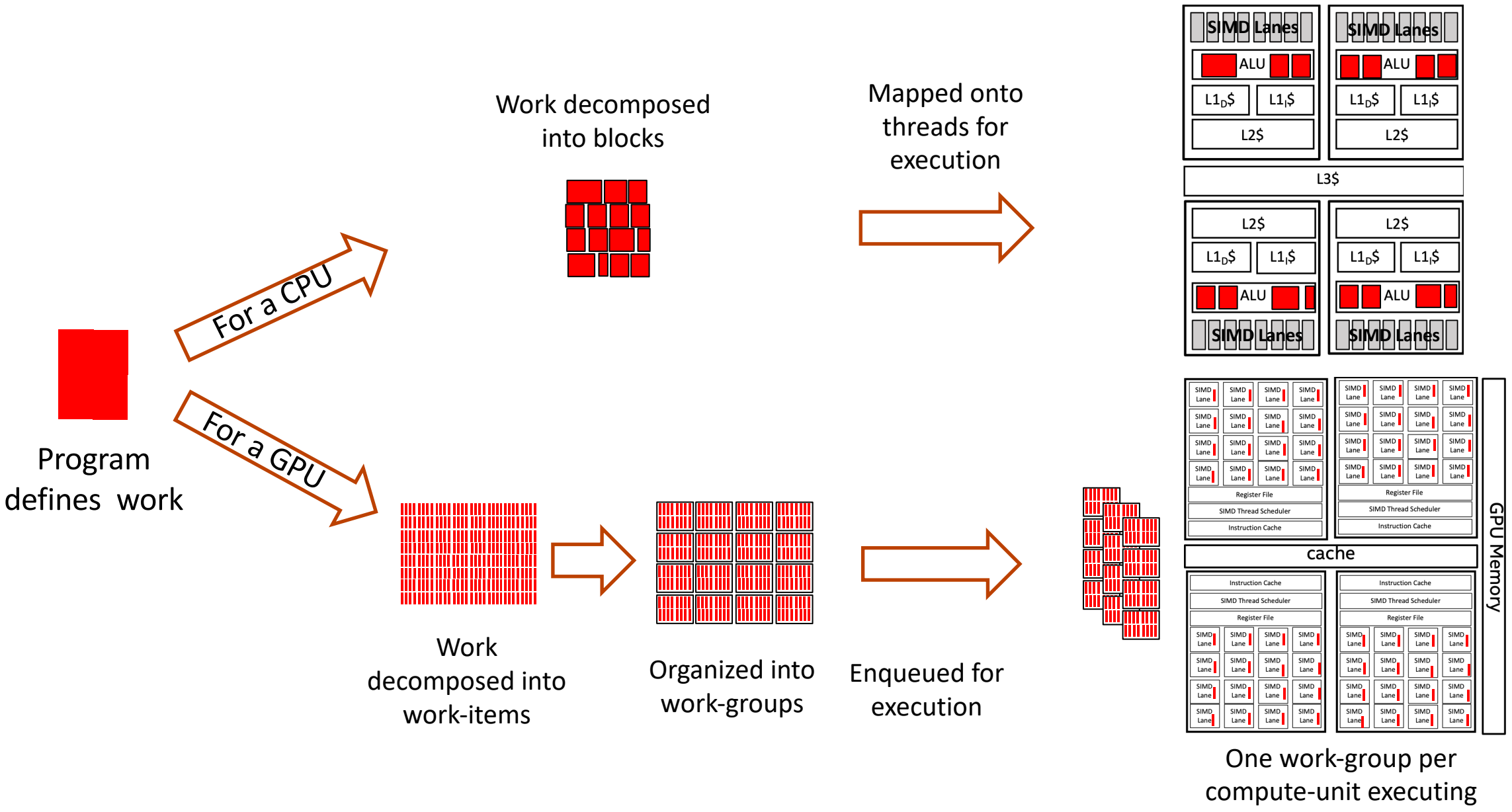
Executing a program on CPUs and GPUs





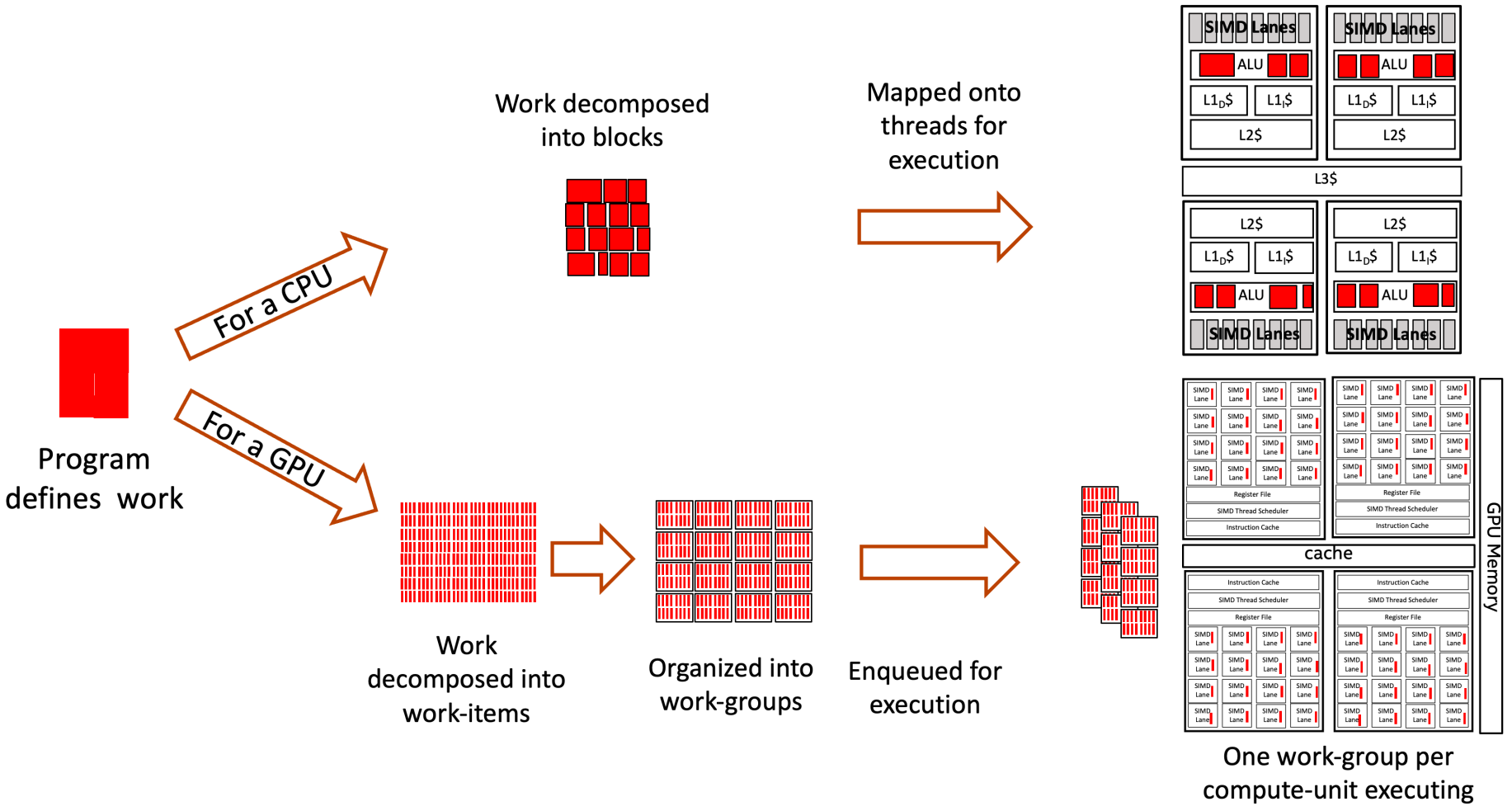
One work-group per compute-unit executing

Executing a program on CPUs and GPUs



CPU/GPU execution models

Executing a program on CPUs and GPUs



For a CPU, the threads are all active and able to make forward progress. Optimized for latency sensitive problems

For a GPU, any given work-group might be in the queue waiting to execute. Optimized for high throughput workflows

5-point stencil: the heat program

- The heat equation models changes in temperature over time.

$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0$$

- We'll solve this numerically on a computer using an explicit **finite difference** discretisation.
- $u = u(t, x, y)$ is a function of space and time.
- Partial differentials are approximated using diamond difference formulae:

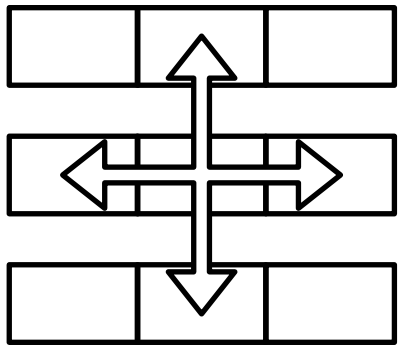
$$\frac{\partial u}{\partial t} \approx \frac{u(t+1, x, y) - u(t, x, y)}{dt}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x+1, y) - 2u(t, x, y) + u(t, x-1, y)}{dx^2}$$

  - Forward finite difference in time, central finite difference in space.

5-point stencil: the heat program

- Given an initial value of u, and any boundary conditions, we can calculate the value of u at time t+1 given the value at time t.
- Each update requires values from the north, south, east and west neighbours only:



- Computation is essentially a weighted average of each cell and its neighbouring cells.
- If on a boundary, look up a boundary condition instead.

Serial CPU code

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
 const double r2 = 1.0 - 4.0*r;

 // malloc and initialize u_tmp and u (code not shown)

 for (int t = 0; t < nsteps; ++t) {

    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]          +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
 }
}
```

Loop over time steps

Loop over NxN spatial domain

Update the 5-point stencil. Boundary conditions on the edges of the domain are fixed at zero.

Parallel CPU code, n=4000

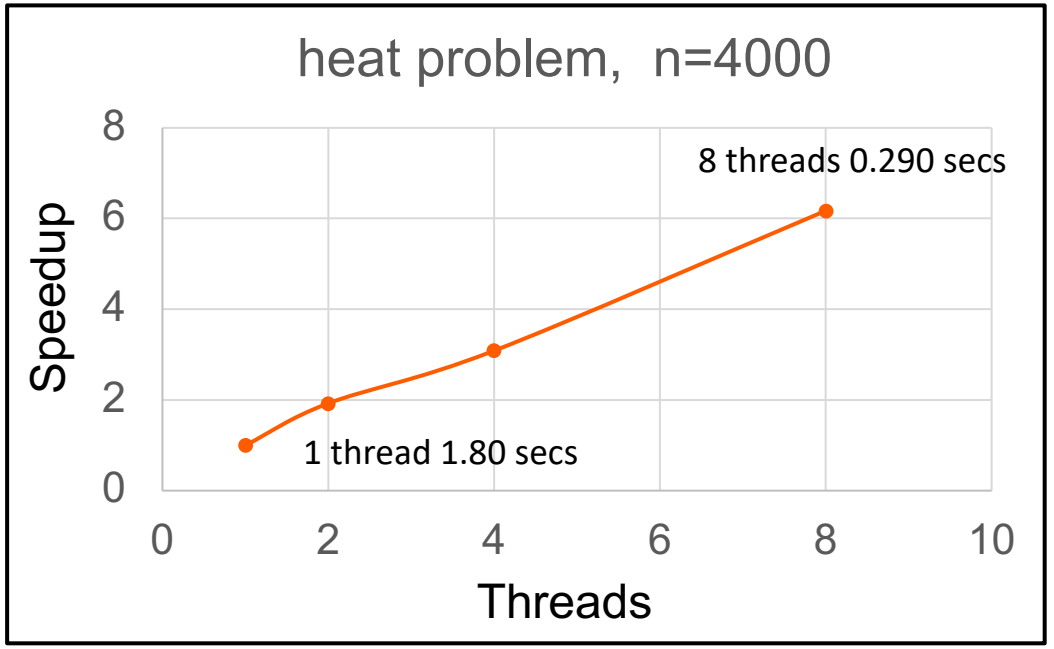
Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {

    #pragma omp parallel for collapse(2)
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]           +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
}
```



Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler
nvc –fast –fopenmp heat.c

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
   #pragma omp loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
}
```

The map() clause creates an association between an address range on the host and an address range on the GPU.

Place map() on a target construct to:

- map(tofrom:) copy data as you enter and exit the construct
- map(to:) copy data as you enter the construct
- map(from:) copy data as you exit the construct

When you map pointers between the host and the device, OpenMP remembers the addresses.

Swapped addresses on the hosts swaps addresses on the device

Heat diffusion problem: 5-point stencil code

Parallel GPU code, n=4000

```
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // malloc and initialize u_tmp and u (code not shown)

  for (int t = 0; t < nsteps; ++t) {
     #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
     #pragma omp loop
     for (int i = 0; i < n; ++i) {
       for (int j = 0; j < n; ++j) {
         u_tmp[i+j*n] =  r2 * u[i+j*n]          +
             r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
             r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
             r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
             r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
       }
      }
      // Pointer swap to get ready for next step
      tmp = u;
      u = u_tmp;
      u_tmp = tmp;
  }
}
```

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps addresses on the device

```
GPU Solver time = 1.40 secs
```

This isn't much better than the runtime for a single CPU (1.8 secs) and worse than 8 cores on a CPU (0.29 secs).

Why is the performance so bad?

```
NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc –fast –mp=gpu –gpu=cc75 heat.c
```

Parallel GPU code, n=4000

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
   #pragma omp loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
   }
   // Pointer swap to get ready for next step
   tmp = u;
   u = u_tmp;
   u_tmp = tmp;
  }
}
```

With a runtime of 1.4 secs (worse than the CPU time) we see that Data Movement dominates performance.

At the beginning of each iteration, copy $(2*N^2)*sizeof(TYPE)$ bytes to the device

We need to create a **data region** on the GPU that is distinct from the target region.

That way, we can keep the data on the device between target constructs

At the end of each iteration, copy $(2*N^2)*sizeof(TYPE)$ bytes from the device

GPU data environments distinct from kernel execution

- Create a data region on the target device (a device data environment) with two standalone directives:

```
#pragma omp target enter data map(…)
#pragma omp target exit data map(…)
```

- The **target enter data** maps variables to the device data environment.

- The **target exit data** unmaps variables from the device data environment.

- Once created, subsequent **target** regions inherit the existing data environment.

Heat diffusion problem: 5-point stencil code

Parallel GPU code, n=4000

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target
   #pragma omp loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
 #pragma omp target exit data map(from: u[0:n*n])
}
```

Create a data region and map indicated data on entry

GPU Solver time* = **0.42** secs

This is a general principal … if you want performance, you must optimize data movement.

*includes time for target enter/exit data

Exit the data region and map indicated data

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc –fast –mp=gpu –gpu=cc75 heat.c

Optimizing our 5-point stencil code

Parallel CPU and GPU results, n=4000

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)


for (int t = 0; t < nsteps; ++t) {

    #pragma omp parallel for
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]          +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap for  next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }

}
```

This is the ij loop order. Swap these loops to get the ji order.

What if I optimize the CPU code?

… just swap the i and j loops to coalesce memory accesses.

CPU

Num threads	ij loop order	ji loop order
1	1.512849	0.262260
2	0.776229	0.132453
4	0.400822	0.064220
8	0.227317	0.046586

Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores.
Nvidia HPC Toolkit compiler nvc –fast –fopenmp heat.c

All times in seconds

GPU

ij without timing enter and exit data	ij loop order	ji without timing enter and exit data	ji loop order
0.056830	0.417887	0.020123	0.358905

This GPU code used the target enter data and target exit data

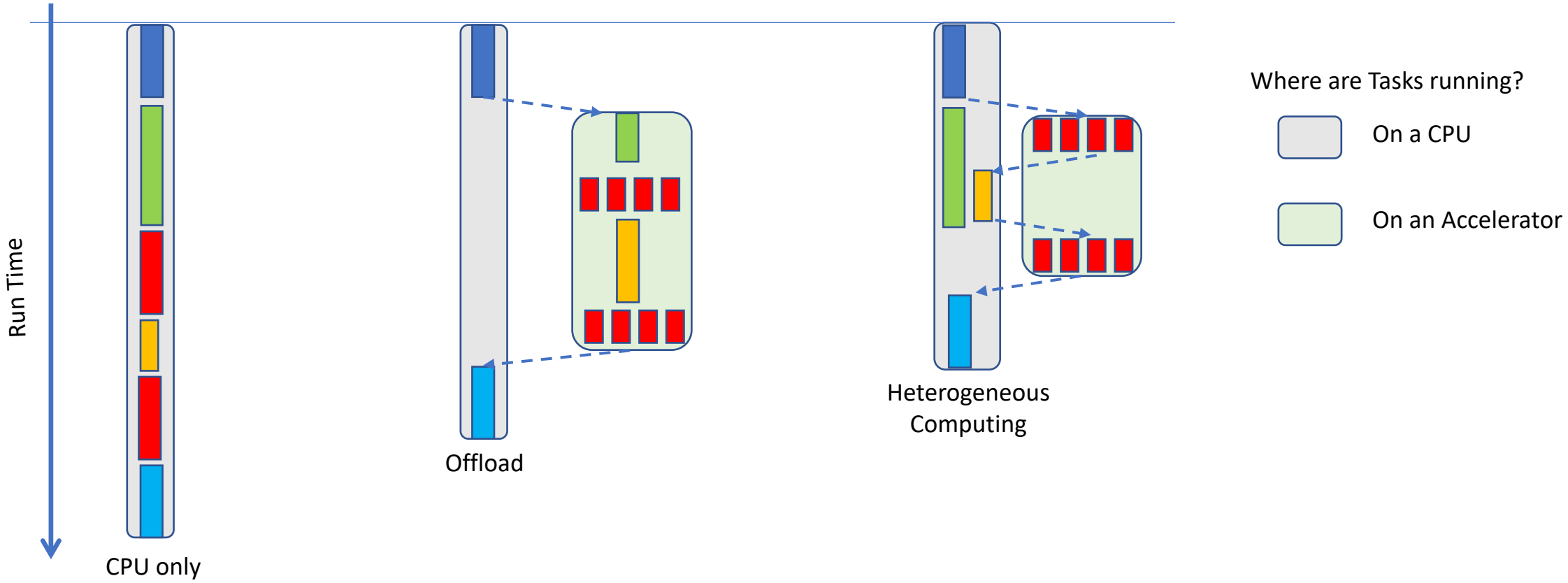
NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler. nvc -fast -mp=gpu heat.c

Why is OpenMP so important for GPU programming?

No single processor is best at everything

- The idea that you should move everything to the GPU makes no sense

- **Heterogeneous Computing**: Run sub-problems in parallel on the hardware best suited to them.



OpenMP: the only option for mixing CPU and GPU programming in a single program

```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

   // fill the arrays
   #pragma omp parallel for
   for (int i=0; i<N; i++){
      a[i] = (float)i;
      b[i] = 2.0*(float)i;
      c[i] = 0.0;
      res[i] = i + 2*i;
   }

   // add two vectors
   #pragma omp target
   #pragma omp loop
   for (int i=0; i<N; i++){
      c[i] = a[i] + b[i];
   }

   // test results
   #pragma omp parallel for reduction(+:err)
   for(int i=0;i<N;i++){
      float val = c[i] - res[i];
      val = val*val;
      if(val>TOL) err++;
   }
   printf("vectors added with %d errors\n", err);
   return 0;
}
```

In real applications, you always have CPU code and GPU code.

It's just dumb to ignore parallelism on the CPU … and handling both with a single application programming interface is powerful

Outline

- Irregular Parallelism and tasks
- Other Systems for task-level parallelism: TBB
- Non-Uniform Memory Architectures (NUMA)
- Programming your GPU with OpenMP
- Recap

The OpenMP Common Core: Most OpenMP programs only use these 21 items

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads.
void omp_set_thread_num() int omp_get_thread_num() int omp_get_num_threads()	Default number of threads and internal control variables. SPMD pattern: Create threads with a parallel region and split up the work using the number of threads and the thread ID.
double omp_get_wtime()	Speedup and Amdahl's law. False sharing and other performance issues.
setenv OMP_NUM_THREADS N	Setting the internal control variable for the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies.
reduction(op:list)	Reductions of values across a team of threads.
schedule (static [,chunk]) schedule(dynamic [,chunk])	Loop schedules, loop overheads, and load balance.
shared(list), private(list), firstprivate(list)	Data environment.
default(none)	Force explicit definition of each variable's storage attribute
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive).
#pragma omp single	Workshare with a single thread.
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.

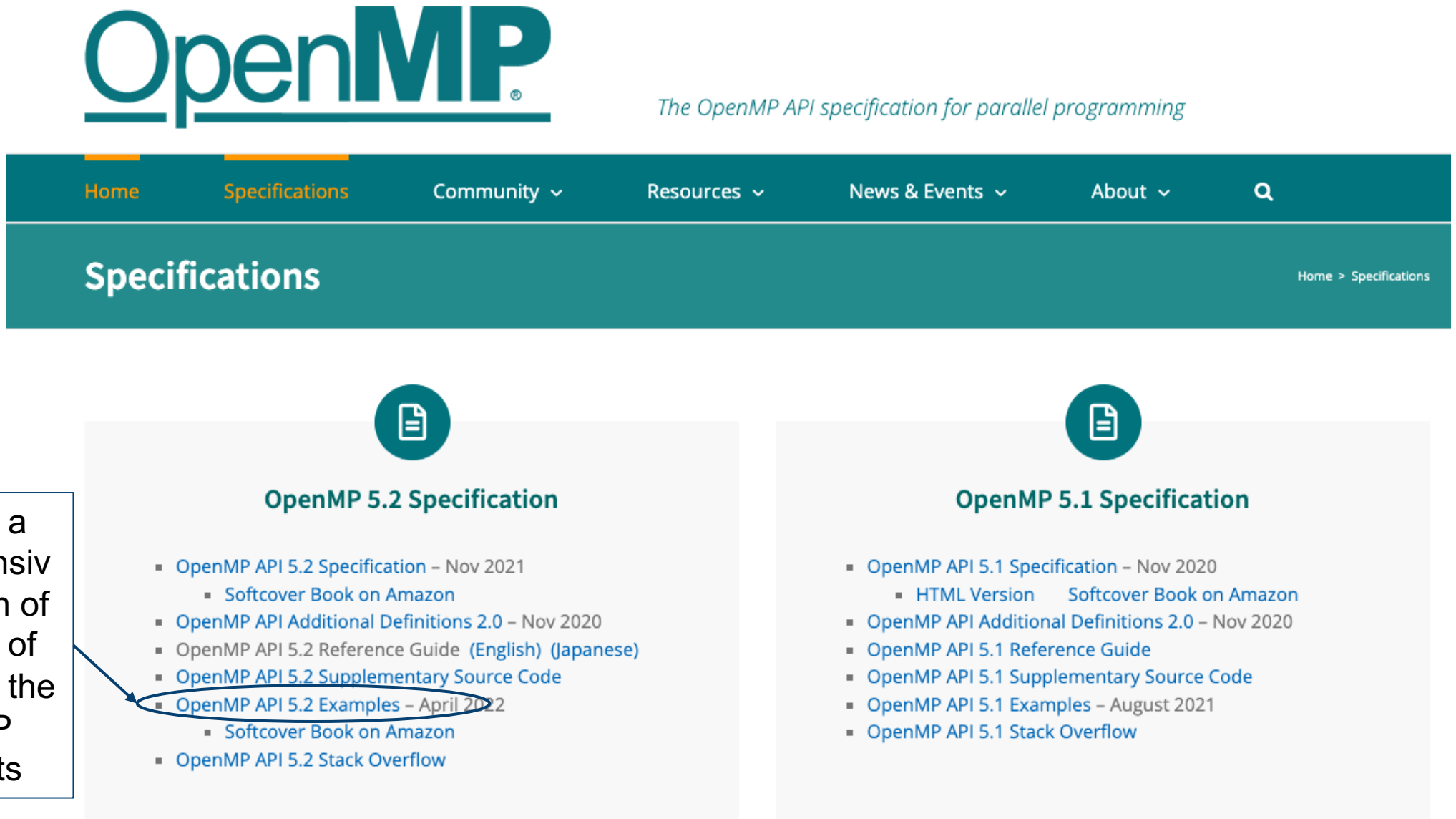
There is Much More to OpenMP than the Common Core

- Synchronization mechanisms
  - locks, synchronizing flushes and several forms of atomic
- Data environment
  - lastprivate, threadprivate, default(private|shared)
- Fine grained task control
  - dependencies, tied vs. untied tasks, task groups, task loops …
- Vectorization constructs
  - simd, uniform, simdlen, inbranch vs. nobranch, ….
- Map work onto an attached device (such as a GPU)
  - target, teams distribute parallel for, target data …
- … and much more. The OpenMP 5.0 specification is over 618 pages!!!

Don’t become overwhelmed. Master the common core and move on to other constructs when you encounter problems that require them.

Resources

- The OpenMP Architecture review Board (ARB) has a wealth of helpful resources on its web site: www.openmp.org



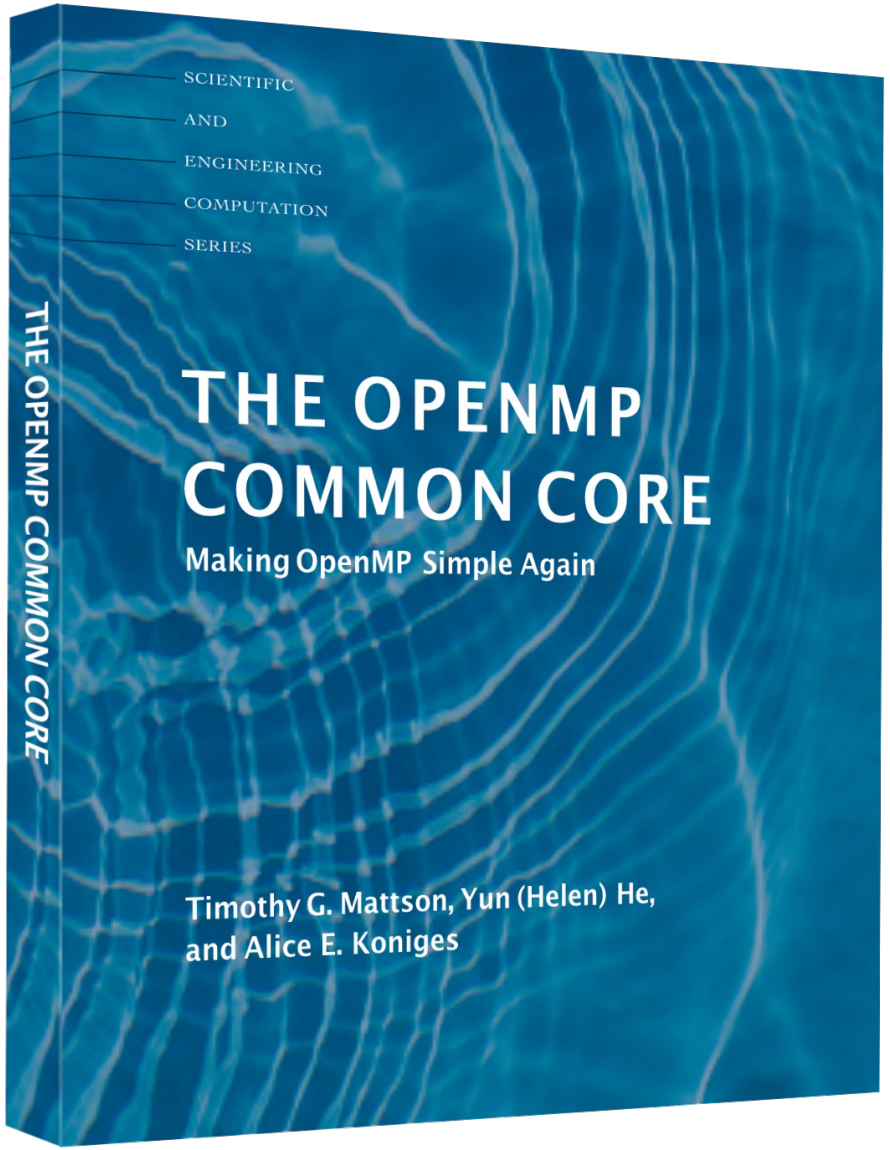
Including a comprehensive collection of examples of code using the OpenMP constructs

To learn OpenMP:

- An exciting new book that Covers the Common Core of OpenMP plus a few key features beyond the common core that people frequently use

- It's geared towards people learning OpenMP, but as one commentator put it … **everyone at any skill level should read the memory model chapters**.

- Available from MIT Press



www.ompcore.com for code samples and the Fortran supplement

Books about OpenMP

A great book that covers OpenMP features beyond OpenMP 2.5

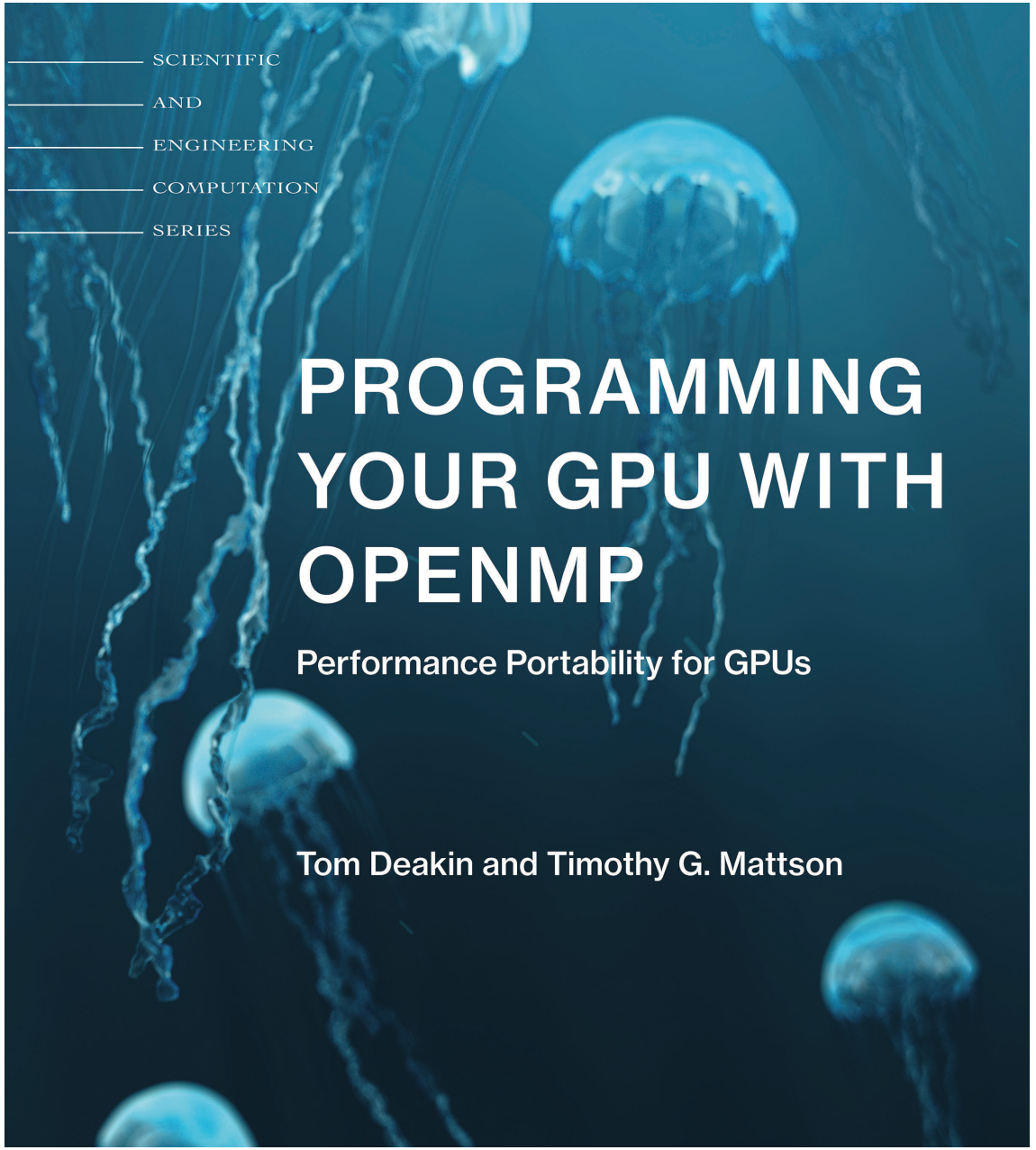


Books about OpenMP

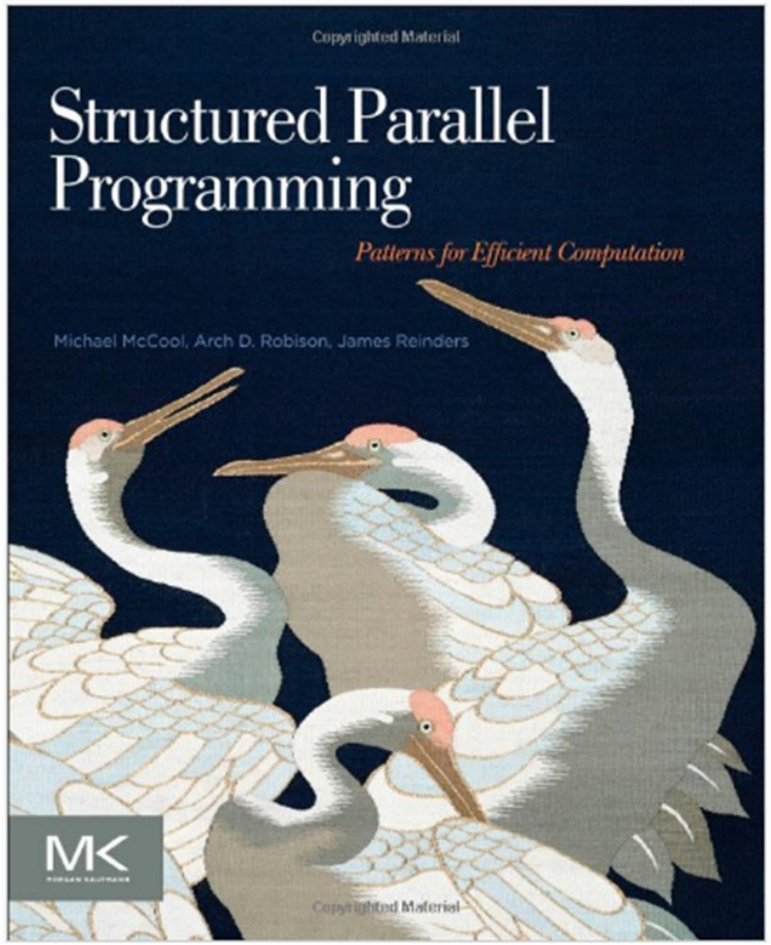
The latest book on OpenMP …

Released in November 2023.

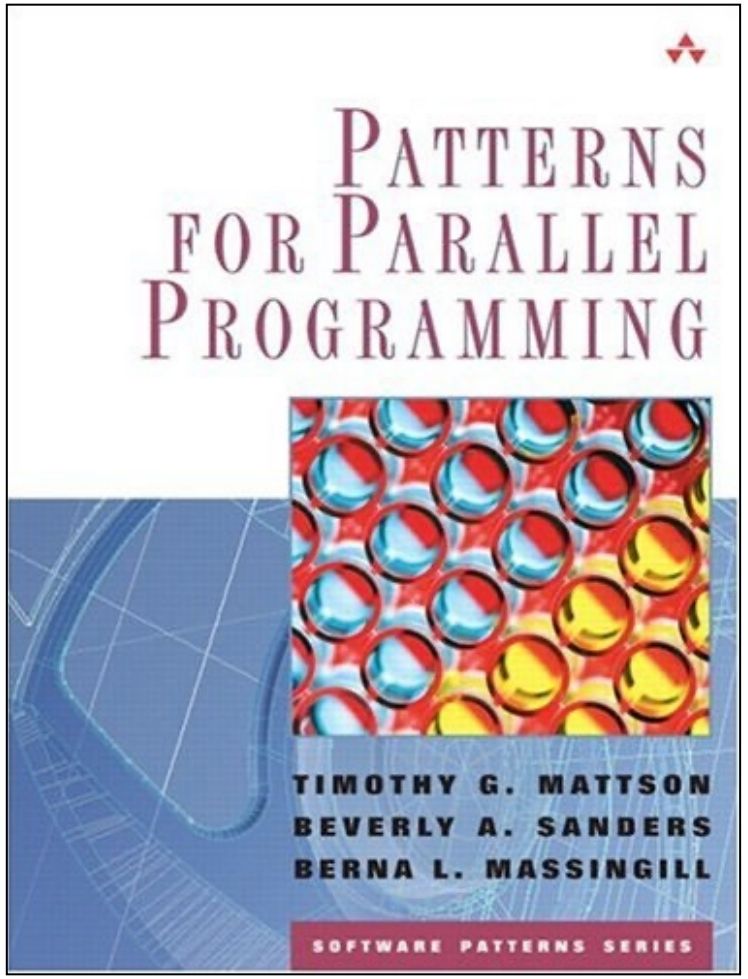
A book about how to use OpenMP to program a GPU.



Background references



A great book that explores key patterns with Cilk, TBB, OpenCL, and OpenMP (by McCool, Robison, and Reinders)



- A book about how to "think parallel" with examples in OpenMP, MPI and java